

EPFL — Spring 2026

CS-233

Introduction to Machine Learning

Dr. Mathieu Salzmann

by Arthur Herbette

Lecture notes — Thursday 19 February 2026

Contents

1	Introduction	3
1.1	Machine Learning Basics	3
	Regression vs Classification	4
1.1.1	Unsupervised Learning	4
2	Supervised Machine Learning	5
2.1	Linear Model	5
2.2	Derivatives and Gradients	9
2.2.1	Interpreting a Linear Model	15
2.2.2	Linear model with multiple outputs	16
2.2.3	Linear Classification	17
2.3	Linear Model for Classification	19
2.3.1	Loss Function and Empirical Risk	19
2.3.2	Iterative, Gradient-based optimization	24
2.4	Classification Metric	28
2.4.1	From binary to multi-class classification	31
2.4.2	Decision Boundary	32
2.4.3	Multi-class logistic regression	33
	Evaluation metrics for multi-class classification	34
2.5	k-nearest Neighbor	36
2.5.1	NN: Algorithm	37
2.5.2	k-nearest neighbors	39
2.5.3	Summary	41
2.6	Nonlinear classification	41
2.6.1	Polynomial curve fitting	42
2.7	Overfitting, Cross-validation and Regularization	45
2.8	Kernel methods	52
2.9	Kernel function	53
2.9.1	Kernelization	54
2.10	Data Representations & Processing	57
2.10.1	Data representations: Attributes	58
2.10.2	Data pre-processing	60
2.11	Deep Learning	63
2.12	Convolutional Neural Network	75
2.12.1	CNN BackPropagation	81
3	Unsupervised Linear ML	85
3.1	Dimensionality Reduction	85
3.1.1	Autoencoder	92
3.2	Clustering	94
3.2.1	Clustering: Another unsupervised learning task	94
3.3	Natural Language Processing (NLP)	102
3.3.1	Transformer	107

Chapter 1

Introduction

- Evaluation**
- Final exam: 80% of the grade
 - Past year's exams are available on Moodle
 - Two graded project milestones
 - Coding exercises build up as a small team project
 - Teams of 3 students: Register the teams by March 22nd
 - Two milestones (tentative: end of Week 8 and end of Week 13)
 - Each is worth 10% of the final grade

1.1 Machine Learning Basics

What is Machine Learning Machine learning takes the same form as how a human would learn, by **experience**. The goal is to give our **algorithm**, **data** which would output **insight** about our data.

What data? Data can be many things, it can be patient information related to births, movie reviews speech (audio file)...

An important thing to highlight is the difference between a **data set** and a **data sample**.

Definition 1. Data sample (or example, or point): is an individual observation

It can be an image, a list of attributes for one patient, etc...

Definition 2. Data set: is a collection of multiple data samples.

Which can be a collection of images, the attributes lists for multiple patients.

What insight? As said before, the goal of machine learning is to output **insight** about our data. This insight can be pretty much anything, the category depicted by an image, the influences between different musical genres.

In this course, we will mostly focus on precise insights. In particular, we will study two classes of machine learning problems:

- Regression
- Classification

Regression vs Classification

Regression predicts **continuous value(s)**¹ for a given sample.

This means that our 'guesses' typically follow an **order**. We can then measure "how accurate" our prediction is: predicting 3002g instead of 2977 is better than predicting 2500g. On the other hand, **classification** use 'classes' as prediction, categories do not follow an order: predicting 'tree' instead of 'cat' is just as wrong as predicting 'car'.

Remark 1. We could first see it as in probability/statistic. Regression is the continuous case and classification is the discrete case. Although this makes 'sense', as we will see in the next remark, regression **doesn't mean continuous**.

You are given a dataset of images of paintings. For each painting, you would like to predict the year it was created. Is this a regression or a classification task?

It is a regression task, guessing by one year is better than being off by a complete decade.

1.1.1 Unsupervised Learning

Unsupervised Data

Definition 3. In Unsupervised learning, each sample consists only of an observation, e.g., an image (without information about its content)

What classes of algorithm?

For each learning there will be different classes of algorithms, in this case (unsupervised), the learning relies on unsupervised data. The goal is rather to analyse the observed data set than to predict an output.

What we mean by this is that instead of trying to have direct information about our data (guess the animal on an image), we will more care about how does the data is divided etc... (splitting creating a graph of music genres)

naftaliharris.com

Learning Stage

Learning in this case consists of only one stage: Analyse the whole data, or transform it for further analysis. As said before the goal is not to make 'guesses'. But to gives more information about our data in order to have better insight.

¹As said right after, it is not continuous in the sense of a function f in $\mathbb{R}$. But value can be **compared**, meaning given a vector x , we can see how 'near' another vector y is to x

Chapter 2

Supervised Machine Learning

Supervised Data

Definition 4. Each sample comes with additional annotations. e.g., category label

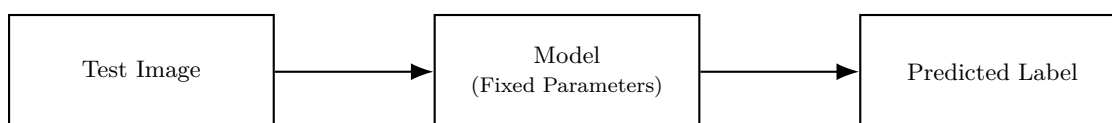
An assumption about the data that we are given is that the annotations (supervisory signal) are the insight that we seek to obtain from the data.

This is what we are more 'used to' when it comes to machine learning.

Learning Stages

This times the learning is divided into two stages:

1. **Training:** Use data with ground-truth labels to optimize model parameters (see playground.tensorflow.org)
2. **Testing:** Predict the output for a new data sample



Here we **don't** use our testing data to update the model parameters. We want to check whether or not our model **generalizes** to the new test data which might not be the case. If we go back to the example digit guessing demo; when we try to draw a number in a corner of the box, the model is not able to correctly predict. The reason for this is that the data that our model trained on had numbers only in the middle of the box → not trained to other cases.

Training set vs Test set

The training set and the test set should **always** be completely separate

Remark

Using the observations (inputs) is occasionally possible and referred to as transductive learning (we will not do it in this course).

Furthermore, we also need that our training and test samples are drawn from the **same statistical distribution**

2.1 Linear Model

Let's us start with a simple supervised learning model, the goal is to start with this as our first building block and then add layer onto it (non-linearity, etc.).

Notation

Definition 5. We denote the i^{th} **data sample** (input) in the collection of N samples as

$$x_i \in \mathbb{R}^d$$

a column vector of dimension D

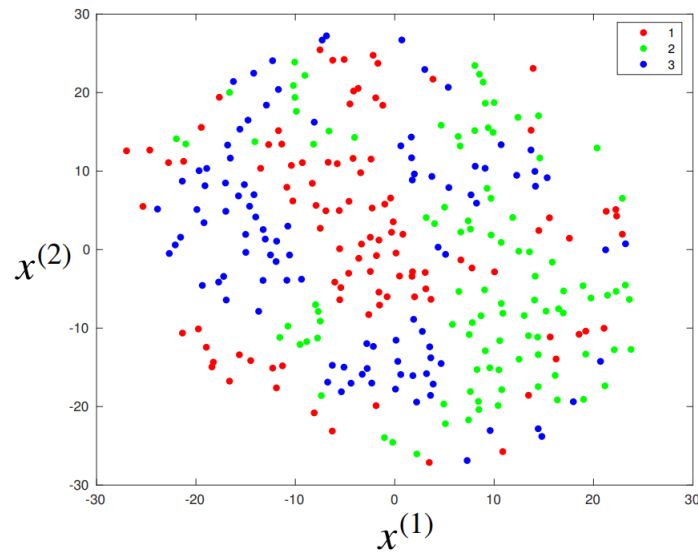
Definition 6. We denote the i^{th} label (output) in the collection of N samples as

$$y_i$$

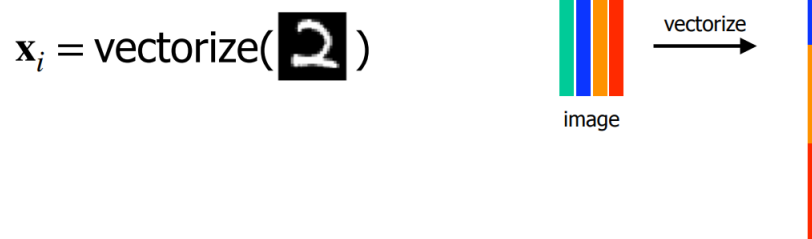
For **classification**, y_i represents the category the samples belongs to. Whereas for **regression**, y_i can be a single continuous value, or a column vector of dimension C .

Example

- In the following toy example, x_i is one 2D point, and thus $D = 2$
- y_i is a discrete value indicating the class (color), i.e., 1, 2, or 3



As another example, we can take the digit recognition example, x_i is a grayscale image. If it has height H and width W , then $D = H \cdot W$.



What we are essentially doing is transforming our image (which is a matrix in our case) to a vector by mapping each element of this message to a 'row' of the vector (this is the same as when we store a matrix in a row-major way).

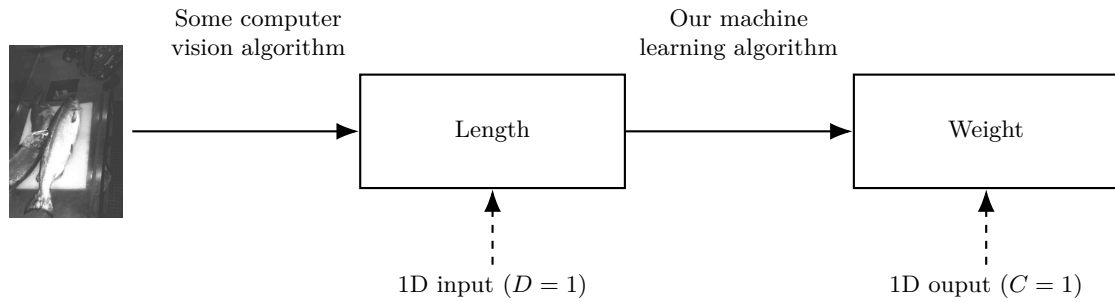
In this case of digit recognition, the output y_i is a single discrete value indicating the digit, e.g. $y_i = 2$

To start things off, let us create the **simplest** model possible:

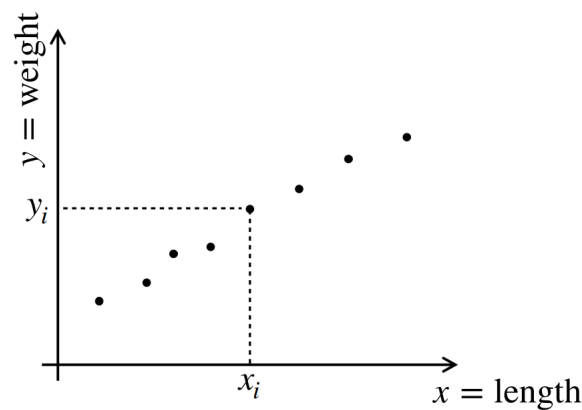
- A single input dimension, $D = 1$
- A single output dimension, $C = 1$

Our first machine learning model

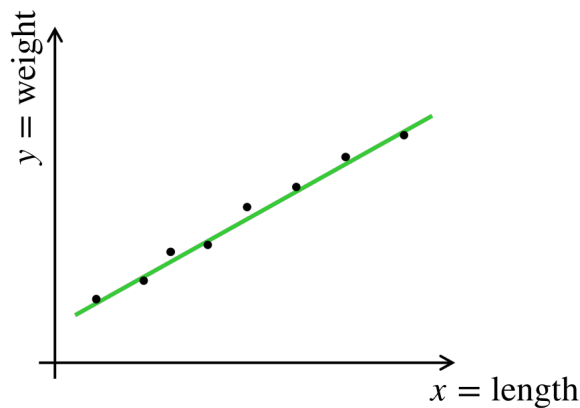
Our goal is to solve a regression task (the output is continuous). We seek to predict the weight of a fish given its length:



Given N annotated data samples $\{(x_i, y_i)\}$ with (length, weight) measurements, we can plot the data ($N = 8$ here):



We can see here that there is a kind of trend going on in this data set, we seek to approximate it with a line:



Once we have defined the line based on the N training samples, we can use it to predict the weight of any new length (again, assuming the test data follows the same distribution as the training data).

A simple parametric model: The line

Now let us think a bit more about the model, what is it made of? To be able to define a line we need 2 parameters:

- the y -intercept, we will call it $\omega^{(0)}$
- The slope $\omega^{(1)} = \frac{\Delta y}{\Delta x}$

With this two parameters, we can express any line

$$y = \omega^{(0)} + \omega^{(1)}x$$

This relation above **is** our model.

Line fitting

Now that we have our model, let us actually fit our model to some data. Given N pairs $\{(x_i, y_i)\}$, the goal is to find the line that passes through these observations. But is this possible? Not really there is always noise in our data. The goal of fitting here is to find the line that **best fits** this observation and if you remember linear algebra, we actually have some tools to do so.

1D Linear Regression training

In essence, fitting a line consists of finding the best line parameters $\omega^{(0)*}$ and $\omega^{(1)*}$ for some given data. This corresponds to the training stage:

- Given N training pairs $\{(x_i, y_i)\}$, we aim to find $(\omega^{(0)}, \omega^{(1)})$, such that the predictions of the model

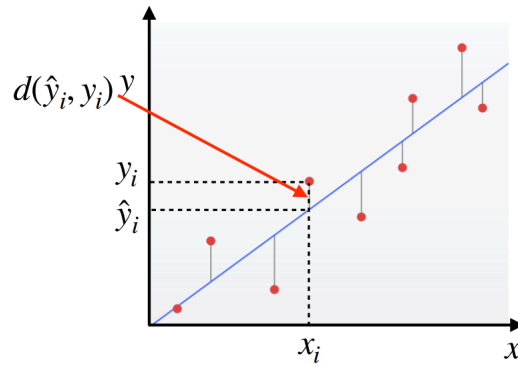
$$\hat{y}_i = \omega^{(0)} + \omega^{(1)}x_i$$

are close to the true values y_i

We then need to define a measure of closeness, or conversely an error, between y_i and $\hat{y}_i$.

As seen in linear algebra, a natural measure of the error is the Euclidean distance (also called L_2 norm)

$$d(\hat{y}_i, y_i) = \sqrt{(\hat{y}_i - y_i)^2}$$



In practice, one prefers to use the square of this distance:

$$d^2(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$$

With this definition, training can be translated as a least-square problem minimization problem:

$$(\omega^{(0)*}, \omega^{(1)*}) = \operatorname{argmin}_{\omega^{(0)}, \omega^{(1)}} \frac{1}{N} \sum_{i=1}^N d^2(\hat{y}_i, y_i)$$

Where $\hat{y}_i$ depends on $\omega^{(0)}$ and $\omega^{(1)}$. A nicer way which will be useful when using multi-dimension model, is to write our model as a matrix W such that

$$\hat{y}_i = Wx_i$$

$$x_i \in \mathbb{R}^2$$

In our case as we have y being a one dimensional vector, we don't have to write our W as a matrix, a vector is sufficient:

$$x_i = \begin{pmatrix} 1 \\ x_i \end{pmatrix} \in \mathbb{R}^2 \quad w = \begin{pmatrix} w^{(0)} \\ w^{(1)} \end{pmatrix} \in \mathbb{R}^2$$

Which gives us:

$$R(w) = \frac{1}{N} \sum_{i=1}^N (w^T x_i - y_i)^2 = \frac{1}{N} \sum_{i=1}^N (x_i^T w - y_i)^2$$

As said above, in general, an input observation x_i is not represented by a single value. If we take a grayscale image, it can be represented by an $W \cdot H$ dimensional vector

$$x_i \in \mathbb{R}^{28 \cdot 28} = \mathbb{R}^{784}$$

Plane

For one dimension we had a line, for two dimension we then have... a **plane!!**

We can define our plane as:

$$y = w^{(0)} + w^{(1)}x^{(1)} + w^{(2)}x^{(2)} = w^T \begin{pmatrix} 1 \\ x^{(1)} \\ x^{(2)} \end{pmatrix}$$

All the thing we said to a one dimension fitting is translated into two dimension: Given N noisy points $\{(x_i, y_i)\}$, where $x_i \in \mathbb{R}^2$, find the plane that best fit these observations. Of course as you could have guess, this can be generalized to higher dimensions.

- In dimension D , we can write

$$y = w^{(0)} + w^{(1)}x^{(1)} + w^{(2)}x^{(2)} + \dots + w^{(D)}x^{(D)} = w^T \begin{pmatrix} 1 \\ x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(D)} \end{pmatrix}$$

Ultimately, whatever the dimension, we can write

$$y = w^T x$$

With $x \in \mathbb{R}^{D+1}$, where the extra dimension contains a 1 to account for $w^{(0)}$.

Multi-input linear regression: Training

Because the output remains to one dimension, we can use the same least-square loss function as before, and write training as:

$$\min_w \frac{1}{N} \sum_{i=1}^N d^2(\hat{y}_i, y_i) \iff \min_w \frac{1}{N} \sum_{i=1}^N (x_i^T w - y_i)^2$$

| *Note*

This has the same form as in the one dimension input case

Solving this minimization problem involves computing its derivatives w.r.t. the model parameters (w) (by using gradient descent).

2.2 Derivatives and Gradients

Definition 7. The derivative of a function $R(w)$ of a single variable w is the rate at which R changes as w changes

Remark 2. Be careful at this definition, if you have done cs-328 you'll know that derivative as in machine learning and many other topics, use auto-differentiation which is really "different" as how analysis defined derivative. (the 2023 exam asked to derive the load instruction...)

The derivative can also be measured for an infinitesimal change in w , starting from a point $\bar{w}$, we can write it as:

$$R'(\bar{w}) = \frac{dR}{dw} = \lim_{\Delta w \rightarrow 0} \frac{R(\bar{w} + \Delta w) - R(\bar{w})}{\Delta w}$$

Minimizing a function

In machine learning, we often seek to minimize a function (e.g., the loss function). To minimize such a function, we seek a point where the derivative vanishes, i.e., we search for w^* such that

$$\frac{dR(w^*)}{dw} = 0$$

For some simple, convex functions, w^* can be obtained algebraically by solving the corresponding equation

Remark

This is referred to as closed-form solution

Remark 3. As former analysis students, we see functions as nice curves that we can draw on a graph which can be specified using usual mathematical language (for instance $f(x) = \sin x$). In context of machine learning and auto-diff, this is not always true. Being able to algebraically solve an equation is something 'rare'.

Dealing with multivariate functions

For more complicated non-convex functions, there are typically multiple, local minima

This is nice for one dimension but even our simple case of a model with one dimension had already two parameters ($w^{(0)}, w^{(1)}$). In practice, we typically have more than one parameter. To handle this, we rely on the function **gradient**. The gradient is a multi-variable generalization of the derivative.

- For a function with D parameters, the gradient is a D -dimensional vector
- Each element of this vector is the partial derivative of the function w.r.t. a single variable, i.e. the derivative of the function w.r.t. one variable while keeping the other constant

Definition 8. Gradient The gradient of a function $R(w)$ with $w \in \mathbb{R}^d$ is denoted by

$$\nabla_w R(w) = \begin{pmatrix} \frac{\partial R}{\partial w^{(1)}} \\ \frac{\partial R}{\partial w^{(2)}} \\ \vdots \\ \frac{\partial R}{\partial w^{(D)}} \end{pmatrix}$$

Where $\frac{\partial R}{\partial w^{(j)}}$ denotes the partial derivative w.r.t. variable $w^{(j)}$.

Properties of gradient

- The gradient at a point w has the direction of greatest increase of the function w or in other word points to the **steepest** 'slope' of the function at the point w

You can see with this property why the gradient may be useful when looking for minima of a function. If the gradient gives us the steepest slope of the function at a given point, when going down this slope, this means that we are going **down** the fastest way possible.

Little remark for the future

This is nice, going down the slope will make our loss function decrease. But what if it converges only to a local minima and not a global one? Doing a gradient descent at a local minima won't work or at least a 'vanilla' gradient descent. A solution to this would be gradient descent with momentum.

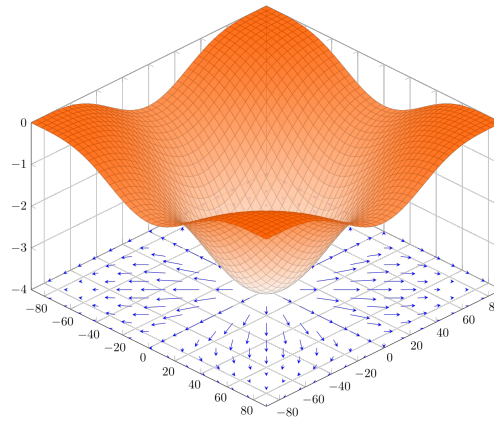


Figure 2.1: The gradient of the function $f(x, y) = (\cos^2 x + \cos^2 y)^2$ depicted as a projected vector field on the bottom plane. (wiki)

Linear regression: Solution

If we retake the least-square loss function we defined previously

$$R(w) = \frac{1}{N} \sum_{i=1}^N (x_i^T w - y_i)^2$$

This is a convex function of w , what this means is that the gradient is 0 only at the global minima. We therefore search a parameter w^* such that

$$\nabla_w R(w^*) = 0$$

Gradient Computation

Before solving our equation, let us first compute our equation $\nabla_w R(w)$. Remember first that $R(w)$ is an average over the N training samples. This means for us that we have N times the same equation for each i , which makes the gradient fairly easy to compute, we only have to compute:

$$\nabla_w (x_i^T w - y_i)^2$$

By using the chain rule, the function $(x_i^T w - y_i)^2$ has the form $f(g(w))$, with $f(g) = g^2$. We can compute its gradient as $\frac{df(g)}{dg} \nabla_w g(w)$.

The last trick that we need is the following:

Theorem 1. Given two vectors $a, b \in \mathbb{R}^D$

$$\nabla_b (a^T b) = a$$

Proof

First, recall what the dot product is:

$$a^T b = \sum_{i=1}^D a_i b_i$$

This give us by definition of the gradient:

$$\begin{aligned}\nabla_b(a^T b) &= \begin{pmatrix} \frac{\partial a_1 b_1}{\partial b_1} \\ \frac{\partial a_2 b_2}{\partial b_2} \\ \vdots \\ \frac{\partial a_D b_D}{\partial b_D} \end{pmatrix} \\ &= \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_D \end{pmatrix} = a\end{aligned}$$

Useful link

Useful rules for matrix (or vector) computations, can be found in the Matrix cookbook:

<http://www2.imm.dtu.dk/pubdb/edoc/imm3274.pdf>

Using everything we said before, the gradient of the function $R(w) = \frac{1}{N} \sum_{i=1}^N (x_i^T w - y_i)^2$ can be computed as:

$$\begin{aligned}\nabla_w R &= \frac{1}{N} \sum_{i=1}^N \frac{\partial (x_i^T w - y_i)^2}{\partial (x_i^T w - y_i)} \cdot \frac{\partial (x_i^T w - y_i)}{\partial w} \\ &= \frac{2}{N} \sum_{i=1}^N (x_i^T w - y_i) \cdot x_i\end{aligned}$$

Solution

To obtain the solution, we search for a w^* such that

$$\nabla_w R(w^*) = \frac{2}{N} \sum_{i=1}^N x_i (x_i^T w^* - y_i) = 0$$

Note

The x_i has moved to the front but this make no difference as $(x_i^T w - y_i)$ is a scalar

If we expand here what is inside the sum we have

$$x_i(x_i^T w^*) - x_i \cdot y_i$$

If we remove the constant factor $\frac{2}{N}$. Our solution has the following equation

$$\left(\sum_{i=1}^N x_i x_i^T \right) w^* = \sum_{i=1}^N x_i y_i$$

Okay, first let us look a bit more into the dimension of our equation. First $x_i \cdot x_i^T$ gives us a $(D+1) \times (D+1)$ matrix. w^* is a $(D+1) \times 1$ vector and $\sum_{i=1}^N x_i y_i$ is a $(D+1) \times 1$ vector.

$$\underbrace{\left(\sum_{i=1}^N x_i x_i^T \right)}_{(D+1) \times (D+1)} \underbrace{w^*}_{(D+1) \times 1} = \underbrace{\sum_{i=1}^N x_i y_i}_{(D+1) \times 1}$$

Matrix form

As said before, we prefer to use matrix when dealing with machine learning in general, you can also see here that we are doing a sum over product which kind of look like a matrix multiplication. Let us now group all $\{x_i\}$ and all $\{y_i\}$ in a matrix

and a vector:

$$X = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(D)} \\ 1 & x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(D)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N^{(1)} & x_N^{(2)} & \cdots & x_N^{(D)} \end{pmatrix} \in \mathbb{R}^{N \times (D+1)}$$

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^N$$

With this new notation, we can re-write the solution as:

$$X^T X w^* = X^T y$$

Sounds familiar to you? this is the definition of the least-square solution we have seen in linear algebra.

kind of proof?

A nice thing I want to put here which is not part of the course is how do we come up to the relation $X^T X w^* = X^T y$. What we are trying to do (from the beginning of this section), is to minimize the relation $\|X w^* - y\|^2$. By definition of the scalar product we have the following:

$$\begin{aligned} \|X w^* - y\|^2 &= \langle X w^* - y, X w^* - y \rangle \\ &= \langle X w^*, X w^* \rangle - \langle X w^*, y \rangle - \langle y, X w^* \rangle + \langle y, y \rangle \\ &= w^{*T} X^T X w^* - 2y^T X w^* + \|y\|^2 \end{aligned}$$

As we want to minimize, we will take its derivative, which make the $\|y\|^2$ goes away:

$$\frac{\partial}{\partial x} w^{*T} X^T X w^* - 2y^T X w^* + \|y\|^2 = 2X^T X w^* - 2X^T y$$

As we are looking to minimize we seek it to be equal to 0 which gives us:

$$X^T X w^* = X^T y$$

The **solution** of this system is given as:

$$w^* = (X^T X)^{-1} X^T y = X^\dagger y$$

Where $X^\dagger$ is known as the Moore-Penrose pseudo-inverse of X (or directly the pseudo-inverse)

kind of proof II?

So if I understood correctly what the wiki says, the pseudo-inverse $A^\dagger$ and A^+ is essentially the same. The definition of the pseudo-inverse of a matrix A A^+ satisfies all of the four criterias, known as the Moore-Penrose condition:

$$\begin{aligned} AA^+A &= A \\ A^+AA^+ &= A^+ \\ (AA^+)^* &= AA^+ \\ (A^+A)^* &= A^+A \end{aligned}$$

The last two are trivially true as we only deal with real number, note still that this is the definition of Hermitian matrix ($A^* = A$)

where the star mean the transpose conjugate (we take the conjugate of every elements and we then take the transpose of this new matrix, we therefore need that both $A^\dagger A$ and $AA^\dagger$ are hermitian), but why $X^\dagger = (X^T X)^{-1} X^T$? Let us define B such that:

$$B = (X^T X)^{-1} X^T$$

If we multiply by X :

$$XB = X(X^T X)^{-1} X^T$$

And by X again but on the other side

$$XBX = X(X^T X)^{-1} X^T X$$

Which simplify to:

$$XBX = X$$

For the second criteria, retake our B and multiply by X :

$$\begin{aligned} BX &= (X^T X)^{-1} X^T X \\ &= I \end{aligned}$$

This directly implies that:

$$\underbrace{BX}_I B = B$$

And if we had proven the two last criteria we could have concluded that $B = (X^T X)^{-1} X^T = X^\dagger$.

Test time

As mentioned before, once we have the optimal parameters w^* , we can predict $\hat{y}_t$ for any new value x_t . This predicted value is given by:

$$\hat{y}_t = \begin{pmatrix} 1 \\ x_t \end{pmatrix}^T w^* = (w^*)^T \begin{pmatrix} 1 \\ x_t \end{pmatrix}$$

Evaluation metrics for regression

We can predict but is our prediction good? To be able to measure this we need two things:

- Test data
- An evaluation metrics

We have already spoke about the test data, separate from the training data, doesn't change the parameters of the model... On the other hand, we have never seen any evaluation metrics before, a value that could tell us whether or not our model is performing well out there in the real world.

We have defined a loss function before, why not use it as the evaluation metrics? This function is 0 if the model matches perfectly with all data points, the greater the loss function the greater the offset to the correct value our model is predicting.

Mean Squared Error (MSE)

Definition 9. Mean Squared Error

$$\text{MSE} = \frac{1}{N_t} \sum_{i=1}^{N_t} (\hat{y}_i - y_i)^2$$

Where $\hat{y}_i$ is the prediction for the test sample i and y_i is the corresponding ground-truth value.

We can also use the square-root of the MSE.

Mean Absolute Error (MAE)

Definition 10. Mean Absolute Error

$$\text{MAE} = \frac{1}{N_t} \sum_{i=1}^{N_t} \|\hat{y}_i - y_i\|$$

Maybe it can be more intuitive to look at the error as a percentage:

Definition 11. Mean Absolute Percentage Error

$$\text{MAPE} = \frac{1}{N_t} \sum_{i=1}^{N_t} \left\| \frac{\hat{y}_i - y_i}{y_i} \right\|$$

2.2.1 Interpreting a Linear Model

First an important thing to check when testing our model is comparing the error value of on the training data **and** for the test data. If those two are close this means that our model effectively can generalize to other data. If for instance our model as a very low error on the training data and a high error for the test data, usually this means that the model is **overfitting**.



The other interesting thing with a linear model is that each value of the model $w^{(i)}$ directly maps to an attribute of the input. If we take the example of the UCI Wine Quality dataset with the following data: After having trained our model we have as the final RMSE:

fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6
7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5

Table 2.1: Wine quality dataset sample

- 0.65 for training data
- 0.63 for test data

Which is good for us, this means as said before that our model can generalize to other wine. Now let us see what is the parameters of our model:

Having a coefficient close to 0 means that this input is completely 'irrelevant' for our output (it is independant as in the probability sense).

Remark 4. Warning: the magnitude of a coefficient will depend on the magnitude of the corresponding feature/attribute:

- A coefficient might be very small imply to compensate for the fact that the range of the feature is very large
- This can be addressed by normalizing the data

	Coefficient
fixed acidity	0.017737
volatile acidity	-0.992560
citric acid	-0.139629
chlorides	-1.590943
free sulfur dioxide	0.005597
total sulfur dioxide	-0.003520
density	0.768590
pH	-0.4374144
sulphates	0.812888
alcohol	0.301484

Table 2.2: Wine quality parameters

2.2.2 Linear model with multiple outputs

Until now we have assumed that the output was a single value, i.e., $y_i \in \mathbb{R}$. But in practice, one may want to output multiple values for a given input, i.e., $y_i \in \mathbb{R}^C$ with $C > 1$

Model To output multiple values, the linear model cannot just rely on a vector w , because the product $w^T x_i$ yields a single value. So let us use a matrix $W \in \mathbb{R}^{(D+1) \times C}$ such that:

$$\hat{y}_i = W^T x_i = \begin{pmatrix} w_{(1)}^T \\ w_{(2)}^T \\ \vdots \\ w_{(C)}^T \end{pmatrix}$$

Since the outputs are multi-dimensional, we need to slightly modify the loss function, the multi-dimensional counterpart of the 1D least-square loss is the squared Euclidean distance. Therefore, we can write multi-output linear regression as:

$$\min_W \sum_{i=1}^N \|W^T x_i - y_i\|^2$$

Multi-output linear regression: Gradient We have our loss function but computing its gradient seems a bit more sophisticated as how we did in the single output linear regression. To compute its gradient, we make use of the following two properties:

- Norm and derivative:

$$\|a\|^2 = a^T a$$

And thus:

$$\nabla_a \|a\|^2 = 2a$$

- Matrix derivative:

$$\nabla_B (a^T B) = a$$

Thus, using the chain rule (and replacing $(W^T x_i - y_i)$ with $(X_i^T W - y_i^T)$), we obtain the gradient:

$$\nabla_W R = 2 \sum_{i=1}^N x_i (x_i^T W - y_i^T)$$

Setting it to zero means that:

$$\underbrace{\left(\sum_{i=1}^N x_i x_i^T \right)}_{(D+1) \times (D+1)} \underbrace{W^*}_{(D+1) \times C} = \underbrace{\sum_{i=1}^N x_i y_i^T}_{(D+1) \times C}$$

For the 'training' part, we can again group in a matrix $X \in \mathbb{R}^{N \times (D+1)}$, but the outputs now need to be grouped in a matrix (and not a vector) as

$$Y = \begin{pmatrix} y_1^T \\ y_2^T \\ \vdots \\ y_N^T \end{pmatrix} = \begin{pmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(C)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(C)} \\ \vdots & \vdots & \ddots & \vdots \\ y_N^{(1)} & y_N^{(2)} & \cdots & y_N^{(C)} \end{pmatrix}$$

Then, following the same strategy as before, we have:

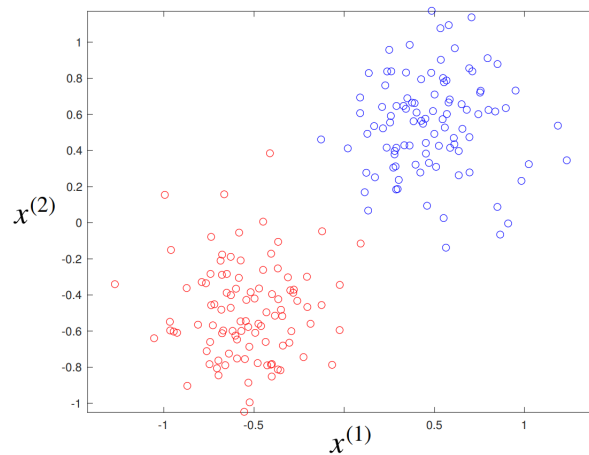
$$W^* = (X^T X)^{-1} X^T Y = X^\dagger Y$$

2.2.3 Linear Classification

The goal for now will be the same as before, however this time instead of having to output a continuous vector, the output will be a class (which is represented by an integer).

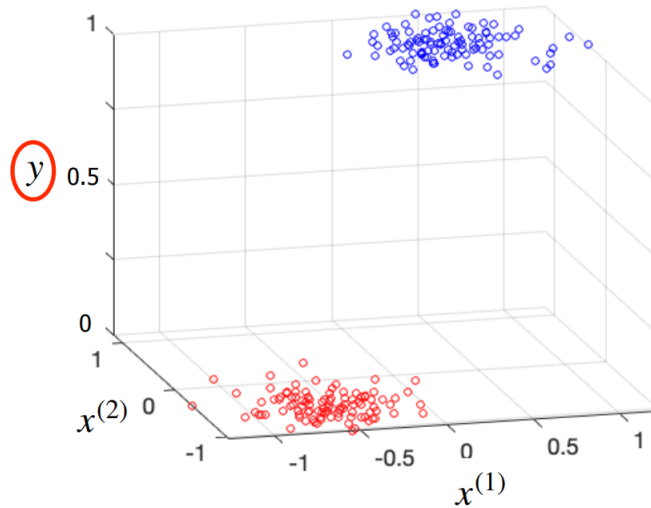
Binary Classification The easiest classification model we can make is a binary classification one. The output of our model is either 0 or 1.

If we take the following data sets:



- We have 2 dimensions input
- The label $y \in \{-1, 1\}$, or $y \in \{0, 1\}$, is the 1D output
- The sample with label 1 are called **positive** samples
- The sample with label -1 (or 0) are called **negative** samples

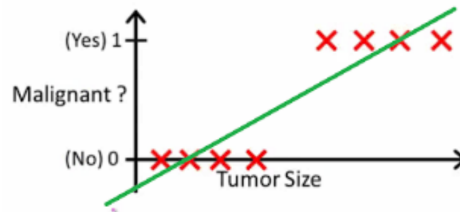
But if I have this data in a 2D space, I can also map it to a 3D space by adding the y label into our space. The label acts as the output dimension, just as in a regression problem:



This means that we can now look at this problem as a regression task, one can tackle classification as a regression task!

Example (from Andrew Ng's course)

The goal is to classify a tumor as benign vs malignant, here's the data:



But we can still create a model on this data, a linear model which can predict the label as:

$$\hat{y} = w^{(0)} + w^{(1)}x$$

If $\hat{y}$ is small there is a high probability of being not malignant and vis versa.

To fit the linear model to the training data, the same strategy as for the linear regression can be used:

Given N pairs $\{(x_i, y_i)\}$, we can use the least-square loss to write:

$$\min_W \sum_{i=1}^N (x_i^T W - y_i)^2$$

Because it has exactly the same formulation as before, the solution is exactly the same, i.e.

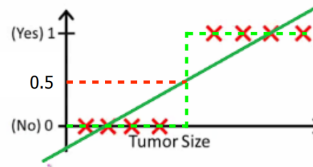
$$w^* = (X^T X)^{-1} X^T y$$

Because we still need to output a **label**, we will transform our continuous value by using a threshold, for instance:

$$\text{label} = \begin{cases} 1 & \text{if } \hat{y}_t \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Where $\hat{y}_t = x_t^T w^*$.

This gives us for instance as a decision boundary



Remark

A general rule when using least-square binary classification is that: the boundary between our two classes is of dimension D . When we had data of dimension 1 the decision boundary was a point (i.e. 0.5 above). If our data are in two dimensions, the boundary will be a line.

2.3 Linear Model for Classification

As said in the previous lecture, a simple linear model, represents an hyperplane. In dimension D , we can write:

$$y = w^{(0)} + w^{(1)}x^{(1)} + \dots + w^{(D)}x^{(D)} = w^T \begin{pmatrix} 1 \\ x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(D)} \end{pmatrix}$$

Ultimately, whatever the input dimension, we can write:

$$y = w^T x$$

with $x \in \mathbb{R}^{D+1}$, where the extra dimension contains a 1 to account for $w^{(0)}$.

2.3.1 Loss Function and Empirical Risk

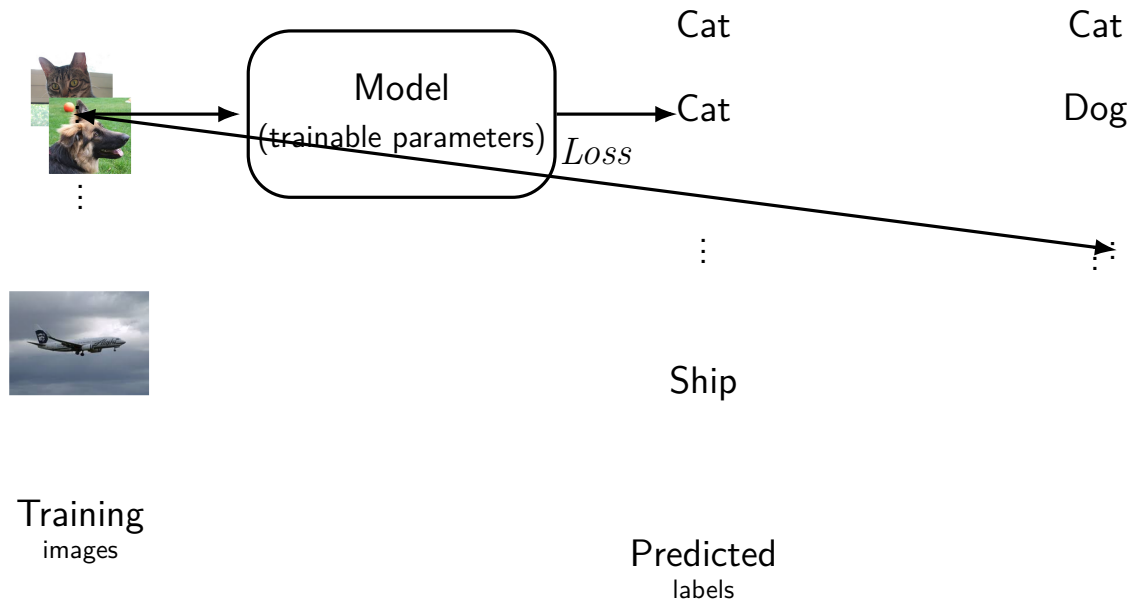
Let us have an interlude in our classification goal in order to get more tools to tackle this problem. Note that those tools here (loss functions and empirical risks) are general machine learning concepts used elsewhere.

Loss function

Definition 12. The loss function $l(\hat{y}_i, y_i)$ computes an error value between the prediction and the true value

This is a general machine learning concept, this is not used only for linear regression, we will see different loss functions in the upcoming lectures.

The goal of the loss function is to **measure** how **wrong** our prediction is. Meaning that if we were to train a model in order to guess whether it is a dog, cat, ship, plane, etc.. The loss function will tell us how wrong are we. (notice that even with classification model we will need to have a loss function (even is it will measure a 'yes' or 'no' type question)).



Loss function: Examples

So far, we have already seen quite a lot of loss functions for linear regression:

- Single output dimension: $l(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$
- Multiple output dimension: $l(\hat{y}_i, y_i) = \|\hat{y}_i - y_i\|^2$

In principle, other loss function could be used for regression, for example, using the MAE evaluation metric

$$l(\hat{y}_i, y_i) = \|\hat{y}_i - y_i\|$$

As we will see later today and in future lectures, other loss function can be used for classification.

Empirical risk

Definition 13. Empirical Risk

Given N training sample $\{(x_i, y_i)\}$, the empirical risk is defined as:

$$R(\{x_i\}, \{y_i\}, w) = \frac{1}{N} \sum_{i=1}^N l(\hat{y}_i, y_i)$$

Where $\{x_i\}$, and $\{y_i\}$ are the sets of training inputs and labels, respectively.

When we are training our model, the goal is to minimize the empirical risk – find the parameters w (e.g., $w^{(0)}, w^{(1)}$) that minimize the empirical risk.

Remark Note that the risk depends on w via $\hat{y}_i$.

Minimizing the risk

This is expressed as the optimization problem:

$$\min_w \frac{1}{N} \sum_{i=1}^N l(\hat{y}_i, y_i)$$

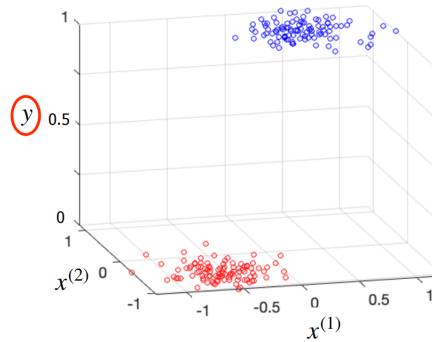
We can also write that the best parameters as a solution to the following equation:

$$w^* = \operatorname{argmin}_w \frac{1}{N} \sum_{i=1}^N l(\hat{y}_i, y_i)$$

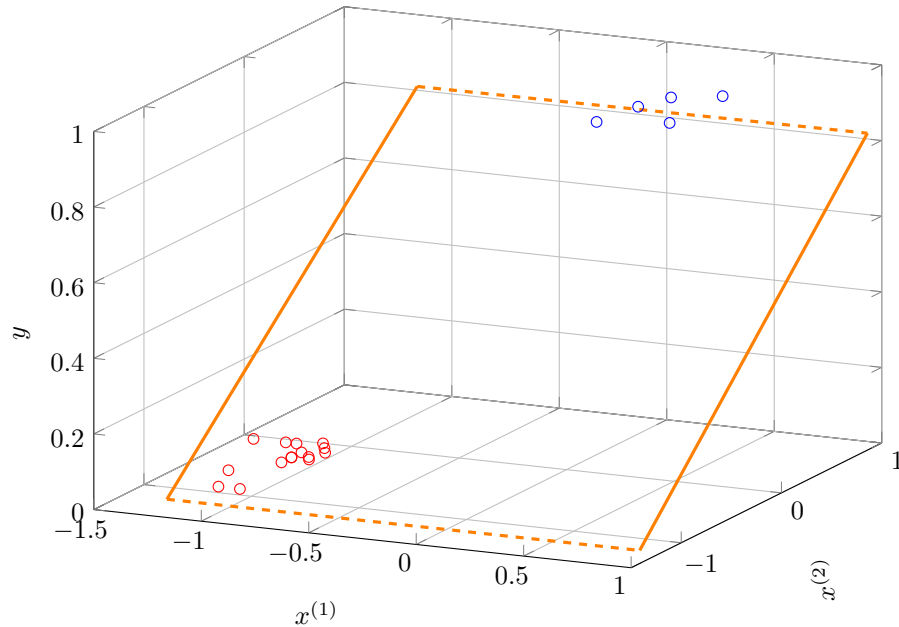
This is what we have done when training our linear regressor. Our least-square minimization problem is one example of empirical risk minimization, using the square loss.

From regression to classification

As said before, how we created classification model was directly based from a regression task. We took our training data and label, and mapped them into A $D + 1$ dimension. For instance given data point x with $x^{(1)}$ and $x^{(2)}$ and two classes 0 and 1, we would output our data as:



Then we would train our regression model with our data which will give us a plane:



The linear model encodes the plane that passes through the samples (orange plane). To turn this into a classification model, we need to find the **decision boundary**

Remark 5. I just want to emphasize what is going on here. We first try to model the data using a linear model of the form

$$f(x) = w^T x + b.$$

The parameters of this model are chosen to minimize a certain **loss function**, which measures how well the model fits the training data.

After this, the goal is to create a classification model from the model we just built. In binary classification, the decision boundary is defined by

$$w^T x + b = 0.$$

In general, for data in D dimensions, this equation defines a $(D-1)$ -dimensional hyperplane. In particular, when $D = 2$, the decision boundary is a straight line.

Adding non-linearity

The output of the linear model $w^T x$ is a continuous value. However classification model output a label (a discrete value). We therefore need to 'cast' our value before output:

$$y = \begin{cases} 1 & \text{if } w^T x \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Step function: Drawback

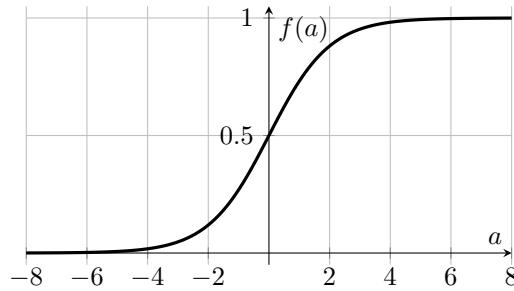
However the gradient of the step function is not continuous, this means that using gradient descent won't work very well, optimizing become pretty hard.

Remark

The perceptron algorithm can nonetheless handle the two-class case, but we will not cover it in this course.

The better way is to use a 'smoother' step function: the logistic sigmoid function:

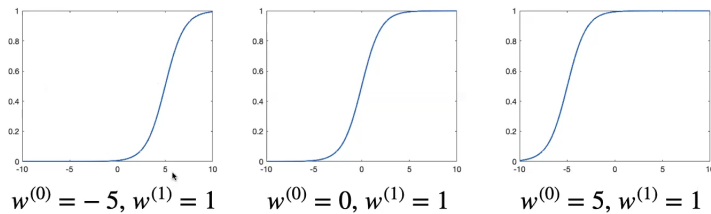
$$f(a) = \frac{1}{1 + \exp(-a)}$$



With a 1 D input x , applying the logistic function to the output of a linear model gives a prediction:

$$\hat{y} = \frac{1}{1 + \exp(-w^{(0)} - w^{(1)}x)}$$

Now an interesting thing to see is the effect of each parameters on our $\hat{y}$:

**Probabilistic interpretation**

In the binary case, with D dimensional inputs, we can write the prediction of the model as:

$$\hat{y} = \sigma(w^T x) = \frac{1}{1 + \exp(-w^T x)}$$

Such a prediction can be interpreted as the probability that x belongs to the positive class (or as a score for the positive class). The probability to belong the negative class is then computed as $1 - \hat{y}$.

Logistic regression

Definition 14. The model whose prediction is defined as

$$\hat{y} = \sigma(w^T x) = \frac{1}{1 + \exp(-w^T x)}$$

Is call a **logistic regression**

Remark

Note that this name is slightly misleading, as this is truly a classification model, not a regression one

Logistic regression: Training

At its heart, logistic regression still use a linear model, but followed by a non-linearity to better encode the underlying binary classification problem.

Given N training samples, to learn the parameters w of the logistic regression model, we need to define a loss function. Instead of using the L_2 loss as in the least-square case, this is achieved by making use of the probabilistic interpretation.

Remark

- Because knowledge of probabilities is not required for this course, the derivation of the loss function that follows will be a bit hand-wavy
- Nevertheless, this loss remains intuitive: The loss is such that it aims to find the parameters that maximize the probability of the correct class for each training sample

Specifically, the logistic regression loss is derived by first writing the following likelihood estimator:

$$p(y|w) = \prod_{i=1}^N \hat{y}_i^{y_i} \cdot (1 - \hat{y}_i)^{1-y_i}$$

Because, a product of terms between 0 and 1 can be unstable to optimize, we take the logarithm of this likelihood and take its negative to define an empirical risk to minimize, written as

$$R(w) = - \sum_{i=1}^N (y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i))$$

This loss function is known as the cross-entropy (AICC II mentioned?)

Since y_i is either 0 or 1, the cross-entropy can be re-written as:

$$R(w) = - \sum_{i \in \text{positive samples}} \ln(\hat{y}_i) - \sum_{i \in \text{negative sample}} \ln(1 - \hat{y}_i)$$

A very nice thing here for us is that, the **cross-entropy is a convex function**. But in this case, its minimization has no closed-form solution. This means that we will need to use an approximation: **gradient descent!!**

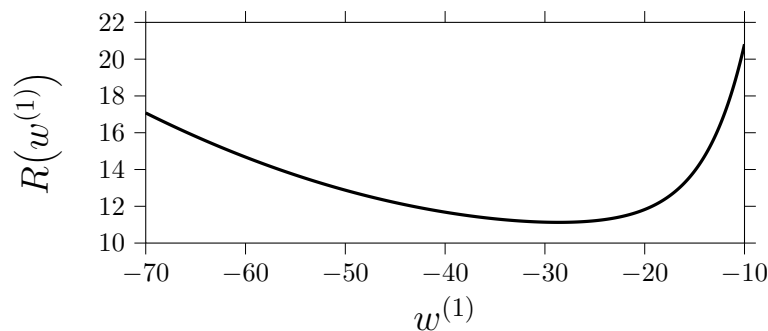


Figure 2.2: Cross entropy

Probabilistic Derivation

As we have now all did probability and statistics, we can still build it ourself. First, for us to train our model via the maximum likelihood estimator, we need to understand from where it comes from. We will do it as follow:

- Choosing a probabilistic model for $y|x$
- Writing the likelihood of the observed labels
- Maximizing it (equivalently minimizing the negative log-likelihood)

First, in logistic regression we have $\theta = (w, b)$. Where $w \in \mathbb{R}^D$ are the weights and b is the bias. (we can also see $\theta \in \mathbb{R}^{D+1}$).

Secondly, logistic regression assumes a probability model for the labels. For each

data point x we have that:

$$P(Y = 1|x, \theta) = \sigma(\theta^T x)$$

What this says to us: given x , the label y follows a **bernoulli distribution**:

$$Y|x \sim \text{Bernoulli}(\sigma(\theta^T x))$$

Then given that we can compute the likelihood estimator as we used to. First defined it:

$$\begin{aligned} \mathcal{L}(\theta) &= P(y_1, y_2, \dots, y_n | x_1, x_2, \dots, x_n, \theta) \\ &= \prod_{i=1}^n P(y_i | x_i, \theta) \end{aligned}$$

If we develop this we get:

$$\mathcal{L}(\theta) = \prod_{i=1}^n \sigma(\theta^T x_i)^{y_i} (1 - \sigma(\theta^T x_i))^{1-y_i}$$

Now that we have defined it we seek to find the θ^* such that:

$$\theta^* = \arg \max_{\theta} \mathcal{L}(\theta)$$

And as we did in statistics, we can take the log of this function and computing the maximum on the log which is 'nicer'. We therefore try to maximize:

$$\sum_{i=1}^n (y_i \log \sigma(\theta^T x_i) + (1 - y_i) \log (1 - \sigma(\theta^T x_i)))$$

Intuition

- The data already exists
- We assume a **probabilistic model**
- The model has parameter θ
- We ask:

For which θ would this observed data be the most likely

2.3.2 Iterative, Gradient-based optimization

The principle of gradient-based optimization and more generally iterative optimization is to use the fact that we are able to found a better solution based on a previous one. This means that if we iter on this 'way' a lot of time we will have better(better(better(⋯(base)))) which will give us not necessarily the best solution but a good enough solution.

Minimizing a function

With 1D function for which the solution cannot be computed in closed form, one can nonetheless use the derivative of this function to minimize it.

- Recall that it represents the slope at a given point, and thus indicates in which direction to move to reach a lower function value

Therefore the algorithm can be define pretty easily:

1. Initialize w_0 (randomly)
2. While not converged do
 - (a) Update $w_k \leftarrow w_{k-1} - \nu \frac{dT(w_{k-1})}{dw}$

The ν defines the step size of each iteration. (often called the learning rate)

Convergence criteria

This is nice but we still need potential convergence criteria to be sure that our algorithm works effectively:

- Change in function values less than a given threshold: $\|R(w_{k-1} - R(w_k))\| < \delta_R$

- Change in parameter value less than a given threshold: $\|w_{k-1} - w_k\| < \delta_w$
- Maximum number of iterations reached, Meaning that we have no guarantee to have reached the minimum

Remark 6. In theory, one would need the function to have Lipschitz continuous gradient in order to have stable steps. What this means is we will assume:

$$\|\nabla R(x) - \nabla R(y)\| \leq L\|x - y\|$$

The rule mentioned above are more practical criteria. In practice we don't want to compute exact optimality, we so care more about:

- change in objective
- change in parameters

Our current criteria work well **because** the function is convex + smooth

When minimizing we have to be careful about:

<i>Influence of ν</i>	Too large steps can lead our algorithm to start jumping around the 'solution' and oscillates till infinity. We are not able to have a magic formula to have a good η , each η is problem-dependent.
<i>Influence of the initial w_0</i>	Assuming that our function is convex, the algorithm will always converge to the same point. Choosing a good w_0 will still converge to the same point, but faster .

Minimizing a non-convex function

As in convex function, we can still use the same iterative algorithm as before. However this time. The output of our algorithm depends now on w_0 . We can be stuck at local minimum and/or at saddle point.

<i>Remark</i>	Note that this is problem-dependent and the values used here are very different from those used in the convex sample before.
---------------	--

In practice, we have more than one parameter, i.e. $w \in \mathbb{R}^{D+1}$, and we thus use the function gradient:

$$\nabla_w R(w) = \begin{pmatrix} \frac{\partial R}{\partial w^{(0)}} \\ \frac{\partial R}{\partial w^{(1)}} \\ \frac{\partial R}{\partial w^{(2)}} \\ \vdots \\ \frac{\partial R}{\partial w^{(D)}} \end{pmatrix}$$

Where $\frac{\partial R}{\partial w^{(j)}}$ denotes the partial derivative w.r.t. variable $w^{(j)}$

Recap: Properties of gradient

- The gradient at a point w has the direction of greatest increase of the function at w
- Its magnitude is the rate of increase in that direction

Gradient descent on multivariate function

The algorithm stays the same but we will use the gradient instead:

1. Initialize w_0 (e.g., randomly)
2. While not converged:
 - (a) Update $w_k \leftarrow w_{k-1} - \eta \nabla_w R(w_{k-1})$

The learning rate η has the same effect as in the 1D case.

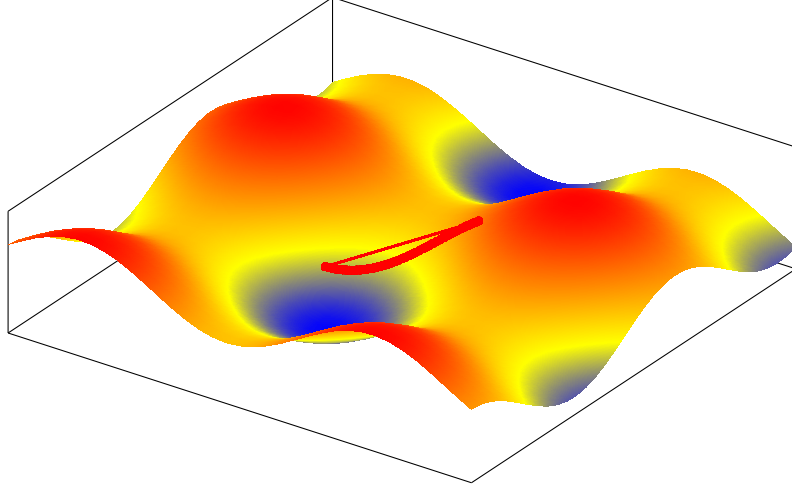
Convergence can again be measured by:

- Thresholding the change in function value
- Thresholding the change in parameters, e.g. $\|w_{j-1} - w_k\| < \delta_w$

Minimizing a non-convex function Example: sin, cos based function of 2 variables

Let our function be:

$$R(w) = \sin(w^{(0)}) + \cos(w^{(1)})$$



As in the 1D cases the same issues can arise:

- Different learning rate can yield different results
- Different Initializations can yield different results

Back to Logistic Regression

With binary ground-truth label y_i , the cross entropy can be written as:

$$\begin{aligned} R(w) &= \sum_{i \in \text{positive samples}} \ln(\hat{y}_i) - \sum_{i \in \text{negative sample}} \ln(1 - \hat{y}_i) \\ &= \sum_{i \in \text{positive samples}} \ln \sigma(w^T x_i) - \sum_{i \in \text{negative sample}} \ln(1 - \sigma(w^T x_i)) \end{aligned}$$

To minimize it via gradient descent, we need to compute its gradient. Using the chain rule, and after some derivation, we have:

$$\nabla_w \left(- \sum_{i \in \text{positive}} \ln \sigma(w^T x_i) \right) = \sum_{i \in \text{positive}} (\hat{y}_i - 1) x_i = \sum_{i \in \text{positive}} (\hat{y}_i - y_i) x_i$$

Similarly, for the negative examples, we have:

$$\nabla_w \left(- \sum_{i \in \text{negative}} \ln(1 - \sigma(w^T x)) \right) = \sum_{i \in \text{negative}} \hat{y}_i x_i = \sum_{i \in \text{negative}} (\hat{y}_i - y_i) x_i$$

Finally, this can be put back together to obtain:

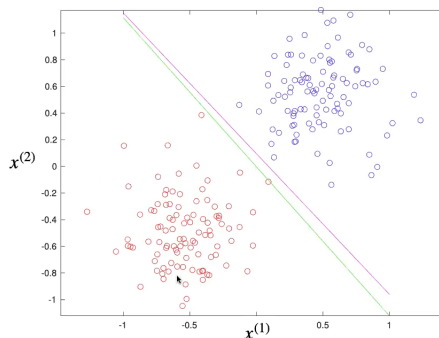
$$\nabla_w R(w) = \sum_{i=1}^N (\hat{y}_i - y_i) x_i$$

Where y_i is the true label for sample i , i.e. 1 for positive sample and 0 for negative samples

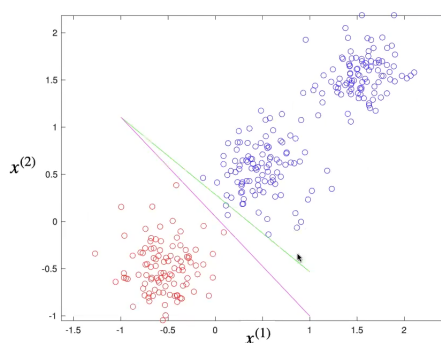
Intuitively, the gradient is given by the error of the current prediction $(\hat{y}_i - y_i)$, multiplied by the input.

Example

If we take the previous example and compare the least-square classification (in green) and the logistic regression in magenta, we get:



But now, with additional samples, the boundary remains correct:



Remark 7. As with linear regression, the learned coefficients of logistic regression can also be analyzed, see a discussion of this at <https://www.displayr.com/how-to-interpret-logistic-regression-coefficients/> (this is not part of the course)

Linear models

The real difference between these models is the loss function.

- The choice of loss function has an impact on the resulting algorithm
- Let's have a look at the intuition behind SVM

Margin

To have a better intuition of the SVM we first need to introduce the notion of **margin**:

Definition 15. The distance between the decision boundary and the nearest sample is called **margin**

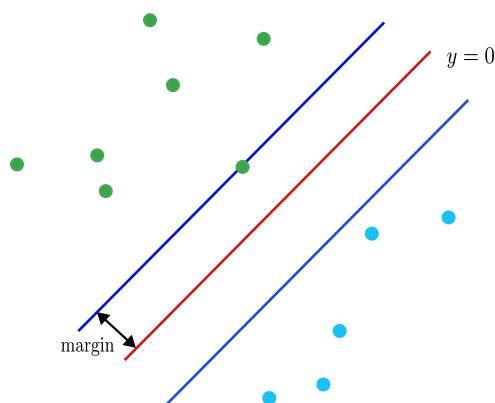


Figure 2.3: margin

Intuitively, a good decision boundary should maximize the margin. Such a boundary is determined by a subset of the data points, named support vectors (indicated with circles) when we have two classes what this essentially means is that our boundary will be centered between both our classes.

Support Vector Machine The SVM classifier is a binary classifier based on the linear model as before:

$$\hat{y} = w^T x$$

It aims to maximize the margin while enforcing a correct classification for each training samples. This is formulated as a convex optimization problem that can be solved to optimality with standard iterative solvers

In the presence of additional samples, the decision boundary is unchanged. The goal of this model is not to have a '*weighted mean*' it's more finding the **best boundary** such that the score is minimize. The only vector that truly matter are the one close to the decision boundary.

Loss function Ultimately, training an SVM can be written as:

$$\min_{w^{(0)}, \tilde{w}} \frac{1}{2C} \|\tilde{w}\|^2 + \sum_{i=1}^N \max\left(0, 1 - y_i \left(w^{(0)} + \tilde{w}^T x_i\right)\right)$$

- The term $\|\tilde{w}\|^2$ corresponds to the inverse of the margin (distance between the decision boundary and the nearest sample); minimizing it seeks to maximize the margin
- The term:

$$\max\left(0, 1 - y_i \left(w^{(0)} + \tilde{w}^T x_i\right)\right)$$

is referred to as the **hinge loss** and encourages correctly classifying each sample.

2.4 Classification Metric

The goal now is to understand what is our model **behavior**. If it guesses correctly, if he is wrong, what type of wrong, etc...

Classification evaluation

Definition 16. We can first define the **confusion matrix** in the binary case (e.g., spam vs non-spam email):

	Pos	Neg
Yes	TP	FP
No	FN	TN
	P = TP + FN	N = FP + TN

Where:

- TP: True positive (number of samples correctly classified as positive)
- FP: False positive (number of samples incorrectly classified as positive)
- TN: True negative (number of sample correctly classified as negative)
- FN: False negative (number of sample incorrectly classified as negative)

Using this confusion matrix, we can study further the result of our model.

Example

Let us use a numerical example for a dataset with 500 positive and 500 negative samples:

	Pos	Neg
Yes	200	100
No	300	400
	P = 500	N = 500

Given this matrix, what we can first compute is the **accuracy**:

$$\text{Acc} = \frac{TP + TN}{P + N}$$

In our example we have:

$$\text{Acc} = \frac{200 + 400}{500 + 500} = 0.6$$

But is it enough? Not really let two classifiers:

	Pos	Neg
Yes	200	100
No	300	400
	P = 500	N = 500

	Pos	Neg
Yes	400	300
No	100	200
	P = 500	N = 500

The two classifiers (matrices) above yield the same accuracy but their behavior is very different:

- Left: Weak positive recognition rate, but strong negative recognition rate
- Right: Strong positive recognition rate, but weak negative recognition rate

Another classification metrics we can compute is called **Precision**:

Definition 17. Precision

The percentage of samples classified as positives that are truly positives:

$$\text{Precision} = \frac{TP}{TP + FP}$$

In our previous example, this would give us:

$$\text{Precision} = \frac{200}{200 + 100} = 0.667$$

(ekip)

Another metric is called **Recall**:

Definition 18. Recall

the percentage of positive samples that are correctly classified as positives:

$$\text{Recall} = \frac{TP}{P}$$

In our example:

$$\text{Recall} = \frac{200}{200 + 300} = 0.4$$

But still even with those metrics, we can still have different behavior and same metrics: let us take the two followings:

	Pos	Neg
Yes	200	100
No	300	400
	P = 500	N = 500

	Pos	Neg
Yes	200	100
No	300	0
	P = 500	N = 100

In this case, the two classifiers (matrices) above yield the same precision and recall (0.667 and 0.4) (the datasets are different). But in this case their behavior is very different. Accuracy in this case would be able to show the different behavior. The goal of this example is to show that the more metrics you have the better your interpretation of the model you will get

Other metrics

Definition 19. False positive rate

Percentage of negative samples that are incorrectly classified as positive:

$$\text{FP rate} = \frac{FP}{N}$$

Definition 20. F1 score

Single number that combines precision and recall:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

ROC curve

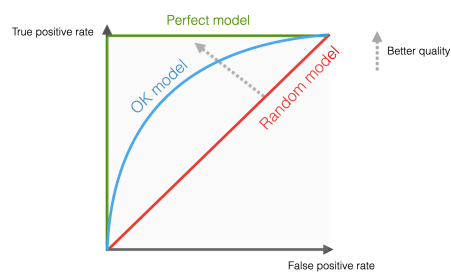
Many classifiers output a score/confidence for their prediction (e.g., $\hat{y}$ is a continuous value between 0 and 1 in logistic regression)

The decision between positive and negative can be computed by thresholding this score:

$$\text{label}_i = 1 \text{ if } \hat{y}_i \geq \tau$$

Where τ is a given threshold.

The Receiver Operating Characteristic (ROC) curve plots the true positive rate (recall) as a function of the false positive rate, obtained by varying the score threshold.



The Area Under the ROC Curve (AUC) can act as a metric:

- The maximum AUC is 1
- Random predictions give an AUC of 0.5

A little example here would be to set our threshold $\tau = 0$, everything gets called positive, in this case, let us compute the recall (true positive rate) and the false positive rate

$$\text{recall} = \frac{TP}{TP + FN}$$

FN = 0 as false negative are instance that are actually positive but predicted negative. But as we put everything to positive, we cannot have any hence we get

$$\frac{TP}{TP} = 1$$

For the false positive rate define as

$$FPR = \frac{FP}{FP + TN}$$

As we don't ever make false prediction, the true negative number is 0, hence

$$\frac{FP}{FP} = 1$$

2.4.1 From binary to multi-class classification

So far, three methods we have studied can only handle two classes: positive vs negative.

In general, one would typically like to discriminate between more than two categories. E.g., for image recognition. Let us now see how our three linear methods defined earlier can handle this scenario.

least-square classification In essence, least-square classification treats the problem as a regression task

- The model predicts a continuous value $\hat{y} = w^{(0)} + w^{(1)}x$
- There is nothing that explicitly makes this value stop at 1, so we could continue the line further

The simplest way to go from binary case to handling multiple classes would then be to use one integer value for each category. (we then round the prediction of the model to the nearest integer to obtain the predicted class value).

This, however makes no sense, there is **no natural** order between the different categories (the prof trolled us). If we were to do so, this would really just comes down to a literal regression task. Furthermore this would implies that guessing 4 instead of 3 would be better than guessing -1 and that's complete nonsense when speaking about classification tasks.

Dealing with multiple classes

The most common approach to modelling a multi-class problem consists of encoding the class label as a vector with a single 1 at the index corresponding to the category and 0's elsewhere

Intuition

We can see it as each label is a dimension in $\mathbb{R}^D$. A good classifier would then output a vector (as a guess) and this vector needs to have the same direction (or at least, near) the label.

This is referred to as one-hot encoding (or 1-of-C encoding)

Multi-output linear model

Predicting such a one-hot encoding is then similar to making the model output multiple values, as we saw for multi-output regression.

We therefore again use a matrix $W \in \mathbb{R}^{(D+1) \times C}$ such that:

$$\hat{y}_i = W^T x_i = \begin{pmatrix} w_{(1)}^T \\ w_{(2)}^T \\ \vdots \\ w_{(C)}^T \end{pmatrix} x_i$$

multi-class classification is then similar to a multi-output regression problem:

We can then write training as:

$$\min_W \sum_{i=1}^N \|W^T x_i - y_i\|^2$$

where each y_i is now a C -dimensional vector with a single one at the index corresponding to the class label, and zeros everywhere else.

Remark 8.

You can also see it as C independent binary classifier. You can separate each $w_{(i)}^T$ and train them independently and then joined them into W and you would get the same result. (In the case of the square loss).

The solution has the same form as in the regression case (obtained by zeroing out the gradient w.r.t. W), i.e.,

$$W^* = (X^T X)^{-1} X^T Y = X^\dagger Y$$

Given the optimal parameters, the prediction for a new sample is computed as:

$$\hat{y} = (W^*)^T x$$

The sample is then assigned to class k is:

$$\hat{y}^{(k)} > \hat{y}^{(j)}, \forall j \neq k$$

Or in other words the final label k is obtained as:

$$k = \arg \max_j \hat{y}^{(j)}$$

2.4.2 Decision Boundary

Remark 9. This following section applies to all linear classification models, not just least-square classification.

Decision boundaries

Definition 21. The decision boundary between two classes k and j is defined as the set of points x such that $\hat{y}^{(k)}(x) = \hat{y}^{(j)}(x)$

This corresponds to the $(D - 1)$ dimensional hyperplane

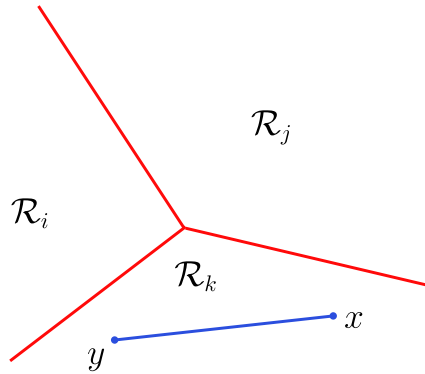


Figure 2.4: Decision boundary

The binary case is a special case of this definition.

A nice feature of those boundaries is that they are **convex**. Recall that a set S is said to be **convex** if for any pair of point $x, y \in S$ and any $\lambda \in [0, 1]$ we have that

$$\lambda x + (1 - \lambda)y \in S$$

Which means that the segment between x and y is a subset of S .

Decision Boundaries for $C > 3$ classes

For 1 and 3 this is quite intuitive but what happens when we have 4 classes, if we take the following data set: (put an image here) We would get this as decision boundary.

Example of least-square classification

Remember to put both the failure and the good case

Multi-class least-square classification: Drawbacks

As in the binary case, multi-class least-square classification suffers from the fact that the loss function does not reflect the underlying classification task.

In the binary case, we addressed this by adding a nonlinearity to the output of the linear model via the logistic function, leading to **logistic regression**. Maybe there is a better way to extend this to the multi-class scenario???

2.4.3 Multi-class logistic regression

As in the least-square classification case, we can use one-hot encodings to represent multi-class labels

Then, we again need to use a matrix $W \in \mathbb{R}^{(D+1) \times C}$ to represent the model parameters.

In this case, the probability for a class k is given by the softmax function:

$$\hat{y}^{(k)}(x) = \frac{\exp(w_{(k)}^T x)}{\sum_{j=1}^C \exp(w_{(j)}^T x)}$$

Multi-class logistic regression

The empirical risk can then be derived from the multi-class version of the cross entropy, which yield:

$$R(W) = - \sum_{i=1}^N \sum_{k=1}^C \hat{y}_i^{(k)} \ln \hat{y}_i^{(k)}$$

As in the binary case, a single $y_i^{(k)}$ is 1 for every sample i , so we really have one term per training sample. And also as in the binary case, training is done by minimizing this empirical risk using gradient descent.

Gradient for multi-class logistic regression

By following the chain rule and using the gradient of the softmax function, we can derive the gradient of the multi-class cross-entropy as (details in Bishop Chap. 4.3.4):

$$\nabla_W R(W) = \sum_{i=1}^N x_i (\hat{y}_i - y_i)^T$$

We have the same intuition as in the binary case, we multiply the error by the input.

Prediction

In order to predict, given a new input sample x , the probability/score for any class k is computed via the softmax function as:

$$\hat{y}_i^{(k)}(x) = \frac{\exp\left(\left(w_{(k)}^*\right)^T x\right)}{\sum_{j=1}^C \exp\left(\left(w_{(j)}^*\right)^T x\right)}$$

The final class label is then predicted as:

$$k = \arg \max_j \hat{y}^{(j)}(x)$$

Remember to put screenshot slide 53

Evaluation metrics for multi-class classification

As in the binary case, we can compute a confusion matrix which is pretty intuitive, for instance here with 3 classes (UCI Iris dataset)

	Iris-setosa	Iris-versicolor	Iris-virginica
Iris-setosa	12	1	1
Iris-versicolor	0	16	0
Iris-virginica	0	1	16

Where the row are the Actual and the column are the predicted value. This means that we have only once predicted Iris-versicolor when in fact it was a Iris-setosa

Metrics for multi-class problems

We still have the **Accuracy** and the **recall**, **FP-rate** and **F1-score**

Accuracy

A global accuracy can be obtained by summing the correct predictions for all classes (diagonal values of the confusion matrix), and dividing this by the total number of samples (sum of all entries in the confusion matrix)

Precision, recall, FP-rate, F1-score

The simplest approach to handling these metrics is to compute them for each individual class, and then average over the classes

In this case, each individual class is in turn considered as the '*positive*' class while the others act as the '*negative*' class

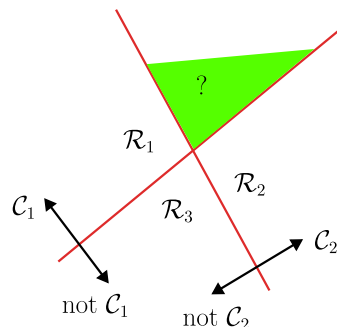
This does not account for class imbalance (some classes may have many more samples than others; more about this in future lecture)

This can be handled by weighting the metric for each class by a value depending on the proportion of samples from that class.

So far, we have only seen how to extend binary least-square classification and binary logistic regression to the multi-class scenario. How about SVM? Unfortunately, there is no ideal solution to handle multiple classes in SVM. Instead, one typically relies on one of the two solution described next.

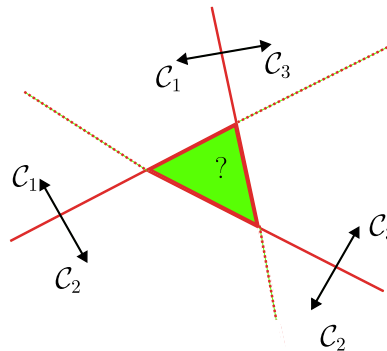
First solution

A solution is to use several binary classifiers. E.g., one-vs-rest: Classify each individual category vs all the other ones jointly



This however, leaves ambiguities (e.g., sample can be classified as being from both class 1 and class 2)

Second solution We can only train it as one-vs-one: train one classifier for every pair of classes. With this technique we can output one class for each input:



This also may result in inconsistencies (e.g., a sample can be classified as being from class 1, class 2 and class 3)

Remark The green part here are elements which would be classified as more than one class (i.e., in the first solution, the green part would be classified as $\mathcal{R}_1$ and $\mathcal{R}_2$).

Comparing linear classifiers

We have now seen 3 different linear (multi-class) classifiers:

- least-square classification
- logistic regression
- SVM

In practice, one almost never uses the least-square one. It does not reflect the underlying classification task, but is useful from a pedagogical standpoint. But so which one is the best?

Comparison on the Iris dataset

UCI Iris dataset:

- 150 samples
- 4 attributes (petal and sepal length and width)
- 3 iris species

	Accuracy	training time	testing time
SVM, linear	88.0 %	0.006s	0.002s
Logistic regression ($C = 1$)	88.0 %	0.011s	0.0001s
Logistic regression ($C = 1000$)	92.00%	0.0010s	0.002s

The value C is a hyper-parameter encoding 'regularization' strength. We will talk about this in a future lecture; let's ignore it for now.

Comparing on the MNIST dataset

MNIST handwritten digit recognition dataset:

- 60'000 training samples, 10'000 test samples
- 28×28 images
- 10 digits, 0, ..., 9

	Accuracy	training time	testing time
SVM, linear	94.16 %	168.6950s	158.0101s
Logistic regression ($C = 1$)	91.47 %	272.1309s	0.0531s
Logistic regression ($C = 1000$)	91.23%	1807.0624s	0.0529s

Remark 10. I think here the C is not the same as the number of classes, our loss is defined using the pure cross-entropy:

$$R(W) = - \sum_{i=1}^N \sum_{k=1}^C y_i^{(k)} \ln \hat{y}_i^{(k)}$$

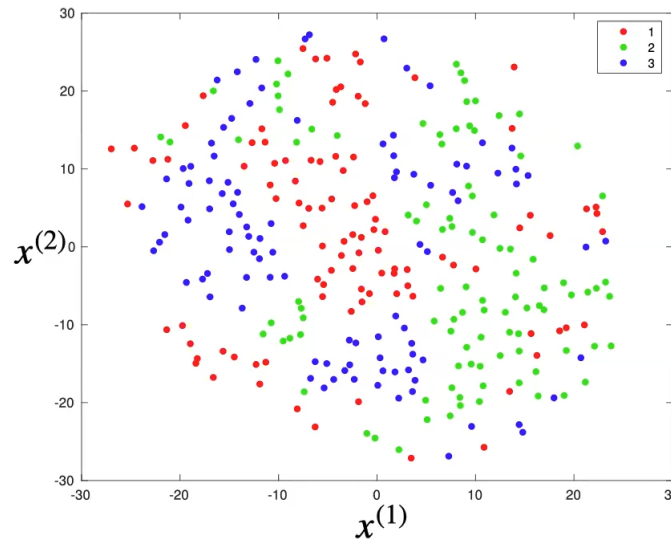
Where we added the regularization:

$$R(W) = \frac{1}{2C} \| \|W\| \|^2 - \sum_{i=1}^N \sum_{k=1}^C y_i^{(k)} \ln \hat{y}_i^{(k)}$$

And If I understand correctly why chatgpt said to me both C here are completely different.

2.5 k-nearest Neighbor

From the beginning of the course we only have used linear methods. the issue about them is that a lot of the data in our world cannot be described as a linear model, take for instance this dataset:



You cannot fit a linear model directly to this data.

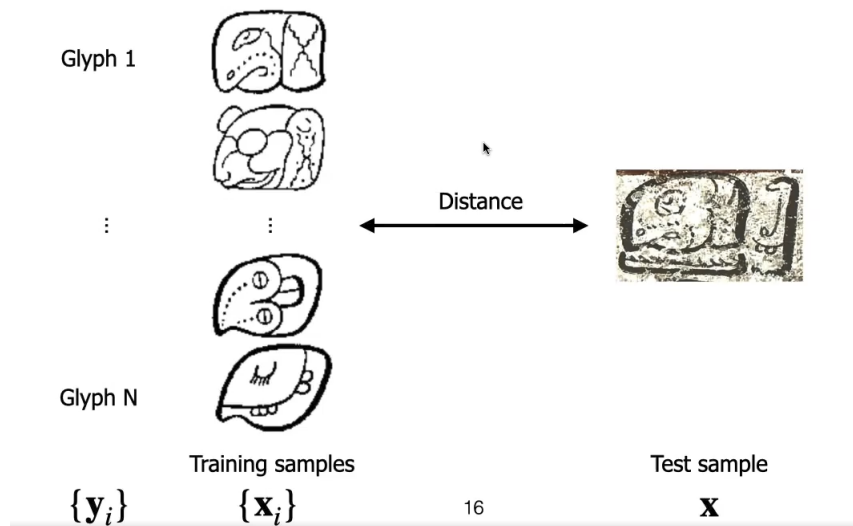
Intuition behind the nearest neighbor method

The intuition is that similar data samples have the same label. For instance if we take Maya glyph recognition (Hu et al., 2017) given a test sample, we look at the training sample that look a lot like the input sample and output the same output. This works with regression and classification.

2.5.1 NN: Algorithm

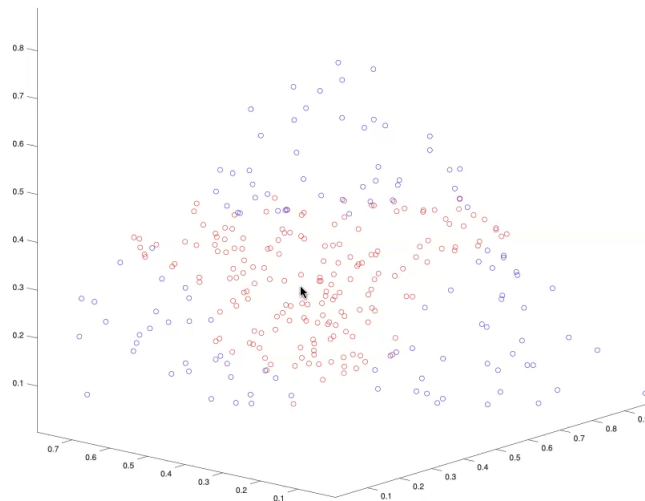
The algorithm can be computed as follow:

1. Compute the distance between the test sample x and all training samples $\{x_i\}$
2. Find the sample x_{NN} with minimum distance
3. Assin the corresponding label/value y_{NN} to the test sample



And this is our first **non-linear** algorithm!! In this case as in the linear model the results may change for different distance functions.

Example Let us see a toy example with 3D histograms belonging to 2 classes (colors):



Histograms

Definition 22. A **histogram** in $\mathbb{R}^D$ is a D -dimensional vector such that:

$$x^{(d)} \in [0, 1], \forall 1 \leq d \leq D$$

$$\sum_{d=1}^D x^{(d)} = 1$$

Given a vector $\mathbb{R}^D$ whose values are all non-negative (and at least one is strictly positive), one can obtain a histogram by dividing each value by the sum of values (normalize).

Example Let the original vector in $\mathbb{R}^9$ be:

$$\tilde{x} = (1 \quad 2 \quad 1 \quad 2 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0)^T$$

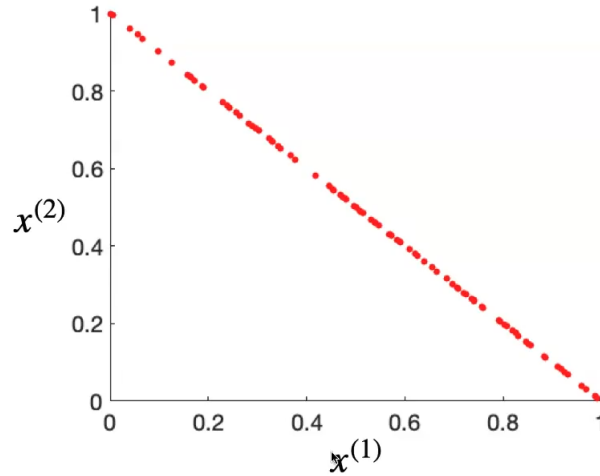
The sum of its value is:

$$\sum_{d=1}^D \tilde{x}^{(d)} = 8$$

Then, the histogram can be computed as:

$$x = \left(\frac{1}{8} \quad \frac{1}{4} \quad \frac{1}{8} \quad \frac{1}{4} \quad \frac{1}{8} \quad \frac{1}{8} \quad 0 \quad 0 \quad 0\right)$$

intristingly, histograms live in a $D - 1$ dimension space, for instance if we take $n = 100$ samples in dimension $D = 2$ then our histogram can be:



Back to NN

The default distance that we will use in our algorithm will be the euclidean distance:

$$d(x_i, x) = \sqrt{\sum_{d=1}^D (x_i^{(d)} - x^{(d)})^2}$$

Another fairly common used distance is the **Chi-square** distance:

$$d(x_i, x) = \sqrt{\chi^2(x_i, x)} = \sqrt{\sum_{d=1}^D \frac{(x_i^{(d)} - x^{(d)})^2}{x_i^{(d)} + x^{(d)}}$$

Another properties of the nn algorithm is that our results may be sensitive to outliers

- A point close to the outlier (blue circle in the center) will be missclassified
- Solution: Look at multiple neighbors instead of just one

This is the idea behind the k-nearest algorithm. Instead of taking only the nearest, we take k nearest.

2.5.2 k-nearest neighbors

The classification can be done as follow:

- classification**
1. Compute the distance between the test sample x and all training samples $\{x_i\}$
 2. Find the k samples $\{x_{NN1}, x_{NN2}, \dots, x_{NNk}\}$ with minimum distances
 3. Find the most common label among these k nearest neighbors (majority vote)
 4. Assign the corresponding label y_{MV} to the test sample

If several labels appear the same number of times among the $k - NN$, one strategy consists of taking the one whose samples have the smallest average distance to the test sample.

- Regression**
1. Compute the distance between the test sample x and all training samples $\{x_i\}$
 2. Find the k samples $\{x_{NN1}, x_{NN2}, \dots, x_{NNk}\}$ with minimum distances
 3. Compute the value $\hat{y}$ for the test sample based on that of these $k - NN$

But how do we compute $\hat{y}$?

Two strategies Intuitively we can either choose two things two in order to compute the label:

1. The number of label for each
2. The distance for each label

Using the average of the $k - NN$ values $\{x_{NNi}\}$:

$$\hat{y} = \frac{1}{k} \sum_{i=1}^k y_{NNi}$$

Using a weighted average of the $k - NN$ values, based on the inverse distances:

$$\hat{y} = \frac{1}{\sum_{i=1}^k w_{NNi}} \sum_{i=1}^k w_{NNi} y_{NNi}$$

Where

$$w_{NNi} = \frac{1}{d(x_{NNi}, x)}$$

Remark 11. To have a better visualization, we can use the demo here: <http://vision.stanford.edu/teaching/cs231n-demos/knn/>

k-nearest: Example What about performance, if we take the MNIST classification (which has 10'000 data samples) for each of our methods we have:

Method	Preprocessing	Error in %	Reference
K-nearest-neighbors, Euclidean (L_2)	none	5.0	LeCun et al.1998
K-nearest-neighbors, Euclidean (L_2)	none	3.09	Kennet Wilder. U. Chicago
K-nearest-neighbors, L_3	none	2.83	Kenneth Wilder. U. Chicago

If now we use preprocessing before using our algorithm defined above we can get:

Method	Preprocessing	Error in %	Reference
K-NN-neighbors, L_2	deskewing	2.4	LeCun et al.1998
K-NN-neighbors, L_2	deskewing, noise removal, blurring	1.80	Kennet U. Chicago
K-NN-neighbors, L_3	deskewing, noise rem., blurring	1.73	Kenneth U. Chicago
K-NN-neighbors, L_3	deskewing, noise rem., blurring, 1 pixel shift	1.33	Kenneth U. Chicago
K-NN-neighbors, L_3	deskewing, noise rem., blurring, 2 pixel shift	1.22	Kenneth U. Chicago

Using even fancier preprocessing (shiftable edges, subsampling to 16x16 pixels etc...) we can get up to 0.52 % error.

Remark 12. Remember to put the face completion example

k-nearest neighbors: Properties

There is no **learning** what we need is:

- a good data representation
- a distance function
- a given value k

This is referred to as a **non-parametric** learning method:

- No parameters to optimize during training
- k is said to be a **hyper-parameter**, set manually (or by validation, which we will discuss in future lecture)

As we have seen before, k-nn can be a very good a predicting but there is always a downside, it is **computationally expensive**. For **each test sample**, one need to compute the distances to **all training samples**. There is solution for this where we approximate nearest neighbor search:

- Hashing
- kd-trees

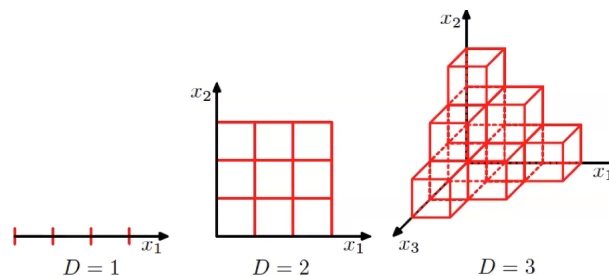
Curse of dimensionality

In the real world, data representations tend to be high dimensional, e.g., vectorizing an image yields a huge vector:

$$x_i \in \mathbb{R}^{536 \cdot 356 \cdot 3} = \mathbb{R}^{572448}$$

In such high dimension, all points tend to be far apart and we tends to have what we call curse of dimensionality (nummet mentionned??)

Let us divide the input space into regular cells: for dimensions $D = 1, 2, 3$ this gives



- The number of cells grows exponentially
- k-NN would then require an exponential growth of the data to cover the entire space

Handling high-dimensional data also make $k - NN$ memory inefficient, we need to store all the training data to compute distances.

Fortunately, real data often lies in much smaller subspaces, what we means by that is we don't necessarily need to span all space in order to make correct enough guess. Spanning a smaller spaces might be enough. In a few weeks, we will talk about how to find the low dimensional subspace of a given dataset.

2.5.3 Summary

Advantages

- Simple method
- Effective to handle nonlinear data
- Only requires defining one parameter (k)

Drawbacks

- The results depend on k
- Becomes expensive as N and/or D grow
- May be unreliable with a large D

2.6 Nonlinear classification

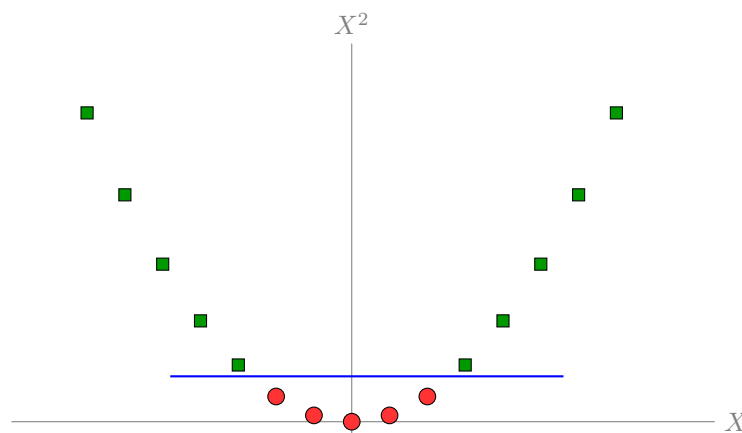
The question we will try to answer now is: how can we handle this data:



Now what if the red circle here are centered around 0. Then maybe by transforming our input data we will be able to handle it. Let us create a mapping:

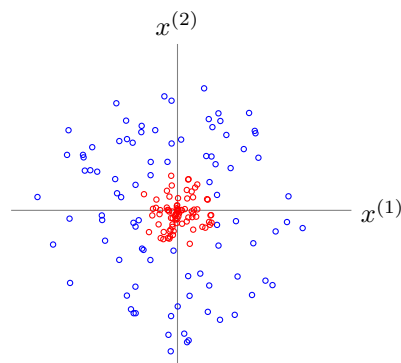
$$x \rightarrow [x, x^2]$$

Then if we mapped this new input:



In this new space we can now linearly separate our data.

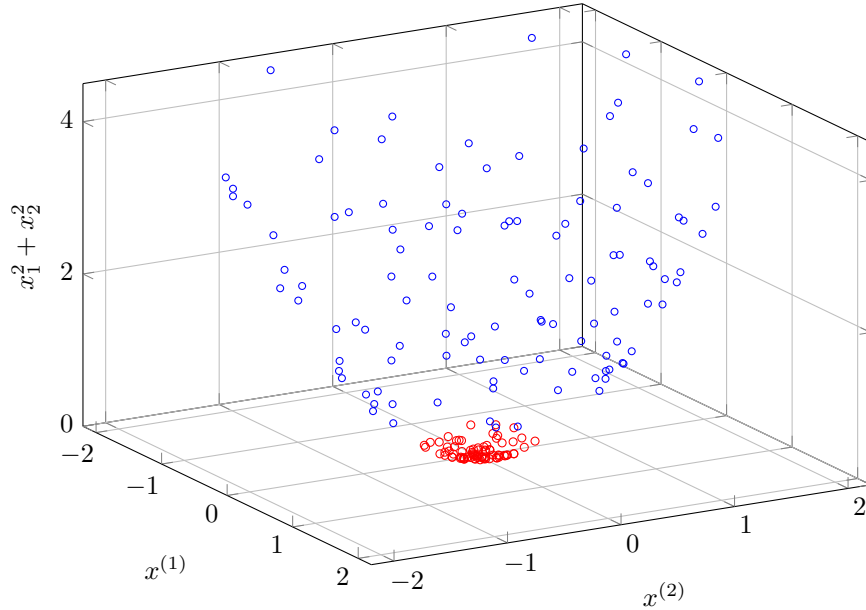
Now what if our data like this:



Intuitively this is the same case as the one before but in 2D, we could create a mapping for instance:

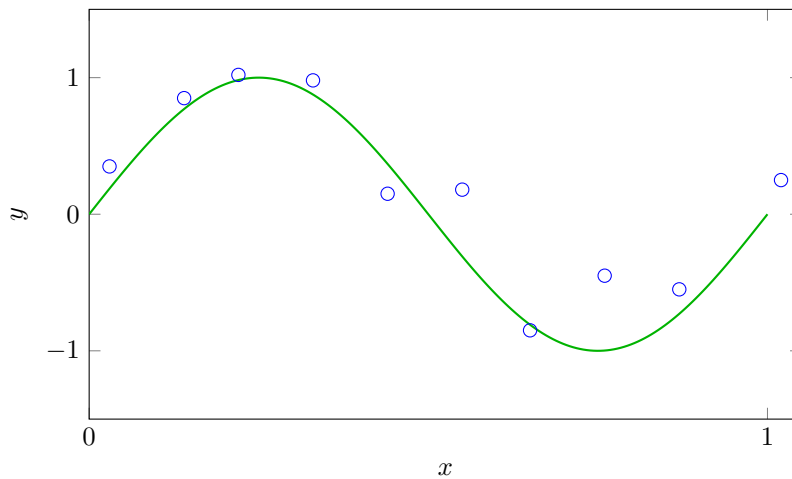
$$\begin{pmatrix} x^{(1)} \\ x^{(2)} \end{pmatrix} \rightarrow \begin{pmatrix} x^{(1)} \\ x^{(2)} \\ (x^{(1)})^2 \end{pmatrix}$$

Which should give the 'same' output as what we have done before. And this is pretty intuitive, we are mapping to the length of the circle as our data was bounded by a circle. And because our boundary is given as a circle, by 'mapping' our data into circle, then the boundary become linear.



2.6.1 Polynomial curve fitting

Given that we have training observations (blue circles, 1D input, 1D output) coming from the true green curve:



What we can do here is to find a polynomial that approximates the true curve using these observations.

Such a polynomial function of degree M can be written as:

$$y = w^{(0)} + w^{(1)}x + w^{(2)}x^2 + \dots + w^{(M)}x^M = \sum_{j=0}^M w^{(j)}x^j$$

Where the $\{w^{(j)}\}$ are the coefficients of the different terms, but x^j represents x to the power j .

Polynomial feature expansion In essence, in the three examples, we performed a change of variables:

- In the first case: $x \rightarrow \varphi(x) = \begin{pmatrix} x & x^2 \end{pmatrix}$

- In the second case: $x \rightarrow \varphi(x) = \begin{pmatrix} x^{(1)} \\ x^{(2)} \\ (x^{(1)})^2 + (x^{(2)})^2 \end{pmatrix}$
- In the last (polynomial) case: $x \rightarrow \varphi(x) = \begin{pmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^M \end{pmatrix}$

In general, for D -dimensional inputs, we can obtain a large $\varphi(x)$ such that:

$$\varphi(x) = \begin{pmatrix} 1 \\ x^{(1)} \\ \vdots \\ x^{(D)} \\ (x^{(1)})^2 \\ \vdots \\ (x^{(D)})^2 \\ \vdots \\ (x^{(1)})^M \\ \vdots \\ (x^{(D)})^M \\ x^{(1)}x^{(2)} \\ \vdots \\ x^{(1)}x^{(D)} \\ \vdots \\ x^{(D-1)}x^{(D)} \\ (x^{(1)})^2x^{(2)} \\ \vdots \end{pmatrix}$$

An important thing to understand here is that we are just expanding our input to our usual linear model, nothing more. In essence, polynomial feature expansion uses a mapping from $x \in \mathbb{R}^D$ to another representation $\varphi(x) \in \mathbb{R}^F$, where $F \gg D$. But we can still use a linear model in this new space and write:

$$\hat{y}_i = w^T \varphi(x_i)$$

Where now $w \in \mathbb{R}^F$ (assuming that the 1 accounting for the bias has been incorporated in φ).

Training expanded features

For linear regression, we used the least-square loss:

$$\min_w \sum_{i=1}^N (w^T x_i - y_i)^2$$

With our expanded features, we can re-write this as:

$$\min_w \sum_{i=1}^N (w^T \varphi(x_i) - y_i)^2$$

Not that the w vector in the two equations above are different:

- In the first, $w \in \mathbb{R}^{D+1}$
- In the second, $w \in \mathbb{R}^F$

This new function is still convex, and its gradient is given by:

$$\nabla R = 2 \sum_{i=1}^N \varphi(x_i) \left(\varphi(x_i)^T w - y_i \right)$$

Setting it to zero means that

$$\underbrace{\left(\sum_{i=1}^N \varphi(x_i) \varphi(x_i)^T \right)}_{F \times F} \underbrace{w^*}_{F \times 1} = \underbrace{\sum_{i=1}^N \varphi(x_i) y_i}_{F \times 1}$$

We then can group the transformed inputs $\{\varphi(x_i)\}$ and the output $\{y_i\}$ in a matrix and vector of the form

$$\Phi = \begin{pmatrix} \varphi(x_1)^T \\ \varphi(x_2)^T \\ \vdots \\ \varphi(x_N)^T \end{pmatrix} \in \mathbb{R}^{N \times F}$$

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \in \mathbb{R}^N$$

Then we have:

$$w^* = (\Phi^T \Phi)^{-1} \Phi^T y$$

**Mapping
to a higher-
dimensional
space**

In all the examples considered before, the mapping φ increases the dimensionality of the input to the linear model:

- From 1 to 2 in the first toy example
- From 2 to 3 in the second one
- From 1 to $M + 1$ in the polynomial fitting case

But why is higher dimension good?

Higher dimension give us more **flexibility**. There is a theorem called Cover's theorem which said that as you increase the dimension **non-linearly**, you increase the probability of making the data linearly separable.

Note that it is important for the mapping φ to be **nonlinear**. A linear mapping would not result in a nonlinear model in the original space. To show this, let us consider the following linear mapping from a 2D input x to a 3D representation $\varphi(x)$:

$$\begin{pmatrix} x^{(1)} \\ x^{(2)} \end{pmatrix} \rightarrow \begin{pmatrix} x^{(1)} \\ x^{(2)} \\ x^{(1)} + x^{(2)} \end{pmatrix}$$

Then, a linear model applied to φ would have the form:

$$\begin{aligned} y &= w^{(0)} + w^{(1)}x^{(1)} + w^{(2)}x^{(2)} + w^{(3)}(x^{(1)} + x^{(2)}) \\ &= w^{(0)} + (w^{(1)} + w^{(3)})x^{(1)} + (w^{(2)} + w^{(3)})x^{(2)} \end{aligned}$$

Which is equivalent to:

$$y = w^{(0)} + \bar{w}^{(1)}x^{(1)} + \bar{w}^{(2)}x^{(2)}$$

Which implies that there is no nonlinear behavior.

Intuition

Adding dimension which are linearly dependent from other is 'redundant'. If φ was a linear function, then the job of this function could already been made by the model. The goal of adding a dimension was to do the job that the linear model couldn't do. Literaly any linear function can be done by the linear model (by definition).

Cover's theorem

Theorem 2 (Cover's theorem). A complex pattern-classification problem, cast in a high-dimensional space nonlinearly, is more likely to be linearly separable than in a low-dimensional space, provided that the space is not densely populated.

2.7 Overfitting, Cross-validation and Regularization

The goal of this section is to understand the notions of model complexity and overfitting. We will introduce several strategies to select a model that does not overfit and introduce the notion of regularization to prevent overfitting.

Model complexity

As we have moved from linear models to nonlinear ones, we have obtained increasing flexibility:

- We do not longer assume that the ouput linearly depends on the input
- We can handle classification problems in which the classes are nonlinearly separable

This flexibility, however, may sometimes make the model too complex for the task at hand. To illustrate this let us look at an example with k-NN:

Example

Using k-NN and some data for instance:

put images

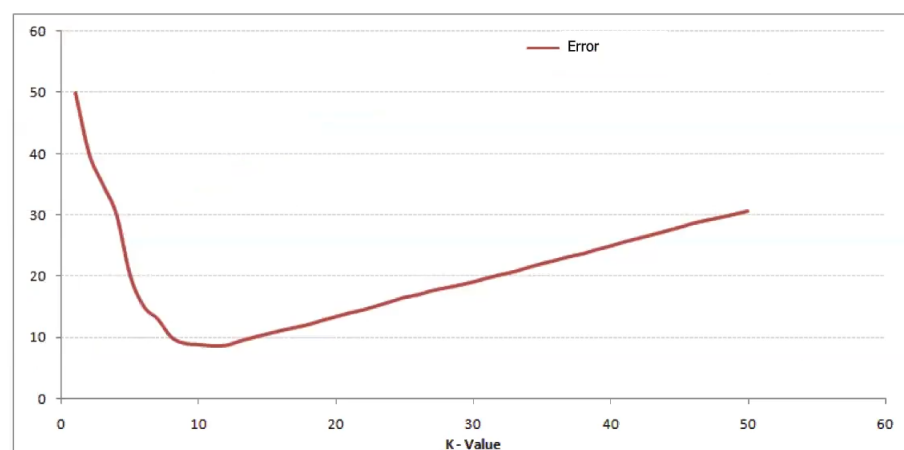
The number of neighbors k influences a lot of our algorithm react for instance let us take some examples of values for k :

put another image here (both)o

As you can see by adding/removing k our algorithm our model differs. The question we are trying to answer is, what k is the best and first, what a low k do, what a high k do?

k-NN: Training vs test

Let's look at typical training and test curves (from T. Srivastava @ AnalyticsVidhya)



Test error

As we can see when having low k we are **overfitting**, meaning that copied our training data, but on on test data, the error will be high. And after passing 10 what we observed is that our model is actually performing worse than before even though we are incrementing k ; our model is **underfitting**. In this case the error is large on both the training data and the test data. This is called **underfitting**.

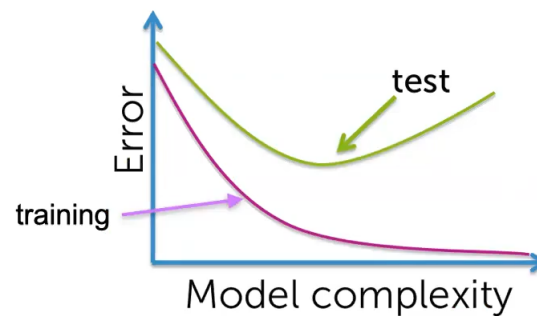
Remark

This behavior is **not** specific to k-NN. It can happen in most machine learning algorithms.

Remark 13. I find the example quite misleading. Here we see a low k as the overfitting case. As if the model is too easy then we are overfitting. But in general and as we will see later, the easier the model the more underfitting our model is doing. And it also works the other way around, the more our model is complex, the more our model is overfitting.

Overfitting vs Underfitting

Those are general phenomena. And we can often link the error/fitting to the model complexity. By increasing the complexity we are also more prone to have better results on our training data, maybe too much which will lead us to have a model that is overfitting.



Model Selection

The question is then:

Since we don't have access to the test data, how do we choose the right complexity for the model?

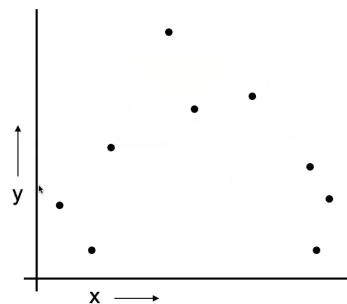
Our solution is **Cross-validation**

Remark

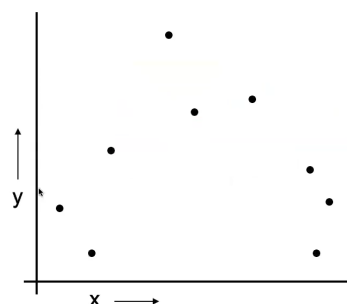
The following material was adapted from Andrew Moore's cross-validation slides

Cross-validation: Toy example

Let us have a regression problem with a 1D input x and a 1D output y . We are given the following training observations:



We will consider 3 different models (of increasing complexity):

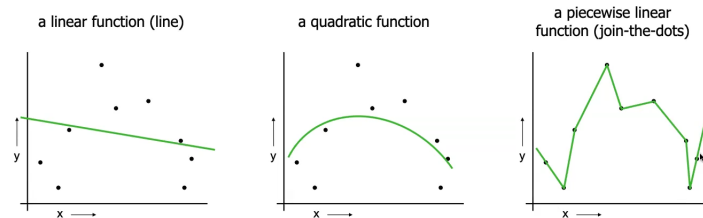


As our goal is to fit our data, why don't choose the one that best fits the data?

What we truly care about is how well the model is going to predict future data drawn from the same distribution. To do so, why not simulate data based on our training data.

Validation set method

We take our training data which we will split into two parts, one as the **training set**, and the other as a **validation set** (e.g., 30% of the data).



Advantages

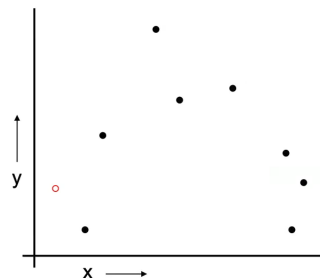
- Very simple
- We can choose the model based on the validation error

Leave-one-out cross-validation (LOOCV)

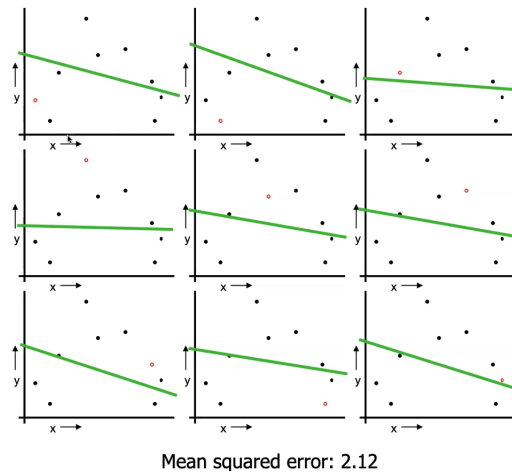
As an alternative method for the validation set method, we can use **LOOCV**: we will take one sample at a time and treat it as a validation sample, and we will loop through each sample as a validation sample.

For $i = 1, \dots, N$

1. Let (x_i, y_i) be the i^{th} sample
2. Temporarily remove (x_i, y_i) from the training set
3. Perform regression on the remaining $N - 1$ samples
4. Compute the error on (x_i, y_i)

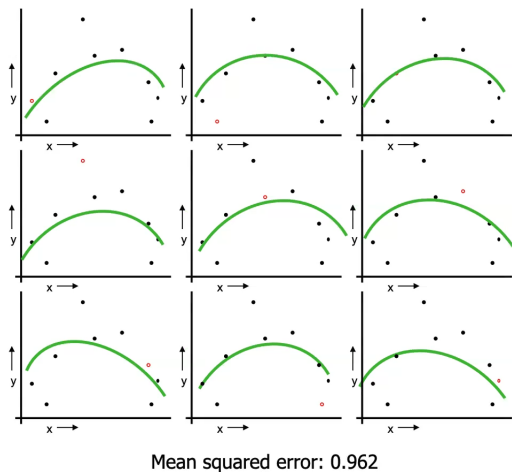


When you have done all points, report the average error. So if we take our 9 samples and we therefore fit 9 models, one for each points, we get the following curve (for a linear model)



For each of those model we have **one** test data which will gives us an error, given the average error for all models, we have 2.12.

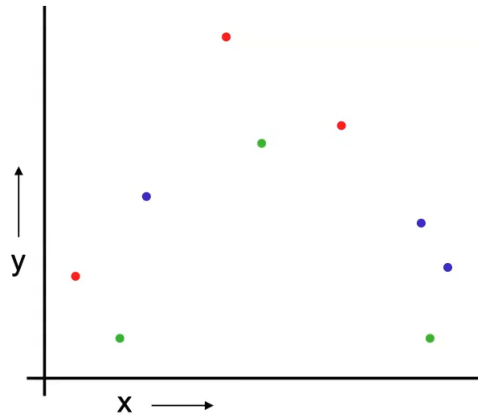
But is this the best model? Now we can check and be pretty sure about our metriics. For instance, let us use a squared model:



So as we can see, this type of cross-validation is pretty expensive, we have to train N times for **each model**! So which one is best?

	Downside	Upside
Validation-set	Unreliable estimate of future performance	Cheap
Leave-one-out	Expensive	Doesn't waste data

Both are a bit extreme, let us now look for something that is lying in the middle, **k -fold cross-validation**. First recall what were the issue about our two previous methods. The first one was that everything was fixed, the size of the test, the data that we used for training **and** for testing. The result of this is that the prediction of the future performance is unreliable (we could just be lucky with our test data). The issue with the second methods was that it was too expensive to compute because we needed to compute a new model **for each point**. But why each point, why don't we just randomly split the data set into k partitions (e.g., $k = 3$, shown with the different colors)



And now we do the same thing as what we did before but on partitions: we iterate through our partitions, we train our data in $\{\text{data set}\} \setminus \{\text{partitions}\}_i$. Then we compute the errors on the partitions $_i$. We then report the average error on all points.

	Downside	Upside
Validation-set	Unreliable estimate of future performance	Cheap
Leave-one-out	Expensive	Doesn't waste data
10-fold	Wastes 10% of the data. 10 times more expensive than Validation-set	Only wastes %. Only 10 times more expensive instead of N
3-fold	Wastier than 10-fold. Expensivier than validation-set	Slightly better than validation set
N -fold	Identival to Leave-one-out	

Remark 14. When we say "Wastes 10%" this doesn't mean that there is litteraly 10% of the data that go to the trash bin, what this means is that when **evaluating** the performance of our model, we loose does 10% for each fold. But not in the final training nor in the overall process of the cross-validation.

But ultimately there is not a best validation method. In practice one usually don't use Leave-one-out as it is too expensive. The k -fold is quite common when the model is not to expensive to train, but in the context of deep neural networks, almost everyone uses validation set.

Cross-validation Cross-validation can help to:

- Select the best model among a small number of candidates
- Find the best value for one hyper-parameter (e.g. k in k-NN)
 - For this, one treats each value of the hyper-parameter as a separate model

However, it becomes impractical with too many candidates, for example, one cannot choose which combination of functions to use for features expansion among a very large set.

Now we have one tool in order to have a model that can generalize well. Let us now look at another way to prevent overfitting by penalizing model complexity via **regularization**. But before that let us go back to linear model.

Overfitting with a linear model

Because the linear model is simple, overfitting occurs fairly rarely. Nevertheless, if there are fewer training samples than input dimensions ($N \leq D + 1$), then one can always fit a hyperplan that passes exactly through the training samples.

Proof

This can easily been check N points in $D + 1$ dimensions. Our linear model will be given as $w^T x + b$. Let us simplify notation by absorbing the bias into the vector $\tilde{x} = (x_1, \dots, x_D, 1)$ then $\tilde{w} \in \mathbb{R}^{D+1}$ and our

model f is given as:

$$f(x) = \tilde{w}^T \tilde{x}$$

The input dimensions is $D+1$ and the parameter vector is $w \in \mathbb{R}^{D+1}$. Given N training samples:

$$(x^{(1)}, y^{(1)}), \dots, (x^{(N)}, y^{(N)})$$

After the augmentation with have that each $\tilde{x}^{(i)} \in \mathbb{R}^{D+1}$. Then what we are saying is that we want our model to fit perfectly i.e.:

$$w^{(T)} \tilde{x}^{(i)} = y^{(i)}, \quad \forall i = 1, \dots, N$$

Now let us stack our vector $\tilde{x}$ and defined $X \in \mathbb{R}^{N \times (D+1)}$, where each row is $(\tilde{x}^{(i)})^T$:

$$X = \begin{pmatrix} - & (\tilde{x}^{(1)})^T & - \\ - & (\tilde{x}^{(2)})^T & - \\ & \vdots & \\ - & (\tilde{x}^{(N)})^T & - \end{pmatrix}$$

This gives us the final system

$$Xw = y$$

This implies that we have a linear systems with N equations and $D+1$ unknowns, this is solvable when $N \leq D+1$

Penalizing over-fitting

The standard way to penalize the complexity of a model is to add a regularizer to the empirical risk. This leads to a new training objective function of the form:

$$E(w) = R(w) + \lambda E_w(w)$$

Where $R(w)$ is the empirical risk and $E_w(w)$ is the regularizer. λ is a hyper-parameter that defines the influence of the regularizer ($\lambda > 0$)

Regularised linear regression

One of the most common regularizer is the sum of squares of the weights (there is a probabilistic motivation behind it)

$$E_w(w) = w^T w$$

This lets us write regularized linear regression as

$$\min_w \sum_{i=1}^N \left(\varphi(x_i)^T w - y_i \right)^2 + \lambda w^T w$$

This remains a convex function (for $\lambda \geq 0$) and so we can still find the solution by zeroing out the gradient. But if you insists to compute the gradient...

$$\begin{aligned} \nabla E &= \nabla R + \nabla E_w \\ &= 2 \sum_{i=1}^N \varphi(x_i) \left(\varphi(x_i)^T w - y_i \right) + 2\lambda w \end{aligned}$$

As we can see the only differences is the new $2\lambda w$, the other terms is the same as what we saw in earlier lectures.

Setting the gradient to zero yields

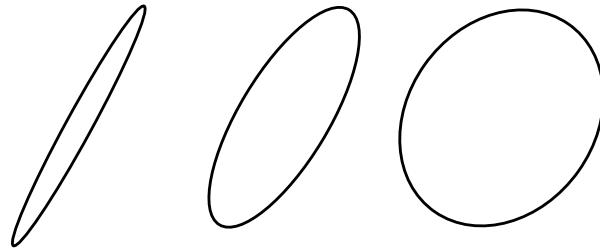
$$\left(\sum_{i=1}^N \varphi(x_i) \varphi(x_i)^T + \lambda I_F \right) w^* = \sum_{i=1}^N \varphi(x_i) y_i$$

Solution ultimately if we group the inputs in a matrix Φ and the output in a vector y , as before, this gives us the final solution:

$$w^* = (\Phi^T \Phi + \lambda I_F)^{-1} \Phi^T y$$

This also ensures that we can actually find a solution, that the matrix (the thing inside the parenthesis) can be inverted.

Remark For people who have done cs-328, this is the Tikhonov regularization technique that we have seen. In cs-328 the goal of regularization was to **improve the condition number** of a problem. The geometric interpretation was that the addition of an identity matrix causes the matrix to scale input vectors uniformly in different directions

small λ medium λ big λ

And this is the same thing as what we said above, the goal here is to improve the condition number $\implies$ having a matrix that is numerically stable, that can be inverted

Note that the regularizer affects the optimal value of the parameters, but not the mathematical form of the prediction which remains the same.

Multi-output ridge regression

When dealing with multiple output dimensions, the parameters are expressed in matrix form. We can write multi-output ridge regression as:

$$\min_W \sum_{i=1}^N \|W^T \varphi(x_i) - y_i\|^2 + \lambda \|W\|_F^2$$

Where $\|W\|_F^2$ is the square Frobenius norm, equivalent to the sum of the square of all values in the matrix W .

By zeroing out the gradient, we obtain the solution:

$$W^* = (\Phi^T \Phi + \lambda I_F)^{-1} \Phi^T Y$$

Regularization and logistic regression

Regularization may also be added to the cross-entropy loss used to train the logistic regression model.

- With a convex regularizer, such as the sum of squares, the overall regularized risk remains convex
- The scikit-learn implementation of logistic regression has such an L^2 regularizer by default

Finding the right regularization strength Note that the use of a regularizer leads to another hyper-parameter for the model, λ . And this parameter cannot be optimized, (the solution for it is $\lambda = 0$ which is not what we wanted and the reason on why we have introduced it in the first place). But if we cannot optimize it, how should we set its value? IDK

2.8 Kernel methods

The goal of today's lecture is to introduce the notion of kernel function and the kernel trick, to do so, we need to go back to the linear model:

$$y = w^{(0)} + w^{(1)}x^{(1)} + \dots + w^{(D)}x^{(D)} = w^T \begin{pmatrix} 1 \\ x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(D)} \end{pmatrix}$$

From this model, we import a non linear machine algorithm by first doing a non linear function and **then** apply our linear model to it, for instance the polynomial feature expansion:

$$\varphi(x) = \begin{pmatrix} 1 \\ x^{(1)} \\ \vdots \\ x^{(D)} \\ (x^{(1)})^2 \\ \vdots \\ (x^{(D)})^2 \\ \vdots \\ (x^{(1)})^M \\ \vdots \\ (x^{(D)})^M \\ x^{(1)}x^{(2)} \\ \vdots \\ x^{(1)}x^{(D)} \\ \vdots \\ x^{(D-1)}x^{(D)} \\ (x^{(1)})^2x^{(2)} \\ \vdots \end{pmatrix}$$

In essence, polynomial feature expansion uses a mapping from $x \in \mathbb{R}^D$ to another representation $\varphi(x) \in \mathbb{R}^F$, where $F \gg D$

We can use a linear model in this new space and write:

$$\hat{y}_i = w^T \varphi(x_i)$$

Given training data, we can find the optimal parameters via standard linear regression

$$\min_w \sum_{i=1}^N (w^T \varphi(x_i) - y_i)^2$$

When having more dimension, we can always transformed our inputs $\{\varphi(x_i)\}$ and the output $\{y_i\}$ in a matrix form and vector of the form:

$$\Phi = \begin{pmatrix} \varphi(x_1)^T \\ \varphi(x_2)^T \\ \vdots \\ \varphi(x_N)^T \end{pmatrix} \in \mathbb{R}^{N \times F} \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

Then we have $w^* = (\Phi^T \Phi)^{-1} \Phi^T y$

Remark 15. Maybe a little remark that I want to do here about the pseudo inverse and its definition. I think it is a bit misleading to **define** the pseudoinverse as $(\Phi^T \Phi)^{-1} \Phi^T y$, a "more intuitive way" would maybe be from the SVD decomposition and how does an inverse (from a non square matrix behaves). We can kind of see it as a generalisation of the inverse, which means if we have n data pointer and n variable then we can create the inverse without any issue $\implies$ We are *overfitting* our model. But if we have too much data point then the pseudoinverse still have to represent something meaningful in terms of the solution of the equation $\implies$ it gives the minimum-norm solution. On the other case if we have a tall matrix $\implies$ it gives the least squares solution.

The conclusion is that we compute the pseudoinverse using SVD and this gives us a generalization of the inverse of a matrix.

But we don't have to limit ourself to polynomials, we could add sine cosine, exponential, etc... But how do we choose which functions to use? The solution to this is **Kernels**

2.9 Kernel function

The solution to many machine learning algorithms ends up relying on a dot product between inputs, i.e. $x_i^T x_j$. $x_i^T x_j$ is proportional to $\cos(x_i, x_j)$ this encodes a notion of similarity between two samples

When doing feature expansion, we then have dot products of the form $\varphi(x_i)^T \varphi(x_j)$, which also encodes a similarity between the two samples, but different from the original one

Thus, instead of explicitly defining the mapping $\varphi()$, one can define a similarity function $k(x_i, x_j)$. This is called a **kernel** function. It corresponds to some mapping φ , i.e., $k(x_i, x_j) = \varphi(x_i)^T \varphi(x_j)$, but this φ may be unknown.

Example One example of commonly used kernel function is the polynomial kernel

$$k(x_i, x_j) = (x_i^T x_j + c)^d$$

It has two hyper-parameters, the bias c and the degree of the polynomial d (often set to 2)

For such kernels, the corresponding mappings φ are know, for example, with a 2D input, for $c = 1$ and $d = 2$ we have

$$\varphi(x) = \left[\left(x^{(1)}\right)^2, \left(x^{(2)}\right)^2, \sqrt{2}x^{(1)}x^{(2)}, \sqrt{2}x^{(1)}, \sqrt{2}x^{(2)}, 1 \right]$$

Given our mapping the dot product between two samples then is:

$$\begin{aligned} \varphi(x_i)^T \varphi(x_j) &= \left(x_i^{(1)}\right)^2 \left(x_j^{(1)}\right)^2 + \left(x_i^{(2)}\right)^2 \left(x_j^{(2)}\right)^2 + 2x_i^{(1)}x_i^{(2)}x_j^{(1)}x_j^{(2)} + 2x_i^{(1)}x_j^{(1)} + 2x_i^{(2)}x_j^{(2)} + 1 \\ &= \left(x_i^{(1)}x_j^{(1)} + x_i^{(2)}x_j^{(2)} + 1\right)^2 \end{aligned}$$

This corresponds to the quadratic kernel:

$$k(x_i, x_j) = (x_i^T x_j + 1)^2$$

Example II

Another common kernel is the Gaussian (or Radial Basis Function (RBF)) kernel

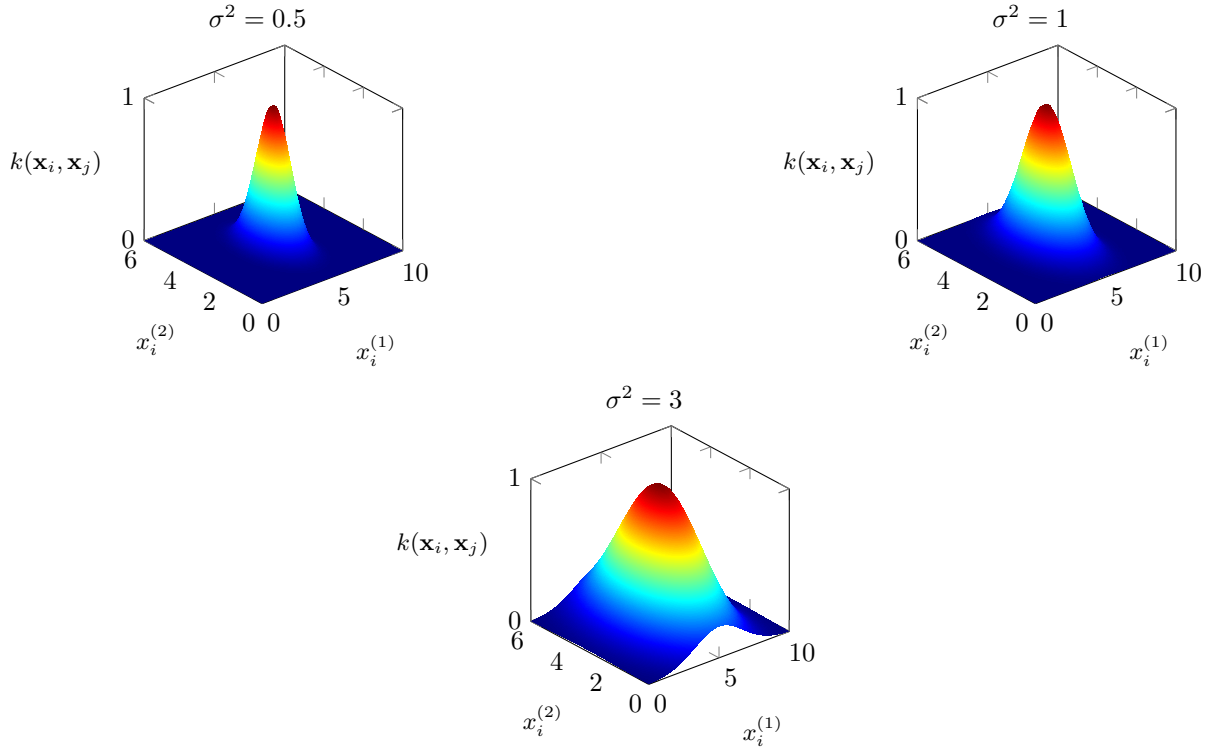
$$k(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

It has one hyper-parameter which is the *bandwidth* σ

For this kernel, there is no explicit corresponding mapping φ

Remark In theory, this kernel corresponds to a mapping to an infinite dimensional space

With 2D inputs, we can visualize the effect of σ , Let one sample be fixed as $x_j = [5, 3]^T$, and vary x_i

**Kernel trick**

Since a kernel function corresponds to a similarity, as a dot product, one can replace any dot product between two samples in the solution of a machine learning algorithm with a kernel function. This is applicable to many algorithms, and is referred to as **kernelizing** an algorithm

Let us now look at how this works for linear regression

2.9.1 Kernelization**Notation**

As mentioned before, the kernel function computes a similarity between two samples. It can be written as $k(x_i, x_j)$

In what follows, with a small abuse of notation, we will also use the kernel function to encode the similarity between one sample and all training samples, represented via the matrix X . This will give:

$$k(X, x_i) = \begin{pmatrix} k(x_1, x_i) \\ k(x_2, x_i) \\ \vdots \\ k(x_N, x_i) \end{pmatrix} \in \mathbb{R}^N$$

Furthermore, we will also use the notion of kernel matrix, in which each entry (i, j) contains the kernel function evaluated between training sample i and training sample j . This matrix is squared and can be written as

$$K = \begin{pmatrix} k(x_1, x_1) & k(x_1, x_2) & \cdots & k(x_1, x_N) \\ k(x_2, x_1) & k(x_2, x_2) & \cdots & k(x_2, x_N) \\ \vdots & \vdots & \ddots & \vdots \\ k(x_N, x_1) & k(x_N, x_2) & \cdots & k(x_N, x_N) \end{pmatrix} \in \mathbb{R}^{N \times N}$$

Kernel regression

As seen before, the solution to linear regression is given by:

$$w^* = (\Phi^T \Phi)^{-1} \Phi^T y$$

At first sight, it may seem that $\Phi^T \Phi$ contains the dot products that we want to replace with a kernel function. However, $\Phi^T \Phi$ is an $F \times F$ matrix encoding dot products between the different feature dimensions, not the different samples. A kernel matrix, containing the dot products between every pair of training samples would be of size $N \times N$ and correspond to $\Phi \Phi^T$. Therefore we need another solution, the Representer theorem

Representer theorem

Theorem 3. The minimizer of a regularized empirical risk function can be represented as a linear combination of the points in the high dimensional space. That is, we can write

$$w^* = \sum_{i=1}^N \alpha_i^* \varphi(x_i) = \Phi^T \alpha^*$$

In general if I am trying to find the minimizer, then I can represent the vector w^* as a linear combination of my training samples, with some coefficients α_i^* . Therefore $\alpha^* \in \mathbb{R}^N$

Kernel regression

So instead of searching for a minimizer of the empirical risk in terms of w , we search for one in terms of new variables $\{\alpha_i\}$

That is, the empirical risk

$$R(w) = \sum_{i=1}^N (w^T \varphi(x_i) - y_i)^2$$

What we will do, is to use the Representer theorem in order to rewrite our function with variables α

$$R(\alpha) = \sum_{i=1}^N (\alpha^T \Phi \varphi(x_i) - y_i)^2$$

But now we have the dot products that we needed before. This implies that we can rewrite using the kernel

$$= \sum_{i=1}^N (\alpha^T k(X, x_i) - y_i)^2$$

The gradient of the risk with regards to α is given by

$$\nabla R(\alpha) = 2 \sum_{i=1}^N k(X, x_i) (k(X, x_i)^T \alpha - y_i)$$

Following the same reasoning as for linear regression, we can write this matrix form as

$$\nabla_{\alpha} R(\alpha) = 2KK\alpha - 2Ky$$

Setting the gradient to 0 yields:

$$2KK\alpha^* = 2Ky \iff K\alpha^* = y$$

Which gives the solution

$$\alpha^* = (K)^{-1}y$$

If we put this solution back in the definition of w^* , we have:

$$\begin{aligned} w^* &= \Phi^T \alpha^* \\ &= \Phi^T (K)^{-1} y \end{aligned}$$

This solution still depends on Φ , which we argued could be unknown? But do we care about computing the weights? In practice, this is not a problem, because our goal is to make predictions for new samples, we do not really care about having access to w^*

For a new sample x , the prediction is given by

$$\begin{aligned} \hat{y} &= (w^*)^T \varphi(x) = y^T (K)^{-1} \Phi \varphi(x) \\ &= y^T (K)^{-1} k(X, x) \end{aligned}$$

Not that the quantity $y^T (K)^{-1}$ only depends on the training data, and thus can be precomputed (i.e., does not need to be recomputed for every test sample)

Exercise: Kernel regression

You are given the following 4 training samples (x_i, y_i)

$$\left(\begin{pmatrix} 1 \\ 1 \end{pmatrix}, 0 \right) \quad \left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, 2 \right) \quad \left(\begin{pmatrix} 2 \\ 0 \end{pmatrix}, 0 \right) \quad \left(\begin{pmatrix} 2 \\ 2 \end{pmatrix}, 2 \right)$$

The task is to compute the kernel matrix corresponding to a polynomial kernel of degree 2 with a bias 1. (If I have time I will do it here)

Recap: Ridge regression

To penalize overfitting, one can use a regularizer such as the sum of squares of the weights

$$E_w(w) = w^T w$$

Adding this to the linear regression empirical risk yields the ridge regression problem

$$\min_w \sum_{i=1}^N \left(\varphi(x_i)^T w - y_i \right)^2 + \lambda w^T w$$

What we would want here is to also translate it to a kernel. To do so we can use the Representer theorem, this way, we can rewrite it as:

$$\begin{aligned} E(\alpha) &= \sum_{i=1}^N \left(\alpha^T \Phi \varphi(x_i) - y_i \right)^2 + \lambda \alpha^T \Phi \Phi^T \alpha \\ &= \sum_{i=1}^N \left(\alpha^T k(X, x_i) - y_i \right)^2 + \lambda \alpha^T K \alpha \end{aligned}$$

Following the same reasoning as without regularizer, we can write the gradient of the risk in matrix form as

$$\nabla_{\alpha} E(\alpha) = 2K(K + \lambda I_N)\alpha - 2Ky$$

Setting it to 0 gives us the solution:

$$\alpha^* = (K + \lambda I_N)^{-1} y$$

Given α^* , one can then express W^* mathematically as in the case without regularization, and eventually use this formulation to make predictions for the test samples

The kernel of the ridge regression strongly depend on the choice of kernel and of the hyper-parameter values for a given kernel. How can you identify a good kernel and good parameter values??

We know how to choose them, using **cross-validation**.

Remark 16. We can do the same thing on ridge regression with multiple outputs (the exact same process), the prediction vector for a test sample is then given as

$$\begin{aligned}\hat{y} &= (W^*)^T \varphi(x) = Y^T (K + \lambda I)^{-1} \Phi \varphi(x) \\ &= Y^T (K + \lambda I)^{-1} k(X, x)\end{aligned}$$

Kernelization

The idea of kernelizing an algorithm does not apply only to linear regression

- It can also be used to obtain a nonlinear version of SVM (via the so-called dual formulation of SVM)
- In a few weeks, we will also do this for dimensionality reduction
- Not that logistic regression can also be kernelized although this is less commonly done

Discussion of kernelization

Advantages :

- Can effectively handle non-linearly separable data
- Often have nice solutions (e.g., closed form, convex)

Drawbacks

- Depend on the choice of kernel and hyper-parameter
- Computational complexity depends on the number of samples
 - E.g., for kernel ridge regression, one needs to invert an $N \times N$ matrix, which requires $(O(N^3))$ operations.

2.10 Data Representations & Processing

The goal of this section is to introduce the idea of data representations and one general method to represent data. We will also study different data processing techniques and introduce the problem of class imbalance and solutions to address it.

Recap: What data?

Let us first remember what data we are talking about when talking about machine learning, data can be:

- Attributes
- Text
- Sound
- Images

The problem with data is that it can be very **heterogeneous** for instance:

I am looking for an inexpensive motorcycle, nothing fancy, have to be able to do all maintenance my self.

Looking in the <\$400 range. If you can help me out, GREAT!, please reply by e-mail.

Whoops!! Wrong group. Soooooooooooooooooorry folks..

I'd like to see this info as well. As for wavelength, I think you're primarily going to find two - 880 nM +/- a bit, and/or 950 nM +/- a bit. Usually it is about 10 nM either way. The two most common I have seen were 880 and 950 but I have also heard of 890 and 940. I'm not sure that the 10 nM one way or another will make a great deal of difference. Another suggestion - find a brand of TV that uses an IR remote, and go look at the SAMS photofact for it. You can often find some very detailed schematics and parts list for not only the receiver but the transmitter as well, including carrier freq. specs. and tone decoding specs. if the system uses that.

As you can see, here this become pretty hard to vectorize right? How can we leverage text documents of different length? With linear model, the number of parameter depends directly of the input dimensions.

The same goes for image. Let us have two images one of size $536 \times 356 \times 3$ and the other of size $500 \times 500 \times 3$. First we have the same issue as before (both image doesn't have the same dimensionality). Second, the size of the dimension input is enormous, for the first image we are in $x_i \in \mathbb{R}^{536 \times 356 \times 3} = \mathbb{R}^{572448}$. Now let us have a big dataset i.e. one million images and this becomes just too big for us. We need data representations.

2.10.1 Data representations: Attributes

One way in order to represent data is to use attributes, let us take the example of UCI Abalone dataset which predict the age of abalone from measurements. The attributes are:

Name	Data Type	Measurement	Description
Sex	Categorical		M, F, and I (infant)
Length	Numeric (continuous)	mm	Longest shell measurements
Diameter	Numeric (continuous)	mm	perpendicular to length
Height	Numeric (continuous)	mm	with meat in shell
WT_Whole	Numeric (continuous)	grams	whole abalone
WT_Shuck	Numeric (continuous)	grams	weight of meat
WT_Vscra	Numeric (continuous)	grams	gut weight
WT_Shell	Numeric (continuous)	grams	after being dried
Rings	Numeric (integer)		+1.5 gives the age in years

As we can see we can have: categorical attributes (like a label), real (continuous), integer (discrete). What is good however with attributes is that all input have the same number of attributes.

Going back to the text examples from above, let us try to represent our data! First we have to see some common point between all input, all samples are written in the same language $\implies$ they relate to the same dictionary. The idea is that for each sample, we count the number of times each word in the dictionary appears. For instance:

(1) John likes to watch movies. Mary likes movies too.

(2) John also likes to watch football games.

The corresponding counts gives us:

- $\text{BoW1} = \{ \text{"John":1, "likes":2, "to":1, "watch":1, "movies":2, "Mary":1, "too":1} \}$
- $\text{BoW2} = \{ \text{"John":1, "also":1, "likes":1, "to":1, "watch":1, "football":1, "games":1} \}$

We don't need to have the whole English dictionary so what we will do is to take all word that are in both texts (union) and remove the common words, e.g. "the", "a", "to"...

With D words in the dictionary, each samples can be represented as a vector in $\mathbb{R}^D$, Using the dictionary we made above ($D = 9$):

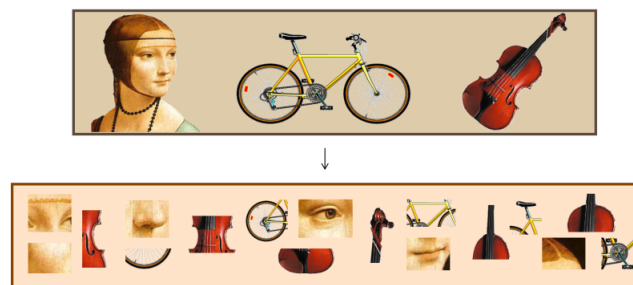
$$\begin{aligned}\tilde{x}_1 &= (1 \quad 2 \quad 1 \quad 2 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0) \\ \tilde{x}_2 &= (1 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 1)\end{aligned}$$

But here you could guess that for text sample of different length, the actual word counts may not be relevant (longer texts will just have larger values). Instead, we will encode the proportion of each word, this is achieved by dividing each vector element by the sum of all elements. Our $\tilde{x}_1$ has a total sum equals to $\sum_{d=1}^D \tilde{x}_1^{(d)} = 8$. The final representation of this text is therefore:

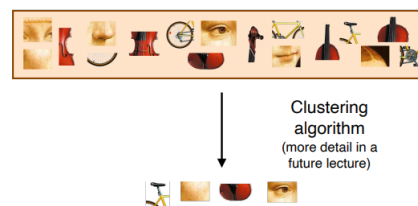
$$x_1 = \left(\frac{1}{8} \quad \frac{1}{4} \quad \frac{1}{8} \quad \frac{1}{4} \quad \frac{1}{8} \quad \frac{1}{8} \quad 0 \quad 0 \quad 0 \right)$$

In essence, what we are doing is computing the frequency of each words in our text.

How about images When trying to create attributes for images we have a big problem, there is no direct notion of 'words' for image data. However, objects are made of common parts/patches. The goal here is therefore to extract the patches from all images and to create a **visual dictionary** of those parts:



In order to create this dictionary, we will have to group the patches based on similarity to obtain prototypes that act as dictionary words. To do so we will use Clustering algorithm.



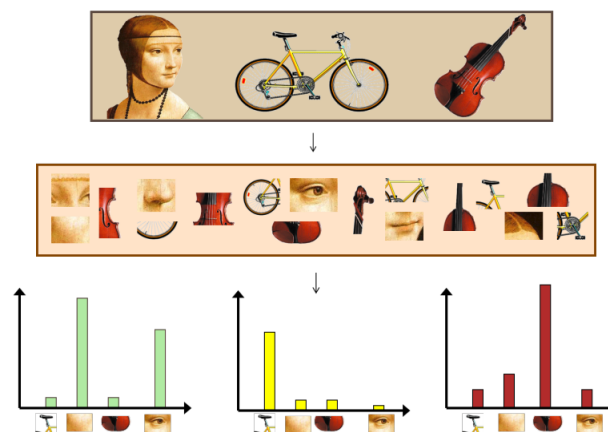
The center of each cluster (group) is taken as a visual word

Bag of visual words

For each patch in an image:

- find the nearest visual word
- add one to the corresponding element in a BoW vector

Normalize the BoW vectors to obtain histograms



This still requires comparing local image patches

- The patches can all have the same size

- They can be small enough for this to be manageable

However this is not great as there is a lot of noise and intensity issues. There is other local image representations which can be used (which goes beyond the scope of this course).

Conclusion Large, heterogeneous data can be made compact and homogeneous.

The resulting representations can be directly used as input to a machine learning algorithm.

- This is applicable to all the methods we have seen so far
- But in general to most machine learning algorithms

2.10.2 Data pre-processing

Even with a good representation, the data might benefit from being pre processed. This is because:

- Some attribute values are missing
- Some attribute values are noisy
- The feature dimensions are of incommensurate magnitude

Missing data For instance let us assume that some attribute values were not recorded (e.g. the Air quality UCI dataset contains missing values, which are tagged with a -200 value)

Potential solutions

- Ignore the corresponding
- Fill in the missing values with global constant
- Fill in the missing values with the corresponding attribute average over the dataset
- Fill in the missing values with the corresponding attribute average over the samples belonging to the same class

Noisy data Noisy data can come from a lot of reasons, for instance random measurement error, faulty data collection, data entry problem ...But our algorithm is not made for that "dirty" data, we need to **clean** it.

A way of doing that is called **binning** which works like this:

- For each attribute:
 1. Sort the data
 2. Partition it into bins
 3. Smooth the data by taking the bin means, or medians

Another way is **clustering**, we detect outliers as the clusters with too few elements and remove these samples.

Data Normalization The goal of this technique is to scale each individual attribute/feature dimension to fall within a specified range

Min-max normalization

For each feature dimension d , compute the normalized feature:

$$\tilde{x}_i^{(d)} = \frac{x_i^{(d)} - x_{min}^{(d)}}{x_{max}^{(d)} - x_{min}^{(d)}}$$

Where

$$x_{min}^{(d)} = \min_{i=1}^N x_i^{(d)}$$

$$x_{max}^{(d)} = \max_{i=1}^N x_i^{(d)}$$

z-score normalization

For each feature dimension d , compute the normalized feature

$$\tilde{x}_i^{(d)} = \frac{d_i^{(d)} - \mu^{(d)}}{\sigma^{(d)}}$$

Where

$$\mu^{(d)} = \frac{1}{N} \sum_{i=1}^N x_i^{(d)}$$

$$\sigma^{(d)} = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i^{(d)} - \mu^{(d)})^2}$$

max normalization For each feature dimension d , compute the normalized feature:

$$\tilde{x}_i^{(d)} = \frac{x_i^{(d)}}{x_{max}^{(d)}}$$

Where

$$x_{max}^{(d)} = \max_{i=1}^N |x_i^{(d)}|$$

Normalization by decimal scaling For each feature dimension d , compute the normalized feature

$$\tilde{x}_i^{(d)} = \frac{x_i^{(d)}}{10^k}$$

Where k is the smallest integer such that

$$\max_{i=1}^N |\tilde{x}_i^{(d)}| \leq 1$$

For all data processing techniques, the normalized (or cleaned) features are then used as input to a machine learning algorithm.

Normalization is not always necessary, and may or may not help depending on the data at hand on the chosen machine learning algorithm.

From data sample to data set

Data representations and data cleaning/normalization acts on the individual samples

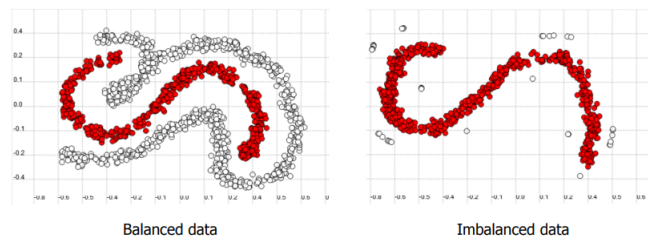
- Although the normalization statistics come from the whole training set

Nevertheless, even with good representations, the data set itself might make from learning more difficult.

A good illustration of this can be imbalanced data. I remember a meme I saw online on a model that output whether or not a number was a prime number in $\Theta(1)$, it was:

```
def isPrime(n):
    return True
```

As you can see this 'model' is very fast and almost always correct but it misses the whole point of the model right? The reason for this is that the data is so imbalanced that assuming it always on label gives us a good accuracy.



Learning with imbalanced data

In order to have a model that still work, there is two main approaches, one act on the **data**: Sampling Methods, the other act on the cost function: Cost Sensitive Methods.

Sampling methods

The issue we are facing here is the lack of data, in order to compensate we can either increase the data set by generating new data points for the smallest class, or decrease the dataset by removing redundant data points from the largest class.

Decrease the data: Under-sampling

Here we will have naive (random) undersampling:

- Randomly remove samples from the largest class until you obtain a balanced data set

However this may remove important samples

A more clever undersampling would be to remove the data point that we 'don't need' – the redundant samples. Imagine that we have two data points that are almost identical $\Rightarrow$ removing one doesn't change much the distribution of the data. This method is good if the class to undersample is densely populated (e.g. low-dimensional representation because of the curse of dimensionality)

Increase the data: Over-sampling

Here again we can use the naive (random) oversampling:

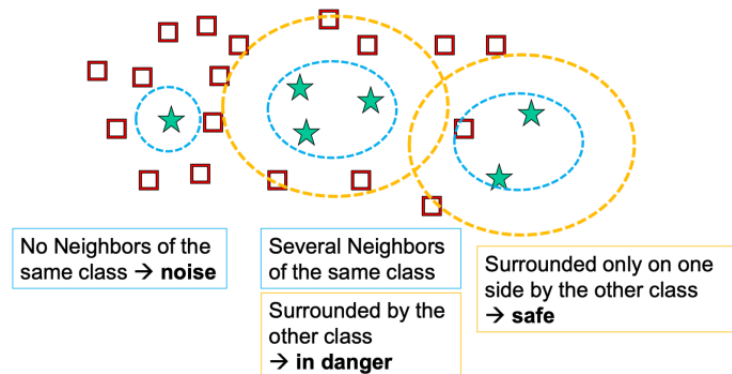
- Replicate the data samples from the smallest class until you obtain a balanced data set

The drawback here is that this may lead to overfitting

A more clever oversampling would be to create new data points based on neighbouring existing ones, meaning we take to sample and create a new one in the middle $\Rightarrow$ can lead to create noisy samples.

However there is a lot of other method to oversampling:

- SMOTE: Synthetic minority oversampling technique (CHawla et al., JAIR 2002)
- AdaSyn: Adaptive synthetic sampling (HE et al., IJCNN 2008)
- Reason about the density of data points and the border with other class



Now let us go to the other side with Cost Sensitive Methods.

Empirical risk Given N training samples $\{(x, y_i)\}$, the empirical risk is defined as

$$R(\{x_i\}, \{y_i\}, w) = \frac{1}{N} \sum_{i=1}^N l(\hat{y}_i, y_i)$$

Where $\{x_i\}$ and $\{y_i\}$ are the sets of training inputs and labels, respectively.

In general, each loss term $l(\hat{y}_i, y_i)$ has the same influence. With imbalanced data, this means that most of the empirical risk comes from the largest class. A simple solution to this problem consist of re-weighting the samples, to put more emphasis

on the rare class. This gives an empirical risk of the form

$$R(\{x_i\}, \{y_i\}, w) = \frac{1}{N} \sum_{i=1}^N \beta_i l(\hat{y}_i, y_i)$$

where β_i is the weight for sample i

Remark

Note that β_i cannot be optimized, because the optimal solution would simply be to set all of them to 0. Instead they are manually, e.g., inversely proportional to the class frequency:

$$\beta_i = 1 - \frac{N_K}{N}$$

If sample i belongs to class k , which contains N_K samples

This strategy is quite general, it can apply to the least-square classification:

$$\min_w \sum_{i=1}^N \beta_i \|W^T x_i - y_i\|^2 + \lambda \|W\|_F^2$$

Logistic regression

$$\min_W = - \sum_{i=1}^N \beta_i \sum_{k=1}^C y_i^{(k)} \ln(w_{(k)}^T x_i)$$

And also support vector machines

$$\min_{\omega^{(0)}, \tilde{W}} \frac{1}{2C} \|\tilde{W}\|^2 + \sum_{i=1}^N \beta_i \max\left(0, 1 - y_i \cdot (w^{(0)} + \tilde{w}^T x_i)\right)$$

In the three cases, since the weights are fixed during optimization, the respective optimization problems remain convex and can thus be solved to optimality

From handcrafted representations to learned ones

The representations and data processing strategies we have discussed so far were all manually designed. There is, however no guarantee that they are the best representations for the task at hand.

Ideally, one would therefore like to learn the representation jointly with the classifier. This will bring us to the topic of artificial neural networks.

2.11 Deep Learning

The goal of this section/lecture is to introduce the multilayer perceptron (MLP) model and to explain the training algorithm for an MLP.

Until now our model has been define as

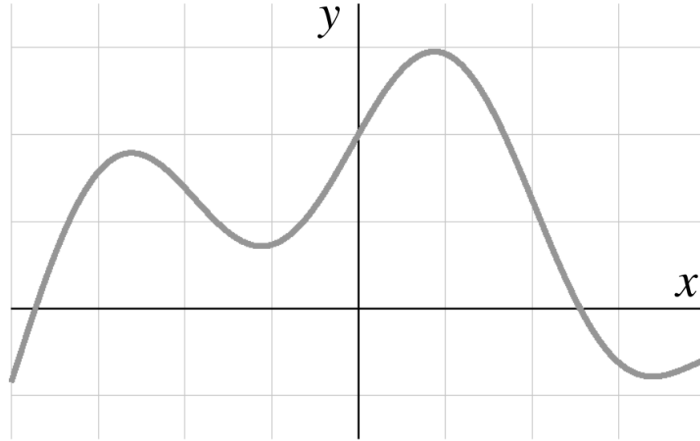
$$\hat{y}(x) = w^T x$$

In order to model data that weren't linearly 'correlated' we introduced feature expansion and kernel method (inherently) which map data to a different representation. The goal was to use a map $\varphi(\cdot)$ in order to transform the input nonlinearly which gives more flexibility.

The issue with all of those were that all of the data processing strategies we have discussed so far were all manually designed (the transformation corresponding to a kernel is still defined manually). There is, however no guarantee that they are the best representations for the task at hand. Ideally, one would therefore like to learn the representation jointly with the classifier.

Motivating example: Curve fitting

In the 1D input and 1D output scenario, we would like to find a mapping from x to y that approximates the curve below.



In previous lecture, we have attempted to do so by performing a polynomial feature expansion:

$$x \rightarrow \begin{pmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^M \end{pmatrix}$$

We then applied a linear model to the expanded features. In this case, the feature expansion was manually defined, independently of the linear model training. Instead, we would like to learn the feature expansion and the linear model jointly.

To achieve this, we can define a parametric feature expansion strategy; as we like linear models, let's use one for this purpose. Naively, we could then write the mapping as:

$$x \rightarrow W_{(1)}^T \begin{pmatrix} 1 \\ x \end{pmatrix}, \quad \text{with } W_{(1)} \in \mathbb{R}^{2 \times M}$$

and then apply a linear model with parameters $W_{(2)} \in \mathbb{R}^{M \times 1}$ to the resulting representation to predict y .

However, as mentioned in a previous lecture, linear feature expansion is useless; you can achieve the same results with just one linear model. Therefore an interesting thing here would be to add **nonlinearity** in the process. To achieve this, we will use a nonlinear function $f : \mathbb{R} \rightarrow \mathbb{R}$. But before that we need to define on what this function will be applied.

$$a_{(1)} = W_{(1)}^T \begin{pmatrix} 1 \\ x \end{pmatrix} = \begin{pmatrix} W_{(1)}^{(1,0)} + W_{(1)}^{(1,1)}x \\ W_{(1)}^{(2,0)} + W_{(1)}^{(2,1)}x \\ \vdots \\ W_{(1)}^{(M,0)} + W_{(1)}^{(M,1)}x \end{pmatrix} = \begin{pmatrix} a_{(1)}^{(1)} \\ a_{(1)}^{(2)} \\ \vdots \\ a_{(1)}^{(M)} \end{pmatrix}$$

But first let us step back a bit and think on what we just did and **why** is it different (we will come back to this later) (I meant like writing it down on your own).

So we first apply a linear model to our parameter and then we apply our

function to that result:

$$x \rightarrow \begin{pmatrix} f(a_{(1)}^{(1)}) \\ f(a_{(1)}^{(2)}) \\ \vdots \\ f(a_{(1)}^{(M)}) \end{pmatrix} f\left(W_{(1)}^T \begin{pmatrix} 1 \\ x \end{pmatrix}\right)$$

Where the final notation applies the function f in an element wise manner to the M -dimensional vector output by the linear mapping. In the context of artificial neural networks the function f is referred to as the **activation function**.

Remark

Note that the same function is applied to **every** $a_{(1)}^{(j)}$, which avoids having to design too many things manually.

Activation function: Examples

There is a lot of activation function out there. The goal of those functions is always to add nonlinearity to our model. There is three main function as such, namely:

- Sigmoid:

$$f(a) = \frac{1}{1 + \exp(-a)}$$

- Hyperbolic tangent:

$$f(a) = \tanh(a)$$

- Rectified Linear Unit (ReLU)

$$f(a) = \begin{cases} a, & \text{if } a > 0 \\ 0, & \text{otherwise} \end{cases} = \max(a, 0)$$

As you can see all of those function has something in common, their slope 'coincide' with the one of the identity function. The goal of the activation function is to 'replace' the identity, therefore an important properties that they must hold is that their derivatives is ≥ 0 .

Hidden representation

The M -dimensional vector

$$z_{(1)} = \begin{pmatrix} f(a_{(1)}^{(1)}) \\ f(a_{(1)}^{(2)}) \\ \vdots \\ f(a_{(1)}^{(M)}) \end{pmatrix} = f\left(W_{(1)}^T \begin{pmatrix} 1 \\ x \end{pmatrix}\right)$$

Is commonly referred to as **hidden representation** (or hidden layer). Where each one of its M elements is referred to as a hidden unit.

Complete model

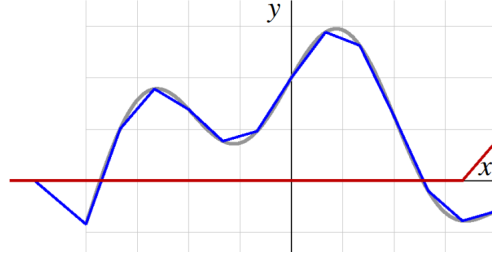
Having all of this, we can create our complete model by applying a second linear model to the hidden representation. This gives

$$\begin{aligned} \hat{y} &= w_{(2)}^T z_{(1)} = w_{(2)}^T f\left(W_{(1)}^T \begin{pmatrix} 1 \\ x \end{pmatrix}\right) = w_{(2)}^T \begin{pmatrix} f(a_{(1)}^{(1)}) \\ f(a_{(1)}^{(2)}) \\ \vdots \\ f(a_{(1)}^{(M)}) \end{pmatrix} \\ &= \sum_{j=1}^M w_{(2)}^{(j)} f\left(W_{(1)}^{(j,0)} + W_{(1)}^{(j,1)} x\right) \end{aligned}$$

There is an example that is quite painful to make in latex so I will just say: page 31 lecture 9.

But the example is on the curve fitting problem introduced above. Instead now of having either an approximation using polynomial or any other feature expansion, we are now 'choosing' which model to **activate** or not **and by how much**.

$$\hat{y} = w_{(2)}^{(1)} f\left(W_{(1)}^{(1,0)} + W_{(1)}^{(1,1)} x\right) + w_{(2)}^{(2)} f\left(W_{(1)}^{(2,0)} + W_{(1)}^{(2,1)} x\right) + w_{(2)}^{(3)} f\left(W_{(1)}^{(3,0)} + W_{(1)}^{(3,1)} x\right) + \dots$$



Dealing with multiple input dimensions

Handling $D > 1$ input dimensions makes very little different in the model formulation. From the 1D input formulation we had

$$\hat{y} = w_{(2)}^T f\left(W_{(1)}^T \begin{pmatrix} 1 \\ x \end{pmatrix}\right)$$

In order to add dimensions – to the D case we literally add the dimension in $W_{(1)}$ which now is of dimension $(D + 1) \times M$ instead of $2 \times M$. (where $x \in \mathbb{R}^{D+1}$).

Dealing with multiple output dimensions

To handle $C > 1$ output dimensions, we can proceed as in the case of linear models. We simply need to replace the vector of parameters $w_{(2)}$ with a matrix $W_{(2)} \in \mathbb{R}^{M \times C}$. Therefore this prediction can be written as

$$\hat{\mathbf{y}} = W_{(2)}^T f\left(W_{(1)}^T x\right)$$

Simple artificial neural network

The resulting simple artificial neural network with one hidden layer can be depicted as shown below.

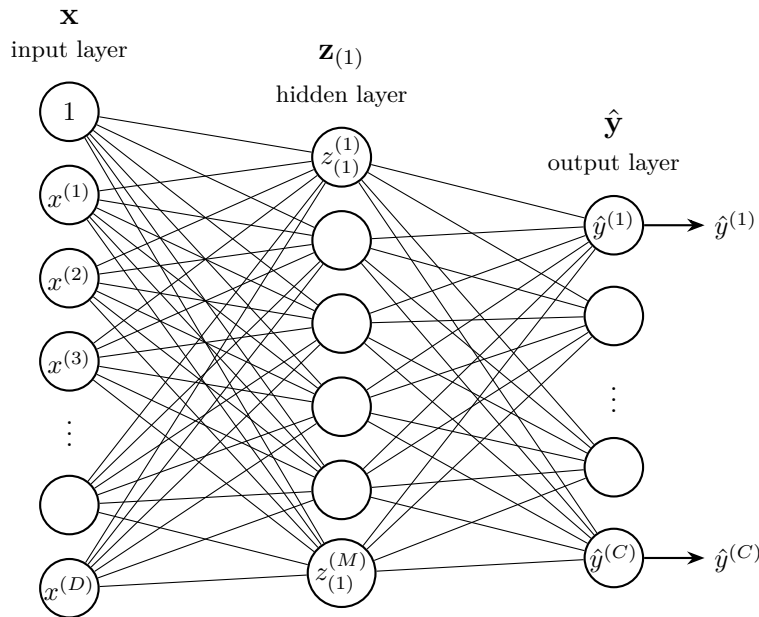


Figure 2.5: Artificial neural networks which I passed a bit too much time on

Because we use a matrix vector product for each layer, the layers are called **fully connected**.

A thing I found interesting here is how this relates to a classical linear models. If we were to also draw the diagram of a classic linear models meaning

$$\hat{y} = f\left(W_{(1)}^T x\right)$$

How would that look like? This architecture output would be the same as the one above except the hidden layer would have only **one** hidden unit.

As we can see here, even with a single hidden layer, one can already model complicated functions. The **Universal approximation theorem** states that a sufficiently large feed forward network with a non-linear activation can approximate any continuous function on a compact domain to arbitrary precision. So why would we care about adding more layers? The answer lies on how most real world function are defined. Most of them are not arbitrary continuous function, they are **compositions of simpler functions** which in the CNN section we will show that having deep networks naturally models it. But first to understand that let's us try and see what happens.

Multilayer Perceptron

In essence, we could just 'stack' horizontally layer of layer of neural network to construct a bigger one. This is what the Multilayer Perceptron does.

- For the first hidden layer, we still have

$$z_{(1)} = f\left(W_{(1)}^T x\right)$$

- For any subsequent layer l , we have

$$z_{(l)} = f\left(W_{(l)}^T z_{(l-1)}\right)$$

- Finally, for a network of L layer, we compute the output as

$$\hat{y} = W_{(L)}^T z_{(L-1)}$$

The process of going from the input to the output is called the **forward pass** of the network

If we wanted to write the prediction as a single equation it would look like:

$$\hat{y} = W_{(L)}^T f\left(W_{(L-1)}^T f\left(W_{(L-2)}^T \left(\cdots f\left(W_{(1)}^T x\right) \cdots\right)\right)\right)$$

This is not very convenient, and it is easier to simply think of $\hat{y}$ as a function of all parameters

$$\hat{y} = \hat{y}(W)$$

Where $W = \{W_{(l)}\}$ encompasses the parameters of all the layers.

Our model has been define and it looks very good on paper however there is still a big issue, the **training**. Given a set of N training sample $l(x_i, y_i)$, we would then like to find the best set of parameters W^* . As usual this is done by minimizing an empirical risk w.r.t. all the parameters W .

For regression, the loss function is typically the square loss

$$R(W) = \frac{1}{N} \sum_{i=1}^N \|\hat{y}_i(W) - y_i\|^2$$

Where the dependence of $\hat{y}_i$ on the parameters W is denoted as said before.

Training an MLP Because an MLP stacks multiple layers, each of which has a nonlinear activation function, the problem does not have a closed form solution and is not convex

Learning is therefore done by gradient descent. While the gradient might

Gradient based learning

seem complicated, it can in fact be computed in a nice manner (this is called backpropagation).

Because the empirical risk is an average over the training samples, its gradient will also be an average of gradients. Thus we can focus on a single loss term $l(\hat{y}_i, y_i)$, where

$$R(\{W_{(l)}\}) = \frac{1}{N} \sum_{i=1}^N l(\hat{y}_i, y_i) = \frac{1}{N} \sum_{i=1}^N l_i$$

For simplicity, we will assume that all hidden layers use the same type of activation function, denoted by f .

For the k^{th} dimension of the output of any layer l , let us define

$$a_{(l)}^{(k)} = \sum_j W_{(l)}^{(k,j)} z_{(l-1)}^{(j)}$$

Which denotes the dot product between the k^{th} column of $W_{(l)}$ and $z_{(l-1)}$. What we aim to compute is the derivative of the loss function l_i w.r.t. any parameter $W_{(l)}^{(k,j)}$. An important thing here to understand is what is $W_{(l)}^{(k,j)}$ recall that this is **an element** of the parameter of the model at the layer l , the parameter at the column k and row j . Following the chain rule, we can write such a derivative as

$$\frac{\partial l_i}{\partial W_{(l)}^{(k,j)}} = \frac{\partial l_i}{\partial a_{(l)}^{(k)}} \frac{\partial a_{(l)}^{(k)}}{\partial W_{(l)}^{(k,j)}}$$

As the definition stated above, we can see that

$$\frac{\partial a_{(l)}^{(k)}}{\partial W_{(l)}^{(k,j)}} = z_{(l-1)}^{(j)}$$

Furthermore, let us define

$$\delta_{(l)}^{(k)} = \frac{\partial l_i}{\partial a_{(l)}^{(k)}}$$

Which directly encodes the first term of the derivative. So now we can rewrite the derivative

$$\frac{\partial l_i}{\partial W_{(l)}^{(k,j)}} = \delta_{(l)}^{(k)} z_{(l-1)}^{(j)}$$

| *Little break* Remainder here to pause and check if I understood everything here.

Because the hidden values $\{z_{(l-1)}^{(j)}\}$ are computed in the forward pass, we only need to compute the $\{\delta_{(l)}^{(k)}\}$ to obtain the derivative of the loss w.r.t. any parameter.

For the last layer (Layer L), we have (assuming no final activation)

$$a_{(L)}^{(k)} = \hat{y}^{(k)}$$

This means that, for the last layer, $\delta_{(L)}^{(k)}$ can be computed explicitly as the partial derivative of the loss function w.r.t. the network's output. E.g., for the square loss, we have

$$\delta_{(L)}^{(k)} = 2(\hat{y}_i^{(k)} - y_i^{(k)})$$

Therefore for all other layers where $l < L$, it can be shown that $\delta_{(l)}^{(k)}$ can be computed from:

- the $\{\delta_{(l+1)}^{(j)}\}$, from the subsequent layer $l + 1$
- the derivative of the activation function w.r.t. its input
- the parameters $W_{(l+1)}$ (of the subsequent layer $l + 1$)

In other words, given $\delta_{(l+1)}$, one can compute $\delta_{(l)}$. And as we can explicitly compute $\delta_{(L)}$ for the last layer, using recursion, we can then compute δ for any layer. Using this fact we get that in order to compute the gradient, we can compute

$$\frac{\partial l_i}{\partial W_{(l)}^{(k,j)}} = \delta_{(l)}^{(k)} z_{(l-1)}^{(j)}$$

The algorithm works for computing the gradient at any layer. To do so we start from the last one:

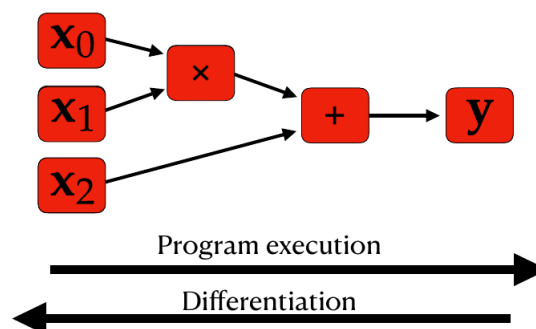
1. Propagate x_i forward through the network
2. Compute $\delta_{(L)}$ depending on the loss
3. Propagate $\delta_{(L)}$ backward to obtain each $\delta_{(l)}$
4. At each layer, compute the partial derivatives

$$\frac{\partial l_i}{\partial W_{(l)}^{(k,j)}} = \delta_{(l)}^{(k)} z_{(l-1)}^{(j)}$$

Where $z_{(l-1)}$ has been computed during the forward pass. Recall that $z_{(l-1)}$ represent the value at the given layer.

Little example on auto differentiation (not in the course)

Something that I want to emphasize here (that I found not clear) is what are we even doing here. So to explain a bit more I will use some stuff from the lecture about autodiff from cs-328. What we were doing above was auto differentiation, there is two main modes **forward**, **backward**. The one we saw above is backward. How it works is first execute the program and store each result and the computational graph structure (the image below) into a tape (this is our $z_{(i)}$ above). Then we go back and compute the differentiation using that tape:



The reason on why we need that tape is the chain rule:

$$\frac{df(g(x))}{dx} = \frac{df}{dg}(g(x)) \cdot \frac{dg(x)}{dx}$$

In the forward pass we are computing $g(x)$ and all intermediate values in the computation graph. We stores these values in a tape. Then when doing the backward pass. We already have everything needed in order to apply the chain rule efficiently. Another great video I recommend in order to better understand how it works

But let us see an example here:

Consider the function:

$$z = f(x, y) = (x + y) \cdot y$$

We decompose it into elementary operations:

$$\begin{aligned} a &= x + y \\ b &= a \cdot y \end{aligned}$$

So the computation graph is:

$$x, y \rightarrow a \rightarrow b$$

Forward pass We compute and store intermediate values:

$$\begin{aligned} a &= x + y \\ b &= a \cdot y \end{aligned}$$

These values are stored in a tape.

Backward pass We start from the output:

$$\frac{db}{db} = 1$$

Now apply the chain rule backwards.

Step 1: propagate through multiplication $b = a \cdot y$

$$\frac{db}{da} = y, \quad \left. \frac{db}{dy} \right|_{\text{path 1}} = a$$

Step 2: propagate through addition $a = x + y$

$$\frac{da}{dx} = 1, \quad \frac{da}{dy} = 1$$

Now combine contributions:

$$\begin{aligned} \frac{db}{dx} &= \frac{db}{da} \cdot \frac{da}{dx} = y \cdot 1 = y \\ \frac{db}{dy} &= \underbrace{\left. \frac{db}{dy} \right|_{\text{direct}}}_a + \underbrace{\frac{db}{da} \cdot \frac{da}{dy}}_{y \cdot 1} = a + y \end{aligned}$$

Gradient descent Once the gradient is computed, the update at iteration k is performed as

$$W_k \leftarrow W_{k-1} - \mu \nabla_W R(W_{k-1})$$

Where W encompasses the parameters of all layers

You can also think of this as updating the parameters in each layer l as

$$W_{(l),k} \leftarrow W_{(l),k-1} - \mu \nabla_{W_{(l)}} R(W_{k-1})$$

Where $W_{(l),k}$ represents the parameters of layer l at iteration k of gradient descent.

The empirical risk is the average of the loss function value over all training samples. Because the gradient of an average is the average of the gradients. We can

write also a standard gradient update as

$$W_k \leftarrow W_{k-1} - \mu \frac{1}{N} \sum_{i=1}^N \nabla_W l_i(W_{k-1})$$

this means that, for each gradient descent iteration, one needs to perform a forward pass and a backward pass through the network for all the training samples. This becomes expensive for large training sets.

Stochastic gradient descent

A way in order to solve this issue is to not use all the training set when doing a gradient descent. But a big issue comes to us: Which subset should we take? Let us take a random subset of our data set right? But how small, at this point let us just take one random point and do the iterations of the gradient descent on only one point. This is called **Stochastic gradient descent**.

Definition

Stochastic gradient descent updates the parameters W based on the gradient of a single sample in each iteration

$$W_k \leftarrow W_{k-1} - \mu \nabla l_{i(k)}(W_{k-1})$$

At each iteration, we use the gradient of a single loss term. The index of the sample depends on the iteration number k .

In practice, one then iterates over the training samples, either in fixed order or in a random one.

SGD with mini-batches

The gradient obtained by a single sample might be a poor approximation of the true, complete gradient. To obtain more reliable gradient estimate, one typically uses mini-batches of B samples (this is what we said above). The parameters are then updated as

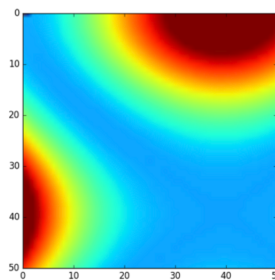
$$W_k \leftarrow W_{k-1} - \mu \sum_{b=1}^B \nabla l_{i(b,k)}(W_{k-1})$$

The index of the samples used for the update now depends on the iteration number k and on the index b of the samples in the mini-batch

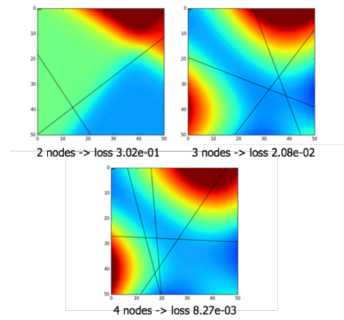
Simple network training: Regression example

The example that we will take now is a model which takes as input a 2D coordinate and output a 1D coordinate. The function that we are trying to model is given by

$$y = 100 \left(x^{(2)} - \left(x^{(1)} \right)^2 \right)^2 + \left(1 - x^{(1)} \right)^2$$



We will try different models, respectively with 2, 3, 4 nodes which gives us:

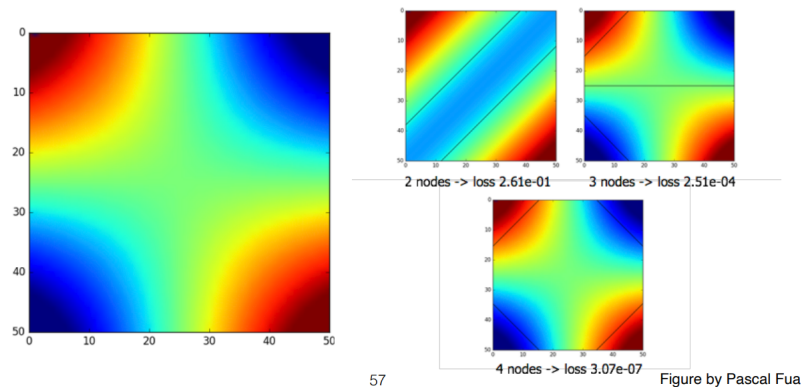


Nodes

Just to have more context a node (also called a neuron or unit) is a single computational element. Formally, it takes an input vector x , computes an affine transformation $w^T x + b$, and then applies a nonlinearity (e.g., ReLU, sigmoid). So one node corresponds to one scalar output.

An example that allows us to have a better interpretation on how does neuron will do is to use the surface/output

$$y = \sin(x^{(1)}) \sin(x^{(2)})$$



57

Figure by Pascal Fua

However even if here in this case we see why it works, usually we don't actually know why they work, meaning that we don't know:

- What do the coefficients actually mean?
- Will the network generalize to other simulations
- Why does the training process actually work?

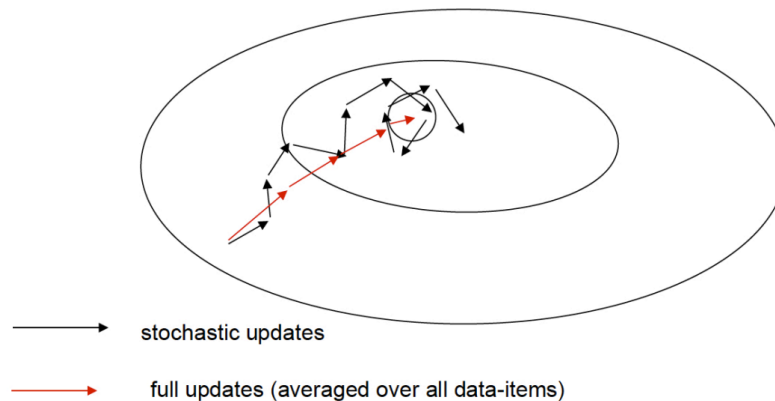
What we know:

- How all the coefficients were obtained (as it is a deterministic calculation after all)
- Which neural network components work well, and which of them work well together

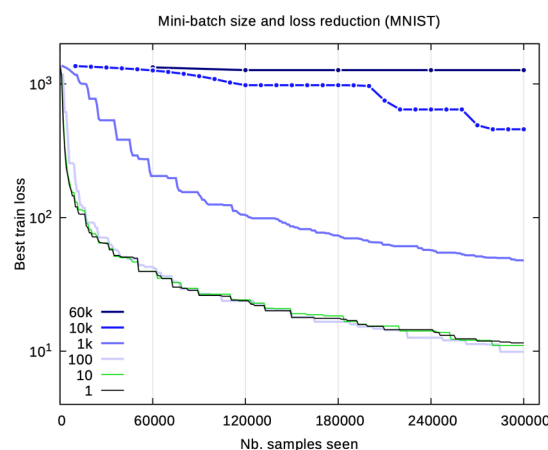
SGD behaviour

Because it approximates the full gradient, SGD will move the parameters less directly towards a local minimum.

- The parameters dance around the minimum, which may require decreasing the learning rate
- Nevertheless, SGD can help to avoid bad local minima



To illustrate how 'important' using mini batch is when doing gradient descent based optimization, let us see this graph



Gradient descent: Gradient descent makes strong assumption about the magnitude and isotropy of the local curvature so that the same step size can be used in all directions. What we mean by that is that in order for our algorithm to work well we need our function to behave nicely. (there is a couple of example of gradient descent in the slide p.64 lecture 9). For instance imagine if we have a very local minima then our function might get stuck there instead of rolling out away. What we are lacking here is **momentum**

SGD Extension The idea we have is to add inertia to the parameter updates

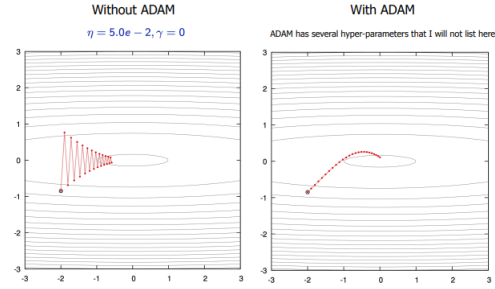
$$\begin{aligned}\nu &\leftarrow \gamma\nu + \mu\nabla R(W) \\ W &\leftarrow W - \nu\end{aligned}$$

With $\gamma > 0$, the moment accelerates if the gradient does not change too much.

As you could have guessed there is a lot of SGD extensions out there some uses adaptive learning rate:

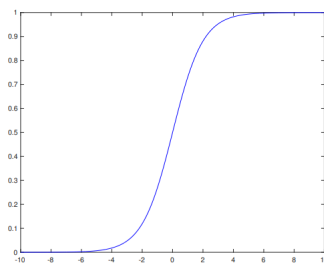
- Adagrad (Duchi et al, JMLR 2011)
- Adadelata (Zeiler, 2012)
- RMSprop (Hinto, 2012)
- ADAM (Kingma et al., ICLR 2015)

For example, ADAM uses a moving average of each coordinate to rescale each coordinate separately.

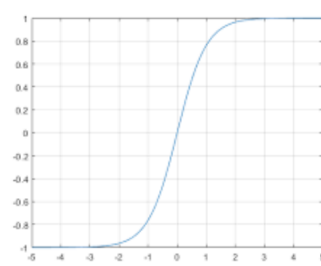


Gradient vanishing & Initialization

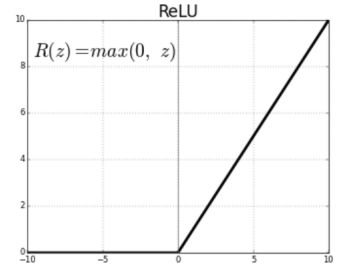
The activation functions often have saturating regions where the gradient becomes (close to) 0



Sigmoid



tanh



ReLU

Glorot and Bengio observed that the gradient vanishes exponentially with the (inverse) depth if the weights are such that they tend to often saturate the activation function. '*saturate the activation function*' refers to regions where the derivative is very small (e.g., sigmoid saturates near 0 or 1, tanh near -1 or 1). Now let us go back to backpropagation. Gradients are computed using the chain rule. For a deep network with L layers, a gradient term typically looks like a product:

$$\frac{\partial \mathcal{L}}{\partial x} = \prod_{l=1}^L \frac{\partial x^{(l)}}{\partial z^{(l-1)}}$$

Each factor contains a weight matrix and the derivative of the activation function. If this activation is saturated, those derivative are small number. So we end up multiplying $0.1 \cdot 0.1 \cdot 0.1 \cdot \dots$. This shrinks exponentially with depth. The gradient becomes extremely close to 0 as the number of layers increases.

To overcome this, Glorot and Bengio introduced a weight initialization strategy where the range of the initial random values of the weights is layer dependent.

From regression to classification

So far, we have focused on the case where the prediction is taken directly as the output of a linear model applied to the last hidden representation, i.e.,

$$\hat{y} = W_{(L)}^T z_{(L-1)}$$

While this is fine for regression task, it is not ideal for a classification one, where we would like each $\hat{y}^{(k)}$ to represent the probability (score) for class k . This can be achieved via the softmax function (as in multi-class logistic regression)

$$\hat{y}^{(k)} = \frac{\exp(w_{(L)(k)}^T z_{(L-1)})}{\sum_{j=1}^C \exp(w_{(L)(j)}^T z_{(L-1)})}$$

Where $w_{(L)(j)}$ indicates the j^{th} column of $W_{(L)}$.

In this case, the empirical risk is typically defined as the cross entropy

$$R(W) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^C \hat{y}_i^{(k)} \ln \hat{y}_i^{(k)}$$

Backpropagation then requires computing the derivative of the cross-entropy w.r.t. each $\hat{y}^{(k)}$ and the derivative of the softmax function w.r.t. each $W_{(L)}^{(j,k)}$.

Remark Note that this is similar to the derivatives computed for multi-class logistic regression.

Working with images

Treating an images as a big vector seems counter-intuitive.

- An image of reasonable size yield a huge vector, thus a huge amount of parameters in an MLP
- Small image regions have similar property and should not be processed independently
- A translation should not affect results

2.12 Convolutional Neural Network

The goal of today lecture (and this section) is to being able to work with images. As said before, an image we reasonable size yields a huge vector, thus a huge amount of parameters in an MLP. Therefore we need a way to 'reduce' the size of this vector will still keeping as much information as possible. To do so we can first see the fact that small image regions have similar property (colour, ...) and therefore should not be processed independently. Another important thing that we need when treating images is that a translation should not affect the results. Using all of those, can we reduce the number of parameters and improve robustness?

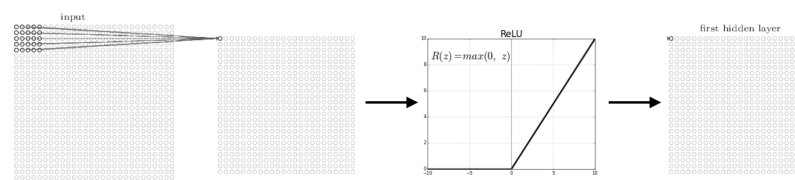
CNN

To do so, we will introduce **Convolutional Neural Network** (CNN) model and its different types of layers. We will also introduce the idea of transposed convolutions for fully convolutional networks.

Convolutional Layer

Normally if we recall the scalashop lab from cs-214, we have already seen what convolution is. The goal was to use a kernel which took as input all the neighbouring pixels and use them to output one pixel. Here this is what we will do, we will take square of input neurons and output only one neurons onto the next hidden layer. And as in cs-214, the parameters are shared across different spatial locations, meaning that the 'loop' still goes parameter by parameter (not by grid of parameters).

Note that each convolutional output is passed in an activation function. So, in fact, we have



We can then use multiple filters to create multiple channels (3 in the example below). As in MLP each filter has its own set of learnable parameters (in the example, we have 3 5×5 filters, i.e., 75 learnable parameters)

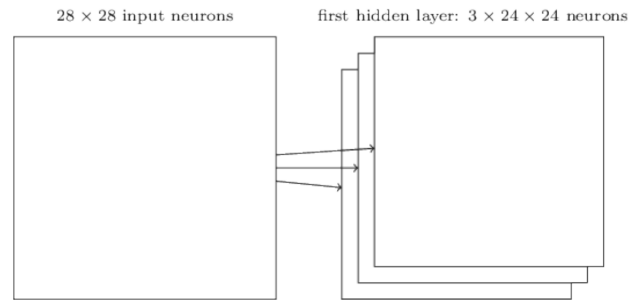


Figure from M. Nielsen's book

Pooling layer

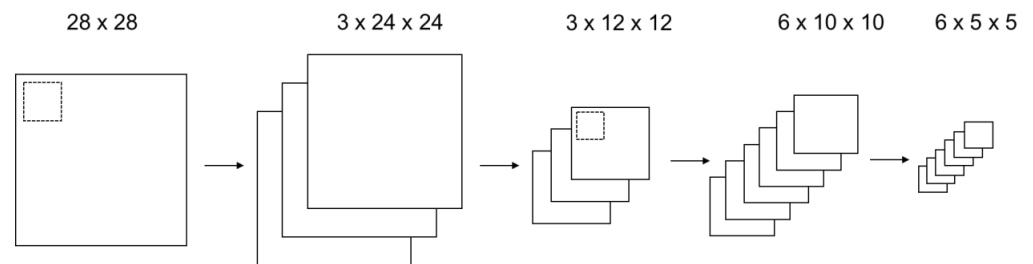
This is nice, but how do we even choose what the output of the convolution is? How do we go from local information to a more global representation? There exists different pooling strategies:

- Max pooling
- Average pooling

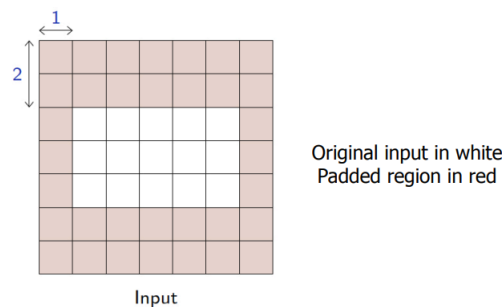
Using that, we can stack these operations together and then stack multiple **block** of such operations.

Remark

- The number of channels can vary (below: first 3, then 6)
- The size of filters can vary (below: first 5×5 , then 3×3)

**Padding**

To be able to center the convolution kernel on the boundary locations, padding extends the input. This can be done by mirroring or copying the boundary values.

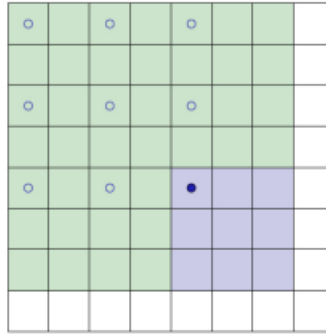
**Convolution Stride**

In a standard convolution, the filter is shifted pixel by pixel (as said before). The can be made larger by increasing the stride.

Having a larger stride gives us less parameters, the output layer will be smaller but at the cost of information.

Discussion: Convolution stride

The goal of strided convolution is to play a similar role to pooling, i.e., decrease the spatial dimensions, but with learnable parameters. However this can have some issues: as if the stride is greater than 1, we may not cover the input map completely, hence ignoring some of the input values.



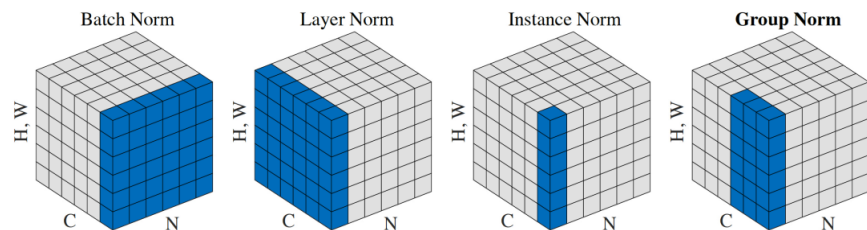
Output

This is nice, but what should we actually output? In general, we usually want to output a vector e.g.,

- A vector of class probabilities for classification
- A vector of 3D joint coordinates for human pose estimation

This can be achieved by appending fully connected layer(s). This requires vectorizing the last convolutional output first ($3 \times 12 \times 12 \rightarrow 432$ -dimensional vector). Inspired by the fact that data normalization can help, several layers have been developed to normalize the feature maps, these layers include:

- Batch normalization (Ioffe & Szegedy, ICML 2015)
- Layer normalization (Ba et al., 2016)
- Instance normalization (Ulyanov et al., 2016)
- Group normalization (We & He, 2018)



N is the number of samples in a mini batch and C is the number of channels; (H, W) is the size of the feature map.

CNN: Learning

Learning the weights of the CNN layers is done in the same manner as for MLPs, we use stochastic gradient descent with mini batches. As we need derivative to do so, we will still use backpropagation in order to compute those.

The gradient of pooling layers depends on the strategy:

- For average pooling, the gradient is backpropagated to all neurons used during pooling
- For max pooling, the gradient is backpropagated only to the neuron that corresponded to the maximum value

And we will also use all the techniques we have used before such as SGD with momentum, adaptive learning rates, Early stopping weight decay, dropout, ...

With CNN, this separation disappears, The network learns both parts simultaneously

- Early layers learn **low-level** representation (edges, textures)
- Middle layers learn **patterns** (shapes, parts of objects)
- Deep layers learnt **semantic** (faces, cars, etc.)
- Final layers act as the **classifier/regressor**

The pipeline becomes

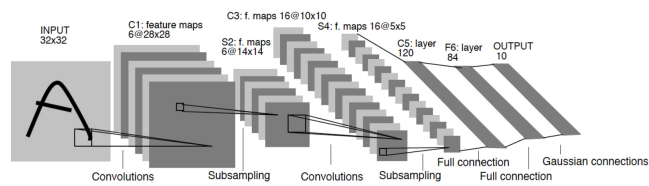
raw image $\rightarrow$ learned feature $\rightarrow$ prediction

Crucially, the features are not fixed, they are optimized **together** with the final objective

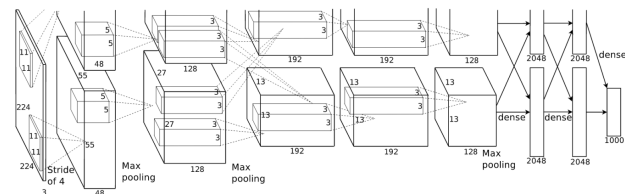
The network alone is saying '*I will invent the features that make my classification easiest*'

Standard architecture There exists a lot of different architecture here is a couples:

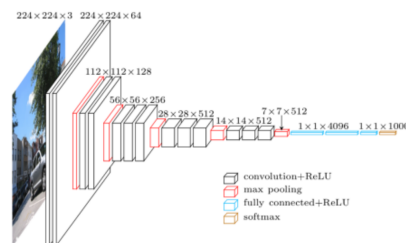
LeNet-5



AlexNet

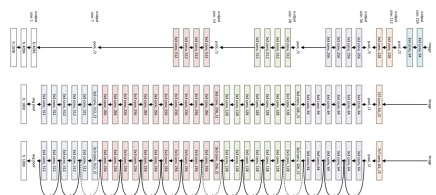


VGG

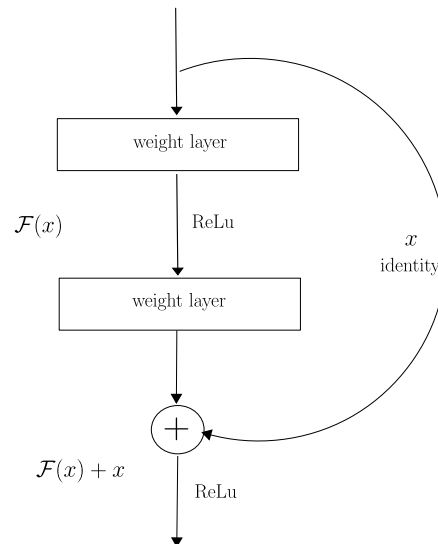


The standard architecture here is **ResNet**

ResNet



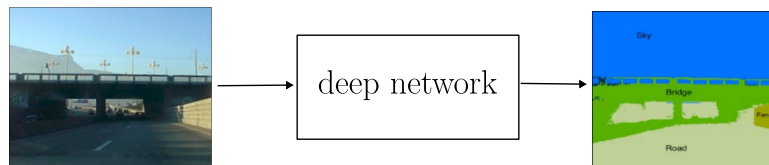
How it works is by adding to each at each layer the result of the previous layer:



All of those are image classification networks, so their practical use are similar. The differences are mainly about when you'd pick one over another. ResNet is the one that gets used in practice, for image classification, backbone inside detection/segmentation models, transfer learning starting point.

Tackling different tasks

When working with images, one does not necessarily want to simply classify them. For instance, semantic segmentation is the task of assigning a label to every pixel in the input image



CNNs for semantic segmentation

However with the semantic segmentation as with convolutions and pooling layers the size of the feature maps decreases at every layer, with the operations seen before, we have no way of increasing this size back to the original resolution.

We have found a way to go from an image to label, but not from label to class. A solution for this is Upsampling, followed by convolutions. To do so, let us first study the 1D convolution. For instance let us have the input:

$$\underbrace{(1 \quad 4 \quad -1 \quad 0 \quad 2 \quad -2 \quad 1 \quad 3 \quad 3 \quad 1)}_W$$

and the Kernel:

$$\underbrace{(1 \quad 2 \quad 0 \quad -1)}_w$$

If we were to compute the output, this would give us:

$$\text{Output} = (9 \quad 0 \quad 1 \quad 3 \quad -5 \quad -3 \quad 6)$$

In essence, a 1D convolution maps a vector (of dimension W in this example) to dimension $W - w + 1$, increasing the dimensionality can be achieved by using the transposed matrix

$$\begin{pmatrix} w_1 & w_2 & w_3 & 0 & 0 & 0 \\ 0 & w_1 & w_2 & w_3 & 0 & 0 \\ 0 & 0 & w_1 & w_2 & w_3 & 0 \\ 0 & 0 & 0 & w_1 & w_2 & w_3 \end{pmatrix}^T = \begin{pmatrix} w_1 & 0 & 0 & 0 \\ w_2 & w_1 & 0 & 0 \\ w_3 & w_2 & w_1 & 0 \\ 0 & w_3 & w_2 & w_1 \\ 0 & 0 & w_3 & w_2 \\ 0 & 0 & 0 & w_3 \end{pmatrix}$$

Therefore using that on the kernel:

$$k = \begin{pmatrix} 1 & 2 & -1 \end{pmatrix}$$

and the input

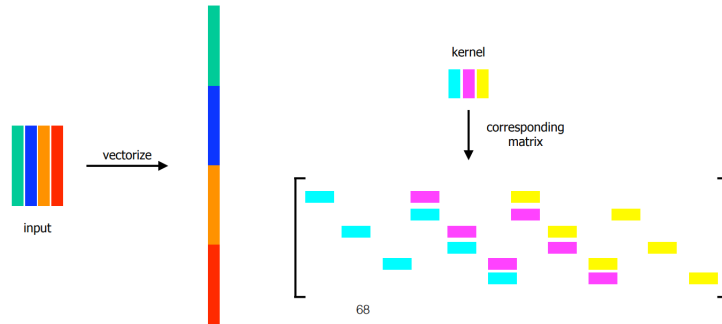
$$I = \begin{pmatrix} 2 & 3 & 0 & -1 \end{pmatrix}$$

The output is given as:

$$\begin{pmatrix} 2 & 7 & 4 & -4 & -2 & 1 \end{pmatrix}$$

2D convolution as matrix product

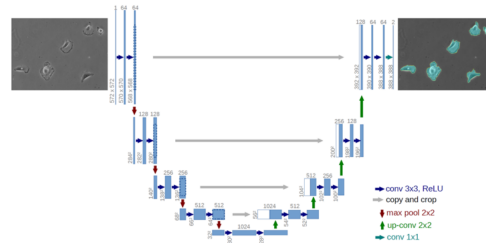
A similar matrix representation can be obtained for 2D convolutions. This input is vectorized, e.g., by concatenating its columns. The matrix is obtained by spreading the different columns of the kernel at the matching locations



Fully convolutional networks

Stacking upsampling convolutions, or transposed convolutions, after regular ones lets us go back to the original spatial resolution.

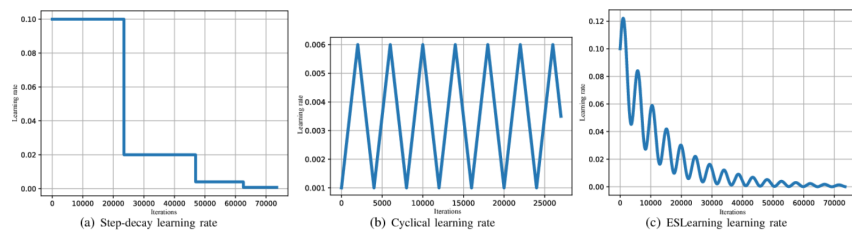
- For binary semantic segmentation, the output is a binary mask of the same size as the image (here overlaid on top of the input)
- Ronneberger et al., U-Net: Convolutional Networks for Biomedical Image Segmentation, MICCAI 2015



Deep Learning: Tricks of the trade

In order to have better performance and be less costly in term of performance, we can use a couple of techniques

- Pre-training: People often start with a standard architecture trained on ImageNet
- Learning rate scheduling: Decrease the learning rate after a pre-defined number of epochs
- Use a cyclic learning rate



- Data augmentation e.g. for images, flip, rotate, affine transform,...

2.12.1 CNN BackPropagation

This subsection is from the slides given in the solution of the exercises 10, the reason for putting it there is that I don't think this materials was taught in class and as far as I know it might be on the exam, so...

Backpropagation in CNNs In the backward pass, we get the loss gradient with respect to the next layer. However, in CNNs the loss gradient is computed w.r.t. the input and also w.r.t. the filter. This is the reason why we had to create the function g_K, g_X in the exercises.

Convolution Backprop with single Stride To understand the computation of loss gradient w.r.t input, let us use the following example:

$$X = \begin{pmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{pmatrix}$$

And the kernel, Filter

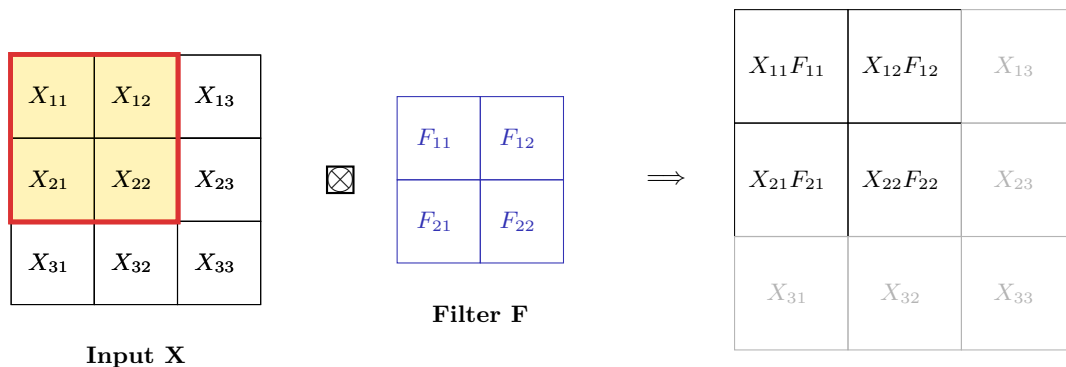
$$F = \begin{pmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{pmatrix}$$

Where we have a horizontal and vertical stride of 1.

In the forward pass, the convolution between the input X and the filter F gives us the output O , which can be represented as

$$\begin{pmatrix} O_{11} & O_{12} \\ O_{21} & O_{22} \end{pmatrix} = \text{Conv} \left(\begin{pmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{pmatrix}, \begin{pmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{pmatrix} \right)$$

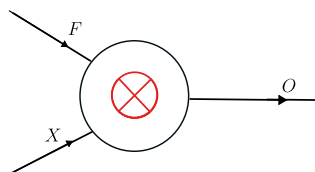
Then for instance if we were to compute O_{11} this would be by using the following



Which gives us

$$O_{11} = X_{11}F_{11} + X_{12}F_{12} + X_{21}F_{21} + X_{22}F_{22}$$

Loss gradient Now we want to calculate the gradients w.r.t. the input X and filter F .



To do so we can use the chain rule to obtain the gradient w.r.t. the filter

$$\frac{\partial L}{\partial F} = \frac{\partial L}{\partial O} \frac{\partial O}{\partial F}$$

And for every element of F :

$$\frac{\partial L}{\partial F_i} = \sum_{k=1}^M \frac{\partial L}{\partial O_k} \frac{\partial O_k}{\partial F_i}$$

By replacing the local gradient of the filter i.e., $\frac{\partial O_i}{\partial F_i}$ we get

$$\begin{pmatrix} \frac{\partial L}{\partial F_{11}} & \frac{\partial L}{\partial F_{12}} \\ \frac{\partial L}{\partial F_{21}} & \frac{\partial L}{\partial F_{22}} \end{pmatrix} = \text{Conv} \left(\begin{pmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{pmatrix}, \begin{pmatrix} \frac{\partial L}{\partial O_{11}} & \frac{\partial L}{\partial O_{12}} \\ \frac{\partial L}{\partial O_{21}} & \frac{\partial L}{\partial O_{22}} \end{pmatrix} \right)$$

If we closely look at it, this represents an operation that we are quite familiar with. We can represent it as a convolution operation between input X and loss gradient $\frac{\partial L}{\partial O}$ as shown here

$$\frac{\partial L}{\partial X_i} = \sum_{k=1}^M \frac{\partial L}{\partial O_k} \frac{\partial O_k}{\partial X_i}$$

First let us rotate the Filter F by 180 degrees. This is done by flipping it first vertically and then horizontally

$$\begin{pmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{pmatrix} \rightarrow \begin{pmatrix} F_{12} & F_{11} \\ F_{22} & F_{21} \end{pmatrix} \rightarrow \begin{pmatrix} F_{22} & F_{21} \\ F_{12} & F_{11} \end{pmatrix}$$

Now we can see that the loss gradient w.r.t. the input $\frac{\partial L}{\partial X}$ is given as a full convolution between the filter (rotated filter) and Loss gradient $\frac{\partial L}{\partial O}$. It is important here to mention the [full](#).

If we were to take the example with the 3×3 input matrices recall that for each X_{ij} the derivative is given as:

$$\begin{aligned} \frac{\partial L}{\partial X_{11}} &= \frac{\partial L}{\partial O_{11}} F_{11} \\ \frac{\partial L}{\partial X_{12}} &= \frac{\partial L}{\partial O_{11}} F_{12} + \frac{\partial L}{\partial O_{12}} F_{11} \\ \frac{\partial L}{\partial X_{13}} &= \frac{\partial L}{\partial O_{12}} F_{12} \\ \frac{\partial L}{\partial X_{21}} &= \frac{\partial L}{\partial O_{11}} F_{21} + \frac{\partial L}{\partial O_{21}} F_{11} \\ \frac{\partial L}{\partial X_{22}} &= \frac{\partial L}{\partial O_{11}} F_{22} + \frac{\partial L}{\partial O_{12}} F_{21} + \frac{\partial L}{\partial O_{21}} F_{12} + \frac{\partial L}{\partial O_{22}} F_{11} \\ \frac{\partial L}{\partial X_{23}} &= \frac{\partial L}{\partial O_{12}} F_{22} + \frac{\partial L}{\partial O_{22}} F_{12} \\ \frac{\partial L}{\partial X_{31}} &= \frac{\partial L}{\partial O_{21}} F_{21} \\ \frac{\partial L}{\partial X_{32}} &= \frac{\partial L}{\partial O_{21}} F_{22} + \frac{\partial L}{\partial O_{22}} F_{21} \\ \frac{\partial L}{\partial X_{33}} &= \frac{\partial L}{\partial O_{22}} F_{22} \end{aligned}$$

Using that, We can see that

$$\frac{\partial L}{\partial X} = \text{Full Conv} \left(\begin{pmatrix} F_{22} & F_{21} \\ F_{12} & F_{11} \end{pmatrix}, \begin{pmatrix} \frac{\partial L}{\partial O_{11}} & \frac{\partial L}{\partial O_{12}} \\ \frac{\partial L}{\partial O_{21}} & \frac{\partial L}{\partial O_{22}} \end{pmatrix} \right)$$

Takeaway What is important here is that both forward pass and the Backpropagation of a convolutional layer are **convolutions**

$$\frac{\partial L}{\partial F} = \text{Conv}\left(X, \frac{\partial L}{\partial O}\right)$$

$$\frac{\partial L}{\partial X} \text{Full Conv}\left(180^\circ F, \frac{\partial L}{\partial O}\right)$$

Stride > 1 Okay this is nice, but now what happen when $s > 1$? For instance let us have a input matrix with only 1 channel with height and width 5. The kernel is a 3×3 of size with both vertical and horizontal stride of 2. The output of that convolution would be a 2×2 Y

$$Y = \begin{pmatrix} y_{00} & y_{01} \\ y_{10} & y_{11} \end{pmatrix}$$

Backward Pass We have already seen how the forward pass would look like, so let us go directly into the backward pass. The assumption we will have is that we have the loss gradient w.r.t. the output pixels. meaning we have

$$\begin{pmatrix} \frac{\partial L}{\partial y_{00}} & \frac{\partial L}{\partial y_{01}} \\ \frac{\partial L}{\partial y_{10}} & \frac{\partial L}{\partial y_{11}} \end{pmatrix}$$

and we want to do into the loss gradient w.r.t. the input (of size 5×5).

As each input contributes to one or more outputs. The total gradient of the loss w.r.t. each input pixel is computed using

$$\frac{\partial L}{\partial x_m n} = \sum_{ij} \frac{\partial L}{\partial y_{ij}} \frac{\partial y_{ij}}{\partial x_{mn}}$$

The gradient computation is done using the chain rule where i and j represent the position of a single output pixel.

Example

Let us consider the input x_{00} , this input contributed to the output y_{00} . On the other hand the input x_{01} contributed to y_{00} therefore, the loss gradient w.r.t. x_{01} is computed as

$$\frac{\partial L}{\partial x_{01}} = \frac{\partial L}{\partial y_{00}} f_{01}$$

Now the input x_{02} contributed to the output y_{00} and y_{01} so the loss gradient w.r.t. x_{02} is computed as shown

$$\frac{\partial L}{\partial x_{02}} = \frac{\partial L}{\partial y_{00}} f_{02} + \frac{\partial L}{\partial y_{01}} f_{00}$$

To visualize the pattern more clearly, we pad the gradient tensor with zeros at the top and bottom as well as to the left and right. The number of zeros padded on either side is equal to the stride (horizontal and vertical). We also dilate the output gradient pixels with the stride (vertically and horizontally)

And as in the previous example we rotate the filter vertically and horizontally and using those two tricks, convolving with a stride greater than 1 is the same as convolving with a stride of 1 and 'dropping' out every rows, and every columns that doesn't count in the stride.

Chapter 3

Unsupervised Linear ML

3.1 Dimensionality Reduction

The goal of this section is to introduce our first unsupervised learning task: **Dimensionality Reduction**. To do so we will introduce a linear dimensionality reduction technique, principal component analysis, and its kernel extension. We will also introduce autoencoders.

Until now: Supervised learning

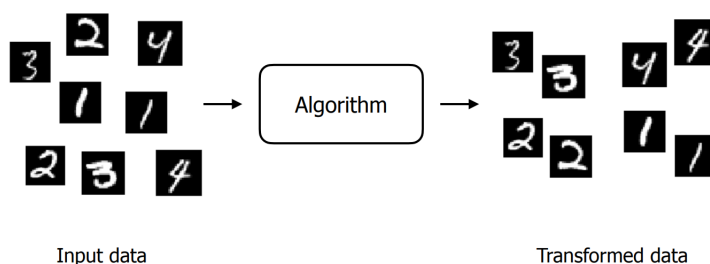
Until this moment our model worked in two stages

- **Stage 1:** Training, we use the data with ground truth labels to optimize model parameters
- **Stage 2:** Testing, we predict the output for a new data sample (with the fixed parameters)

Supervised learning is good when we have a concrete prediction task to solve and labelled data. However it is not suited when we don't have labels and when we are trying to get an understanding of the data.

Unsupervised learning

Today, we will do it differently, we will have only a single state which transforms the data for further analysis.



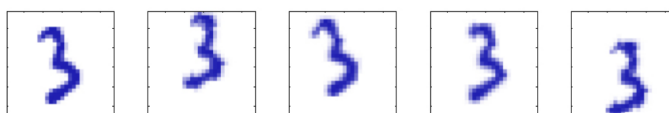
Dimensionality Reduction

To have a better understanding let us introduce it via an example, **data size reduction**. Let us have a vectorized image, a huge vector:

$$x_i \in \mathbb{R}^{536 \cdot 356 \cdot 3} = \mathbb{R}^{572448}$$

Dealing with such large data is cumbersome, is there a way for us to compress it while keeping the relevant information?

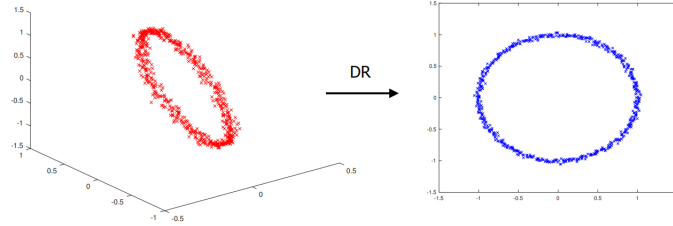
Another example would be by images created by translating and rotating the same digit 3



This data lies in a high dimensional space (32×32). However there is only 3 degrees of freedom here (2 translations and 1 rotation). Inherently this data lies in a 3 dimensional space. In practice, the data often lies in a much lower dimensional space, referred to as manifold.

Definition 23. Dimensionality Reduction is the task of
Discovering the data manifold

- Getting a low-dimensional representation of the data
 - Sometimes at some loss of information (hopefully loss)



Therefore we have as input $x_i \in \mathbb{R}^D$ and as output $y_i \in \mathbb{R}^d$ where $D \gg d$. Our goal is to find a mapping

$$y_i = f(x_i)$$

Let us try first with a linear one:

$$y_i = W^T x_i$$

Principal Component Analysis (PCA)

Given N samples $\{x_i\}$, PCA yields a projection of the form

$$y_i = W^T(x_i - \bar{x}) \quad \text{s.t. } W^T W = I_d$$

where $\bar{x}$ is the mean of the data

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

Okay sure why not, but the data is **unsupervised** so we don't have and other test label y_i that we can refer to in order to train our model. The question that this brings is: *What do we want this projection to achieve?*

PCA: Objective

We want two things

- Keep most of the 'important' signal
- Remove the noise

This can be achieved by finding directions that have a large variance, i.e., for the j^{th} output dimension, we want to maximize

$$\text{var}\left(\left\{y_i^{(j)}\right\}\right) = \frac{1}{N} \sum_{i=1}^N \left(y_i^{(j)} - \bar{y}^{(j)}\right)^2$$

Where $\bar{y}^{(j)}$ is the mean of the j^{th} dimension of the data after projection.

Variance maximization

Let us look at the projection to a 1D space. That is, we use a vector D -dimensional $w_{(1)}$ such that $w_{(1)}^T w_{(1)} = 1$ instead of a matrix $W \in \mathbb{R}^{D \times d}$

In this case, the mean of the data after projection is

$$\begin{aligned}\bar{y} &= \frac{1}{N} \sum_{i=1}^N y_i = \frac{1}{N} \sum_{i=1}^N w_{(1)}^T (x_i - \bar{x}) \\ &= w_{(1)}^T \left(\frac{1}{N} \sum_{i=1}^N x_i \right) - w_{(1)}^T \bar{x} = 0\end{aligned}$$

Then for the variance of the data projection,

$$\begin{aligned}\text{var}(\{y_i\}) &= \frac{1}{N} \sum_{i=1}^N (y_i - \bar{y})^2 = \frac{1}{N} \sum_{i=1}^N \left(w_{(1)}^T (x_i - \bar{x}) - 0 \right)^2 \\ &= \frac{1}{N} \sum_{i=1}^N \left(w_{(1)}^T (x_i - \bar{x}) \right)^2 = \frac{1}{N} \sum_{i=1}^N w_{(1)}^T (x_i - \bar{x}) (x_i - \bar{x})^T w_{(1)} \\ &= w_{(1)}^T \left(\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x}) (x_i - \bar{x})^T \right) w_{(1)} = w_{(1)}^T C w_{(1)}\end{aligned}$$

Where C is the input data covariance matrix

$$C = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x}) (x_i - \bar{x})^T$$

Therefore what we seek to solve is the problem

$$\begin{aligned}\max_{w_{(1)}} & w_{(1)}^T C w_{(1)} \\ \text{s.t. } & w_{(1)}^T w_{(1)} = 1\end{aligned}$$

To do so, we can write the Lagrangian of this problem (don't ask why)

$$\mathcal{L}(w_{(1)}, \lambda_1) = w_{(1)}^T C w_{(1)} + \lambda_1 (1 - w_{(1)}^T w_{(1)})$$

Setting the gradient of the Lagrangian w.r.t. $w_{(1)}$ to 0 yields

$$C w_{(1)} = \lambda_1 w_{(1)}$$

This is the definition of an eigenvector. Therefore $w_{(1)}$ must be an eigenvector of C , with eigen value λ_1

Remark 17. Let us just do the calculation:

$$\nabla_w \mathcal{L} = 2Cw - 2\lambda w = 0 \implies Cw = \lambda w$$

However which eigenvector? By multiplying both sides of the eigenvector equation from the left by $w_{(1)}^T$ yields

$$w_{(1)}^T C w_{(1)} = \lambda \underbrace{w_{(1)}^T w_{(1)}}_{=1} = \lambda_1$$

The resulting term of the left hand side is the variance of the projected data. Since we seek to maximize it, we should take $w_{(1)}$ as the eigen vector corresponding to the largest eigenvalue λ_1

To obtain an output representation that is more than 1D, i.e., $d > 1$, we can perform recursively

**Dealing with
more than 1D
projections**

- The second projection vector $w_{(2)}$ corresponds to the eigenvector of C with the second largest eigenvalue
- The third vector $w_{(3)}$ with the eigenvector with the third largest eigenvalue
- ...

The matrix W is obtained by concatenating the resulting vectors

$$W = [w_{(1)} \quad w_{(2)} \quad \cdots \quad w_{(d)}] \in \mathbb{R}^{D \times d}$$

this guarantees to satisfy the constraint $W^T W = I_d$ (as eigenvectors of a matrix are orthogonal and of norm 1)

In the limit, one can use all dimensions, i.e., set $d = D$, this implies

- There is therefore no reduction of dimensionality
- In 3D, we can think of this as a rotation of the data
- This incurs no loss of information
- The $d = D$ dimensions in the new space are uncorrelated

Another option is to keep all the eigenvectors that correspond to non-zero eigenvalues.

- This means that the data is truly low-dimensional
- E.g., this happens when we have fewer samples than dimensions ($N < D$)
- The resulting $\{y_i\}$ are lower dimensional ($d < D$)
- But there is no loss of information.

In practice, one typically truncates the eigenvalues so as to discard some that are non-zero

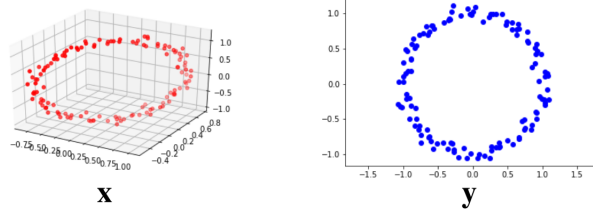
- This can be achieved by aiming to retain a pre-defined percentage of the data variance, measured as the sum of eigenvalues
- E.g. to retain at least 90% of the variance, one can search for d such that

$$\sum_{j=1}^d \lambda_j \geq 0.9 \sum_{k=1}^D \lambda_k$$

Assuming the eigenvalues are sorted in decreasing order. The resulting $\{y_i\}$ have an even lower dimension.

Error minimization

This incurs some loss of information



However the corresponding matrix W is the orthogonal matrix that minimizes the reconstruction error

$$e_i = \|\hat{x}_i - x_i\|^2$$

where

$$\hat{x}_i = \bar{x} + W y_i = \bar{x} W W^T (x_i - \bar{x})$$

Mapping

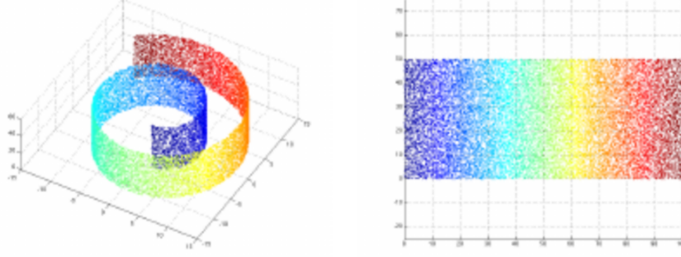
PCA not only reduces the dimensionality of the original data. It provides a continuous mapping from the low-dimensional space to the high-dimensional one. That is, for any $y \in \mathbb{R}^d$, we can compute a point in the high-dimensional space as

$$\hat{x} = \bar{x} + W y$$

This mapping constrains $\hat{x}$ to lie in a subspace, and thus provides a form of regularization.

Dealing with nonlinear data

Here all we have done is still linear, PCA is a linear dimensionality reduction method (it just gives projections of the data). How can we achieve nonlinear dimensionality reduction

**Kernel PCA**

We will use the same intuition as in Supervised kernel methods, we will use a map to high dimensional space

$$x_i \rightarrow \varphi(x_i)$$

And then perform PCA on it.

Our goal will again be to replace dot products with kernel values

$$k(x_i, x_j) = \varphi(x_i)^T \varphi(x_j)$$

Let us first assume that the data is centred in feature space. That is

$$\frac{1}{N} \sum_{i=1}^N \varphi(x_i) = 0$$

Then, the covariance matrix can be written as

$$C = \frac{1}{N} \sum_{i=1}^N \varphi(x_i) \varphi(x_i)^T$$

Note that this is an $F \times F$ matrix, not an $N \times N$ one, so we still do not have kernel values here.

Applying PCA in feature space means that we need to solve the eigenvalue problem

$$Cw_{(1)} = \lambda_1 w_{(1)}$$

Where $w_{(1)}$ now is an F -dimensional vector. From the previous definition of C , this can be written as

$$\left(\frac{1}{N} \sum_{i=1}^N \varphi(x_i) \varphi(x_i)^T \right) w_{(1)} = \lambda_1 w_{(1)}$$

Recall from the Representer theorem, $w_{(1)}$ can be expressed as a linear combination of the $\{\varphi(x_i)\}$ i.e.,

$$w_{(1)} = \sum_{j=1}^N a_j \varphi(x_j)$$

Let us then substitute this alternative representation of $w_{(1)}$ in our eigenvector equation. This yields

$$\left(\frac{1}{N} \sum_{i=1}^N \varphi(x_i) \varphi(x_i)^T \right) \sum_{j=1}^N a_j \varphi(x_j) = \lambda_1 \sum_{j=1}^N a_j \varphi(x_j)$$

After some computations (skipped here), we can obtain a new eigenvalue problem

$$Ka = \lambda_1 Na$$

Where $a \in \mathbb{R}^N$ is the vector containing the new parameters of the problem, and K is the kernel matrix.

Solving this problem gives us a vector a , from which we could, in theory, obtain $w_{(1)}$ as

$$w_{(1)} = \sum_{i=1}^N a_i \varphi(x_i)$$

Note, However, that it depends on $\{\varphi(x_i)\}$ which are unknown.

As in kernel ridge regression, this can be addressed by noting that, ultimately, we are not interested in $w_{(1)}$, but rather in obtaining the low-dimensional projections $\{y_i\}$.

For 1D representation, these projections can be computed as

$$y_i = \varphi(x_i)^T w_{(1)} = \sum_{j=1}^N a_j \varphi(x_i) \varphi(x_j)^T = \sum_{j=1}^N a_j k(x_i, x_j)$$

Which depends on the kernel function k . As in the linear case, multiple output dimensions can be obtained by taking the d eigenvectors with largest eigenvalues

Non centred data

So far, we have assumed that the data was centred in feature space. There is however no reason for this to be satisfied

- Even if the original data is centred, the data after the mapping to feature space might not be

To handle more realistic scenario, where it isn't, let us define

$$\tilde{\varphi}(x_i) = \varphi(x_i) - \frac{1}{N} \sum_{j=1}^N \varphi(x_j)$$

The $(i, j)^{\text{th}}$ elements of the corresponding kernel matrix is then given by

$$\tilde{K}_{i,j} = (\tilde{\varphi}(x_i))^T \tilde{\varphi}(x_j)$$

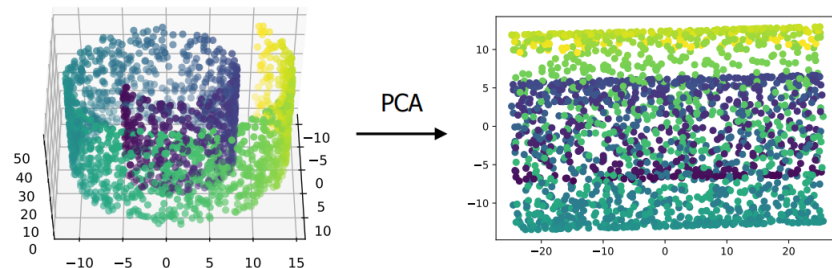
This kernel matrix can in fact be computed based on the original one as

$$\tilde{K} = K - 1_N K - K 1_N + 1_N K 1_N$$

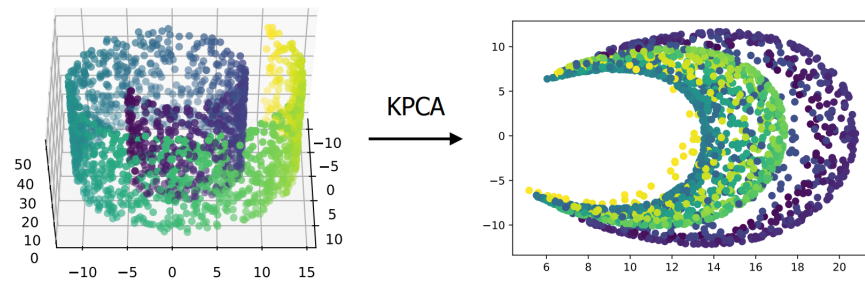
Where 1_N is an $N \times N$ matrix with every element equal to $\frac{1}{N}$

Example

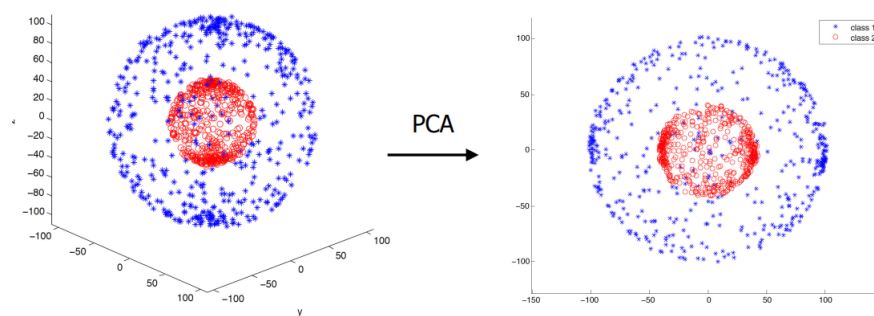
Here are a couple of examples on how that look like: Swiss roll: PCA results



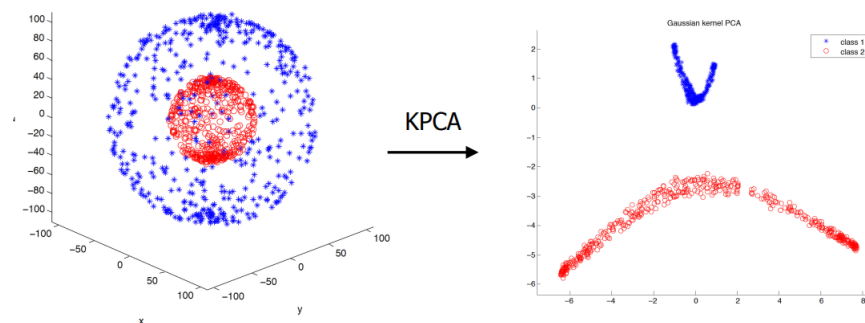
Using a RBF kernel now



3D inputs, 2 classes: PCA resulting



Using the same data but with a RBF kernel



Kernel PCA vs PCA

Here we can see that RBF kernel are pretty good here, but why?

As PCA, Kernel PCA defines a mapping from the high-dimensional data to the low dimensional one. However, in contrast to PCA, Kernel PCA does not provides a mapping from the low-dimensional space to the high-dimensional one.

- We cannot simply compute the high-dimensional version of any point in the low dimensional space

Can we nonetheless have this inverse mapping with nonlinear methods? The solution is to rely on deep networks

Another look at PCA provides a mapping from high to low dimension and back

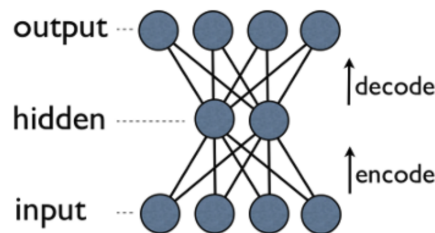
$$x \rightarrow y = W^T(x - \bar{x}) \rightarrow \hat{x} = \bar{x} + Wy$$

Both mappings are linear, so PCA cannot handle nonlinear $\rightarrow$ linear $\rightarrow$ non linear

In order to use deep networks, we will introduce **autoencoder**

3.1.1 Autoencoder

We have used linear mapping before but there is no reason to limit ourselves to linear, therefore following neural networks, the simplest extension consists of using a nonlinear activation function in the middle. This is referred to as an **autoencoder**.



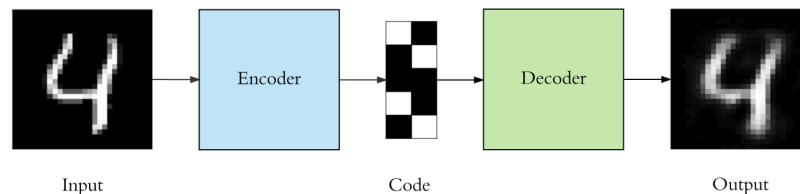
$$\hat{x} = Wf(W^T x)$$

with f being a nonlinear, element wise activation function.

General form

In general, an autoencoder can be represented by two mappings

- The encoder going from the input to a latent representation
- The decoder going from the latent representation to the output



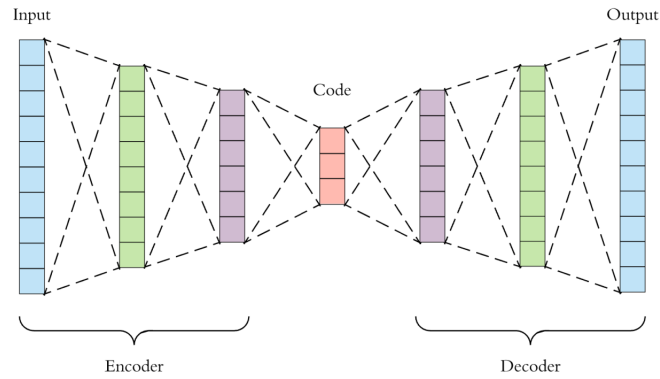
As in the PCA case, the output of the decoder is meant to be similar to the original input

Deep autoencoder

And as previous, there is no reason to restrict the encoder and decoder to consist of a single layer, we can then create deep autoencoders by stacking multiple layers each with an activation function.

Each layer gives a mapping of the form

$$z_{(j)} = f_{(j)}\left(W_{(j)}^T z_{(j-1)}\right)$$

**Training**

As mentioned before, with PCA, the reconstructed inputs $\{\hat{x}_i\}$ minimize the least square error

$$e_i = \|\hat{x}_i - x_i\|^2$$

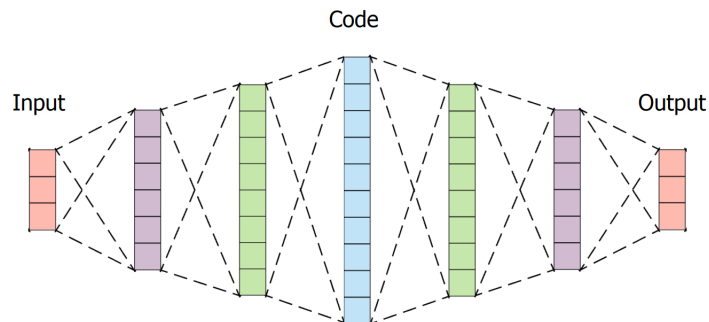
following this intuition, an autoencoder can be trained by minimizing the empirical risk

$$R(\{W_{(j)}\}) = \frac{1}{N} \sum_{i=1}^N \|\hat{x}_i(\{W_{(j)}\}) - x_i\|^2$$

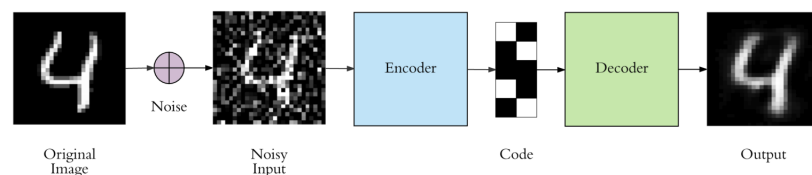
As for MLPs, this is non convex and the minimization process is done via stochastic gradient descent.

Dimensionality expansion

Because the mapping is nonlinear, there is no reason to restrict (again...) the latent representation to be low dimensional. One can then perform dimensionality expansion

**Denoising autoencoders**

Having a low-dimensional latent representation encourages the autoencoder to learn an '*intelligent*' mapping. With dimensionality expansion, one could simply learn to copy the input. To avoid this, one can add noise to the input and aim to reconstruct a noise-free version of the input.

**Sparse autoencoder**

Another approach to learn a meaningful latent representation consists of adding a regularizer.

- Let y_i denote the latent representation (or code) for sample i
- Then, one can encourage a large number of the elements $\{y_i^{(k)}\}$ to go to zero

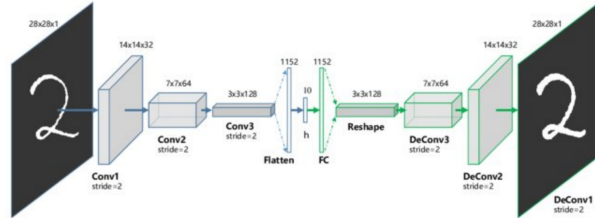
- This can be achieved by a sparsity-including regularizer, such as the L_1 norm

$$E_r(\{W_{(j)}\}) = \sum_{i=1}^N \sum_{k=1}^M |y_i^{(k)}|$$

With M dimensionality of the latent representation

Convolutional autoencoder

With convolutions and upsampling-convolutions, or transposed convolutions, one can also design convolutional autoencoders. Note that fully connected layers can also be used in the middle

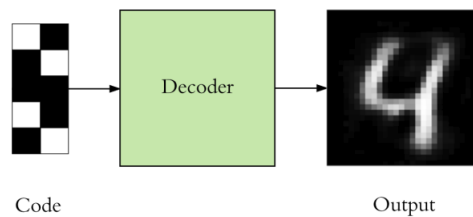


Autoencoder: Application

A code provides a compact representation of the input. It can be used for retrieval in a large data collection, e.g., via k nearest neighbours.

Mapping

As in PCA, the decoder provides a mapping from the latent space to the output (same as input) space. One can thus generate new samples (e.g., images) by taking arbitrary points in the latent space



3.2 Clustering

The goal of this section is to introduce another unsupervised learning task: **clustering**. We will also introduce K-means clustering and the Fisher Linear Discriminant Analysis, which combines notions of clustering and of dimensionality reduction.

3.2.1 Clustering: Another unsupervised learning task

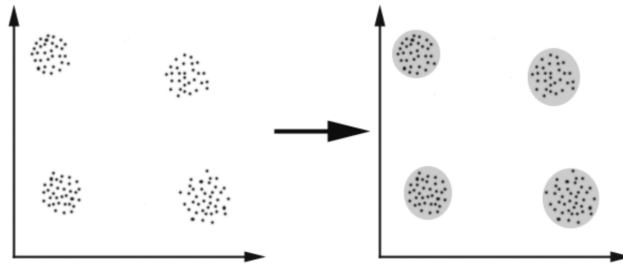
The goal of this task is: given some input data, we want to answer the question

'can we identify groups, without performing any data transformation?'

This operation is called **clustering**.

A simple clustering algorithm Let us now define the K -means clustering:

- Given a set of input samples, group the samples into K clusters
- K is a hyper-parameter, assumed to be known/given
- In the toy example below, each data sample is a point in 2D.



The goal is therefore to have the distance between the points **within** a cluster being small and the distance **across** cluster being large. The distance between clusters can be encoded via distances to cluster centers $\{\mu_1, \dots, \mu_k\}$

Algorithm

The algorithm in order to compute the K clusters is described below:

1. Initialize $\{\mu_1, \dots, \mu_k\}$ (e.g., randomly)
2. While not converged
 - (a) Assign each point x_i to the nearest center μ_k
 - For each point x_i , compute the Euclidean distance to every center $\{\mu_1, \dots, \mu_k\}$
 - Find the smallest distance
 - The point is said to be assigned to the corresponding cluster (not that each point is assigned to a single cluster)
 - (b) Update each center μ_k based on the points assigned to it
 - Recompute each center μ_k as the mean of the points that were assigned to it

More details

In the second step, as the algorithm iteratively updates the cluster assignment for each point and the clusters centers, we need to eventually stop iterating. This could be done using an hyper-parameter, we give as input to our algorithm a fixed number of iterations. However this is not ideal as it can lead to either a too big number of iterations or too small which would lead to bad results. A way to know whether or not we are far from the '*local optimal solution*' is to check the difference between the previous center location and the current. We could set a δ such that while the distance between the k and the $k - 1$ iterations is $> \delta$ we continue. However to do so we need to be sure that the algorithm is guaranteed to converge to a stable solution, here it is the case.

Remark 18. An important thing to know is that the algorithm will always converges, but it doesn't always converge to the best (desired) solution (this is why I put the optimal solution inside " before).

K-means clustering: Properties In conclusion we have that:

- **Pros:** It is a simple algorithm that is always guaranteed to converge
- **Cons:** It does not always converge to the best solution

In order to counter attack this cons above, we could run the algorithm several time with different initialization and,

- For each solution, sum the distances of each point to its assigned cluster
- Choose the solution with the smallest sum

To continues our pros/cons:

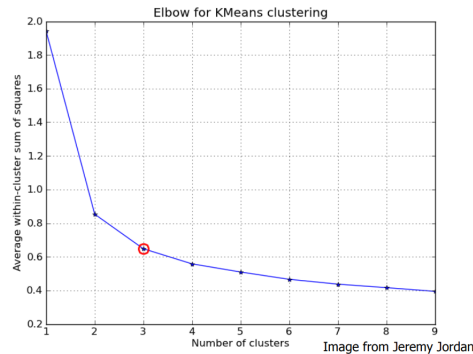
- **Pros:** Returns exactly K clusters
- **Cons:** Some clusters might be empty (no samples assigned)
- **Cons:** Requires the user to determine K

Remark	And as this is unsupervised learning, cross-validation is not an option
--------	---

Elbow method

But here we can use the same technique as above (in order to determine K),

- Run the algorithm with different values of K
- For each solution, sum the distances of each point to its assigned cluster
- Plot the resulting curve



And as you could have guessed the average within-cluster distance typically decreases towards zero.

Other Properties

As we don't have any annotations, it requires human analysis to provide a semantic meaning to the clusters. However if annotations are available, they can be used to label each cluster, e.g., with a corresponding category label.

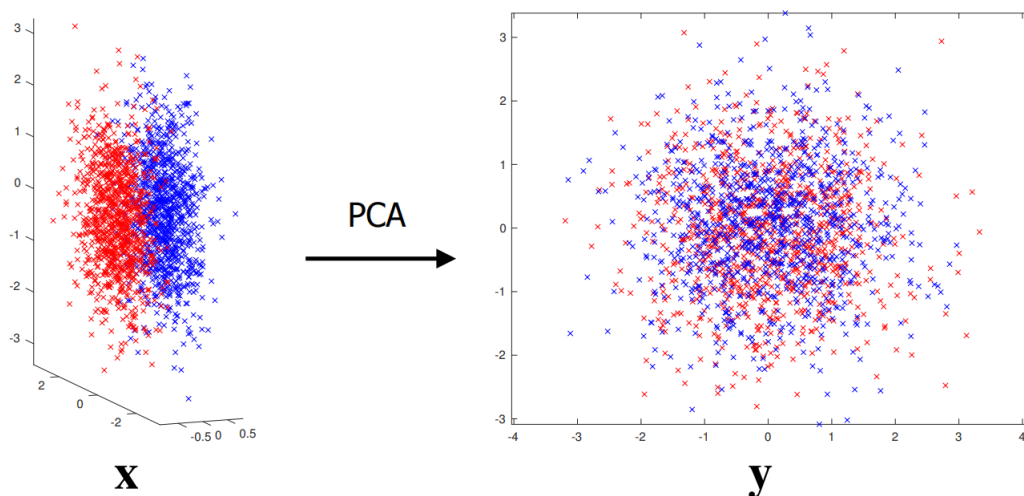
- For each cluster, find the number of samples of each category
- Assign the cluster category with the largest number of samples

Another property is the **euclidean distance**: this distance works well for data with homogeneous dimensions, meaning we don't have spike to some random dimensions. However in practice this is not always the case

- Different data dimensions can have completely different magnitudes
- They can encode completely different types of information

What this implies is that large values will dominate other part of the data point. A solution to this '*problem*' (this isn't really a problem in essence, it is maybe a property that we do want to have) can be solved using normalization and/or different distance function.

Clustering and dimensionality reduction The dimensionality reduction algorithms we saw last week were unsupervised and thus may not preserve category information. For instance



Remark 19. Categories are not known, when doing K -means nor when doing PCA. The goal or the hope of clustering as we have seen before was to 'guess' those categories by making clusters of data. So it would feel like supervised learning but it really isn't. When training/running our algorithm we don't have any ground truth label to refer to.

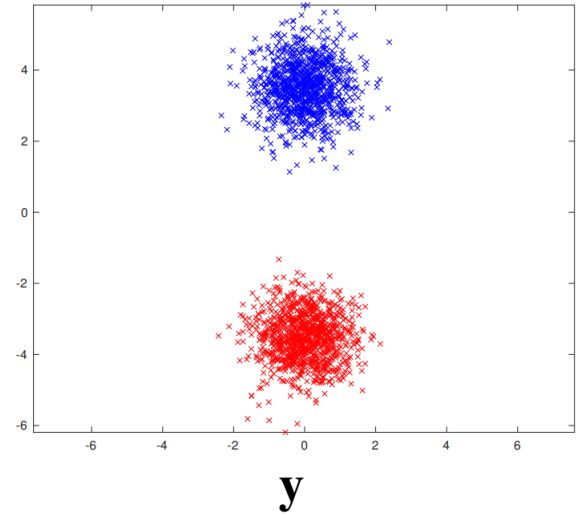
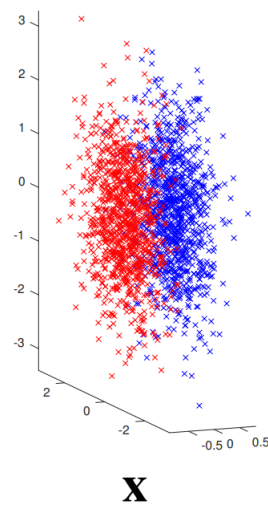
I feel like the image above is a bit misleading. When doing PCA, we wouldn't have any colour. Therefore the goal is to be able to find those cluster in the new dimension with the use of the class label.

What could have been done is to use the actual information that we are missing when doing the dimension reduction: the **label**. The goal would be to use those label during the dimensionality reduction to encourage clustering in the low-dimensional space.

Fisher Linear Discriminant Analysis (LDA)

As we have said before the goal that we would like to obtain is

- The samples from the same class to be clustered
- The different classes to be separated



Therefore now we are back to supervised learning (even though the supervision is not given for the $\{y_i\}$ directly but via the class labels).

Clustering the sample from the same class

Mathematically, this means that we want a low variance within each class after projection.

For a 1D projection, encoded via a vector $w_{(1)}$, and C classes, this can be expressed as aiming to minimize

$$E_W(w_{(1)}) = \sum_{c=1}^C \sum_{i \in c} (y_i - \nu_c)^2$$

Where ν_c is the mean of the samples in class c after projection, and $i \in \nu_c$ indicates that sample i belongs to class c .

| *Remark* Note that both y_i and ν_c depend on $W_{(1)}$

As in the PCA case, the variance after projection is equal to the projection of the covariance matrix. This lets us rewrite the previous objective function as

$$E_W(w_{(1)}) = w_{(1)}^T S_W w_{(1)}$$

Where

$$S_W = \sum_{c=1}^C \sum_{i \in c} (x_i - \mu_c)(x_i - \mu_c)^T$$

With μ_c the mean of the data in class c **before** projection. S_W is referred to as the **within** class scatter matrix.

Separating the different classes

In addition to clustering the samples according to the classes, we want to separate the different clusters. This can be achieved by pushing the means of the clusters away from each other. Mathematically, this can be expressed as maximizing

$$W_B(w_{(1)}) = \sum_{c=1}^C N_c (\nu_c - \bar{y})^2$$

Where ν_c is defined as before, $\bar{y}$ is the mean of all samples after projection, and N_c is the number of samples in class c . Again following the same reasoning as before, this can be re-written as

$$E_B(w_{(1)}) = w_{(1)}^T S_B w_{(1)}$$

Where

$$S_B = \sum_{c=1}^C N_c (\mu_c - \bar{x})(\mu_c - \bar{x})^T$$

With $\bar{x}$ the mean of all samples and $\{\mu_c\}$ the class specific means.

S_B is referred to as the **between-class** scatter matrix

Fisher LDA

Altogether, we want to

- minimize $E_W(w_{(1)})$
- maximize $E_B(w_{(1)})$

This can be achieved by maximizing

$$J(w_{(1)}) = \frac{E_B(w_{(1)})}{E_W(w_{(1)})} = \frac{w_{(1)}^T S_B w_{(1)}}{w_{(1)}^T S_W w_{(1)}}$$

And we can easily see that the previous objective function is invariant to scaling

$$J(\alpha w_{(1)}) = J(w_{(1)}) \quad \forall \alpha \in \mathbb{R}$$

We can therefore fix the scale by constraining $w_{(1)}$ to be such that

$$w_{(1)}^T S_W w_{(1)} = 1$$

This yields the Fisher LDA formulation

$$\begin{aligned} & \max_{w_{(1)}} w_{(1)}^T S_B w_{(1)} \\ & \text{s.t. } w_{(1)}^T S_W w_{(1)} = 1 \end{aligned}$$

And as before, we rely on the Lagrangian, written as

$$\mathcal{L}(w_{(1)}, \lambda_1) = w_{(1)}^T S_B w_{(1)} + \lambda_1 (1 - w_{(1)}^T S_W w_{(1)})$$

Zeroing out the gradient of $\mathcal{L}$ w.r.t. $w_{(1)}$ yields

$$S_B w_{(1)} = \lambda_1 S_W w_{(1)}$$

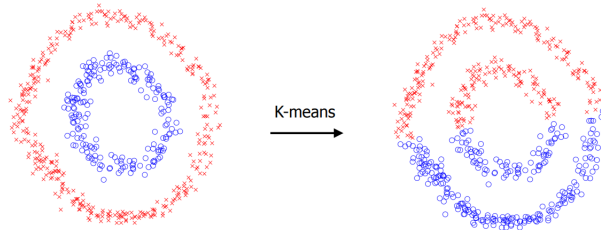
this implies that $w_{(1)}$ must be the solution to a generalized eigenvector problem.

<i>Remark</i>	By left multiplying both side by $w_{(1)}^T$ and dividing by $w_{(1)}^T S_W w_{(1)}$ tells us that $w_{(1)}$ should be eigenvector with largest eigenvalue
---------------	--

Dealing with $d > 1$ To project the data to more than a single dimension, we can follow a recursive strategy similar to the PCA one. Ultimately, this consists of taking the d eigenvector with the largest eigenvalues.

Note that it can be shown that S_B has rank at most $C - 1$ and therefore, we can project data only to at most $C - 1$ dimensions (as the remaining eigenvalues will all be 0 and thus carry no information)

Compactness K -means clustering (and Fisher LDA) exploit the notion of **compactness** of clusters. The question now is: what would happen if the data is not compact, for instance

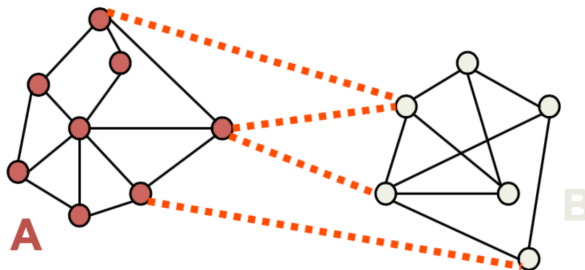


Even in this case, we are still able to clearly identify the two 'true' clusters.

But before that let us concentrate ourself on the image on the left. Here, the two clusters we observe arise from the connectivity of the points. We would like to have a method that works well for those type of relation.

Spectral clustering: Graph-based connectivity

The idea now is to group points based on edges in a graph, strong connections indicate points that should be clustered whereas weaker ones suggest that the graph can be cut into pieces/partitions



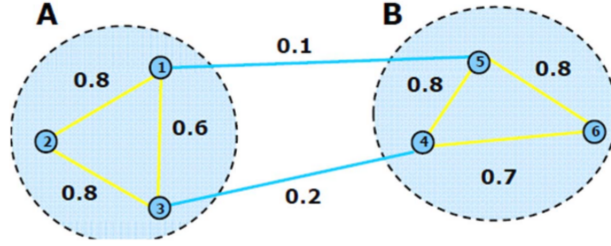
The question that we have now, is how do we create this graph in the first place. To do so we need to introduce the notion of similarity between the points. For instance we can take the definition below for given point i and j

$$W_{ij} = \exp\left(\frac{-\|x_i - x_j\|^2}{\sigma^2}\right)$$

Where σ is a hyper-parameter. And as you could have guessed, this would lead to a fully connected graph (an edge with a weight W_{ij} for every pair of points). The graph can also be restricted to the K -nearest neighbour of each point. Therefore a graph can equivalently be represented by a similarity matrix W .

Graph cut

Consider a partition of a graph into two parts A and B .



(vraiment rude le screen si j'ai la force je refais en svg)

A cost for cut is obtained by summing the weight of the edges that connect the two groups

$$\text{cut}(A, B) = \sum_{i \in A, j \in B} W_{ij}$$

In the above example, this would give us 0.3.

Intuitively, for clustering, we would like to find partition that minimizes such a cut.

Graph terminology

Definition 24. degree The degree of a node in the graph is given by

$$d_i = \sum_j W_{ij}$$

Definition 25. volume The volume of a set (partition) is given by

$$\text{vol}(A) = \sum_{i \in A} d_i$$

Remark

I will still need to check but, here we have that all edge within a graph are counted double whereas crossing edges are counted only once.

Normalized cut

Minimizing the cut might favor imbalanced partitions (e.g., a single point in A and all others in B) instead we could use a normalized cut

$$\text{Ncut}(A, B) = \frac{\text{cut}(A, B)}{\text{Vol}(A)} + \frac{\text{cut}(A, B)}{\text{Vol}(B)}$$

Transformed formulation

Finding the partition that minimizes the normalized cut is NP hard. A way to counter this is to re-write it as an optimization problem

$$\begin{aligned} \min_y & \frac{y^T (D - W) y}{y^T D y} \\ \text{s.t. } & y^T D \mathbf{1} = 0 \\ & y_i \in \{1, -1\}, \quad \forall 1 \leq i \leq N \end{aligned}$$

Where

- $W \in \mathbb{R}^{N \times N}$ is the similarity matrix between all pairs of points
- $D \in \mathbb{R}^{N \times N}$ is the diagonal matrix degree matrix, with $D_{ii} = d_i$
- y indicates to which partition each point belongs

However even with that, this remains an NP hard problem. But we are not done! We can again relax this problem as

$$\begin{aligned} \min_y & y^T (D - W)y \\ \text{s.t. } & y^T D y = 1 \end{aligned}$$

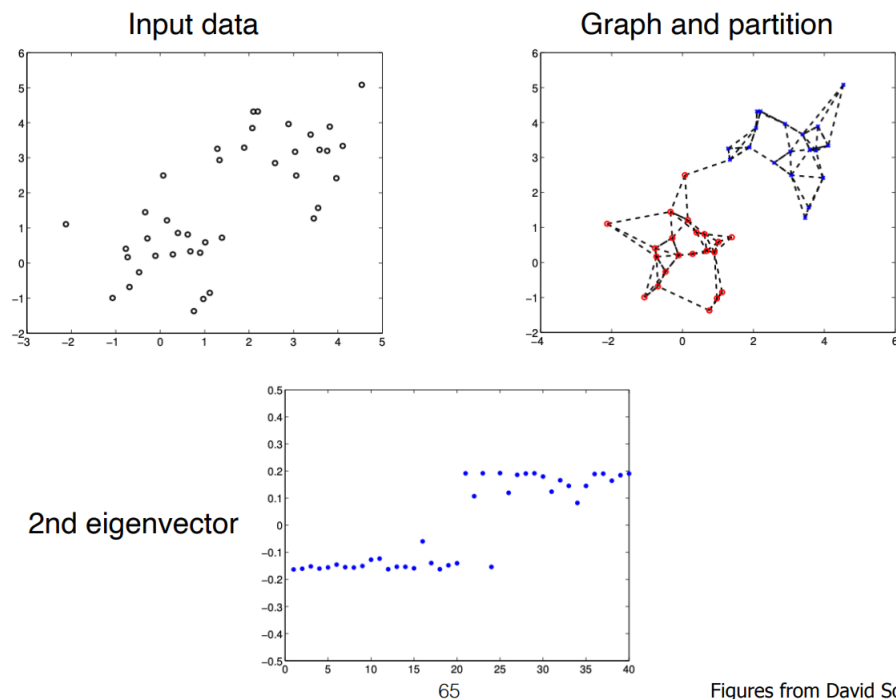
And using the Lagrangian, the solution can be expressed as a generalized eigenvalue problem

$$(D - W)y = \lambda D y$$

| *Note* $(D - W)$ is referred to as the graph Laplacian.

The question now is **which eigenvector**, the eigenvector with the smallest eigenvalue is a vector of all 1, its eigenvalue is 0. Therefore we cannot take this one, we need to take the second one.

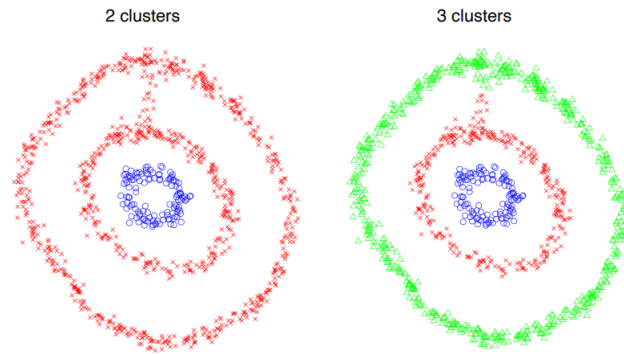
- ideally, a positive value in this vector indicates that the corresponding point belongs to one partition, and a negative value to the other
- Because of the relaxation, this is not so ideal; one then needs to threshold the values (e.g., by taking the median value as threshold for balanced data)



Figures from David Sontag's slides

***K*-way partition** To obtain more than 2 clusters, one can either

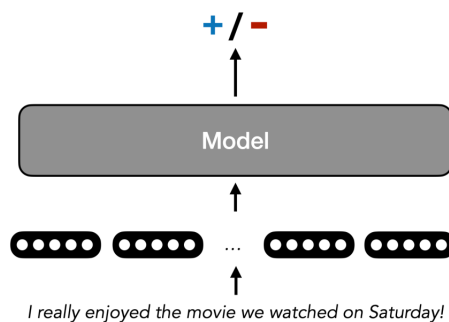
- Recursively apply the 2-way partitioning algorithm (Not efficient unstable)
- Use multiple (i.e., K) eigenvectors
 - Each point is represented as a K -dimensional vector
 - Apply K -means clustering to the resulting N vectors
 - Interpretation: Dimensionality reduction followed by K -means



3.3 Natural Language Processing (NLP)

The goal of this section is the study of deep neural networks model with a focus on text processing. We will introduce the problem of word representation and solutions to it. To be able to treat those, we will also introduce recurrent neural networks and transformers (this is what is used in general LLM today).

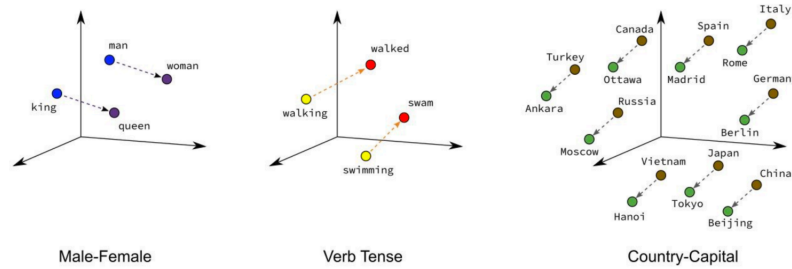
Heterogeneous data	As we have seen in the section about preprocessing of data. Samples of text can have different size, language etc... For supervised learning task (e.g., sentiment analysis), we assumed the input to always have the same size.
Bag of words	Recall on what we have seen before, a way to store data from a text was to actually store the frequency of each word in the text. There BoW encodes an entire document in a single vector, as this is fine for targeted ML task at the document level however this won't work for word-level tasks such as named entity recognition, translation, text generation.
Word representation	In neural natural language processing, we treat words as vectors .



A first naive approach to this would be to have a one hot encoding for all words in the vocabulary. We would first define a vocabulary V , then we will encode each word into a vector $x_i \in \{0, 1\}^{|V|}$. However here we have a big limitation, how do we compute 'similarity'? We don't have any information if x and y are similar. Let us take for instance the word *house* and *home*. Those would be pretty similar however in a sparse encoding as defined above this are as similar as *cow* and *pneumonoultramicroscopicsilicovolcanoconiosis*. We therefore need to have a sort of more complex way of representing our word as a vector. To do so we can introduce word embeddings

Here what we will do is to represent each word as a high-dimensional, real-valued vector (When we say high-dimensional, it is still pretty low in comparison to the V dimension sparse representation but still $O(10^2 - 10^3)$). In that case, similarity of vectors represents similarity of meaning for particular words.

Ultimately the goal would be to have a space as this one below



But the question that this brings up is how do we train semantics-encoding embeddings of words?

Training

In a supervised model, the training comes down to optimizing the embeddings for a specific task (e.g., sentiment analysis). However this may or may not generalize well. And as for unsupervised we also need something in order to learn or to **self supervised**. The solution to this

Rely on the context in which words occur to learn their meaning

The context in this case is the set of words that occur nearby, to do so there is some standard approaches such as Word2Vec or GloVe.

Individual words

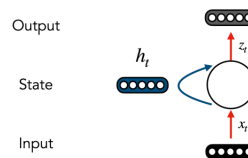
Here what we have done so far is to find a representation for each word, however this is not the best thing to do as not all words are indicative of the category, some words are even misleading. Therefore, instead of modelling words, the standard approach consists of modeling a complete sentence or a document as a sequence.

Remark

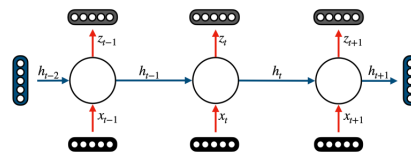
Note that each individual word is still modelled as a vector as discussed previously.

Recurrent neural networks

The solution to model each of this block is **recurrent neural networks**, what this essentially is, it is a neural network with a feedback loop.



By unrolling the RNN across all time steps we get a full computation graph



This allows learning from entire sequence history, regardless of length. Therefore what this is essentially doing is: We have a hidden state h_t which gets updated as each new word is processed. The update rule is typically

$$h_t = f(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

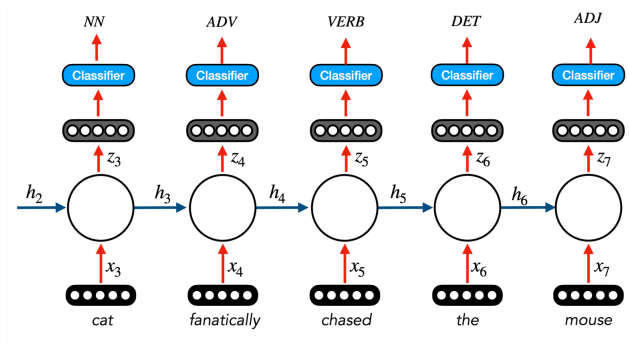
Where

- h_{t-1} is the previous hidden state (the 'memory' of everything seen so far)
- x_t is the current word vector
- W_h, W_x are learned weight matrices
- f is a non-linear activation

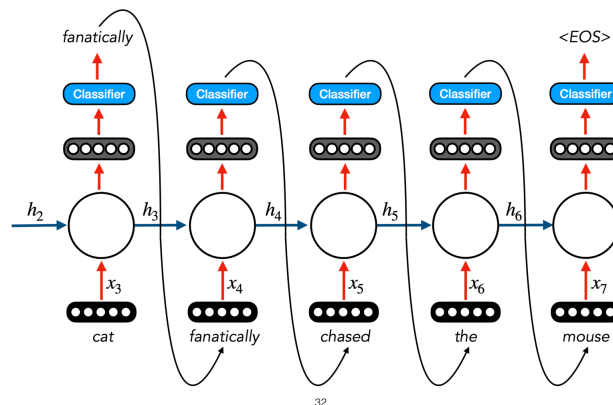
The important thing to understand here is that h_t acts as a **compressed summary of the entire sequence up to position t** .

RNN for classification

Remember how did we define classification before, it was just an output projection followed by a softmax. Therefore, all we need to do here is to take our RNN model, and after the last given output, we apply our classifier. For instance if our goal was to see whether each word is a noun, adverb, etc... Our model could look like this



Where as if we wanted to generate text, we would output a word at each step and use it as a new word onto the next NN:



Different types of RNNs

As we have seen before, the standard RNN definition is as follows

$$h_t = \sigma(W_{hx}x_t + W_{hh}h_{t-1} + b_h)$$

$$z_t = \sigma(W_{zh}h_t + b_z)$$

we also have other types such as

- Long Short-Term Memory (LSTM) - Hochreiter & Schmidhuber, 1997
- Gated Recurrent Unit (GRU) - Cho et al., 2014

Training an encoder-decoder model

Before knowing what encoder-decoder models are first, what are encoder and decoder models?

Encoder

The encoder reads the **input sequence** word by word, updating its hidden state at each step. Its job is to produce a representation of the input. In the basic version, you just take the **final hidden state h_T** as a summary of the whole input sentence. (this is the classical RNN)

Decoder

The decoder takes that representation and **generates the output sequence, one word at a time** (this is the RNN for generation)

The flow of this model

$$\underbrace{x_1, x_2, \dots, x_T}_{\text{encoder reads this}} \rightarrow \underbrace{h_T}_{\text{bottleneck}} \rightarrow \underbrace{y_1, y_2, \dots, y_{T'}}_{\text{decoder generates this}}$$

As we already know, the goal is to minimize the average cross-entropy over all tokens:

$$\mathcal{L} = \sum_{t=1}^T -\log(P_{y_t|y_1, \dots, y_{t-1}, x_1, \dots, x_n})$$

What this loss is asking is 'at each step t , how surprised were we by the correct next token y_t given everything before it?'. So for each position t , the decoder outputs a probability distribution over the whole vocabulary, and you just check how much probability mass was assigned to the correct word y_t . To do so we backpropagate gradients through both the decoder and the encoder.

Limitation

'you can't cram the meaning of a whole %@&ing sentence into a single \$*(@ing vector!)' – Ray Mooney (NLP Prof. at UT Austin)

However turning a whole sentence into a single vector leads to a massive compression of information. This challenges to learn long-range dependencies/interactions. The reason on this is the same as we have already talked about previous section, when dealing with long series of network (meaning gradients of gradients of gradients... Often the first time steps (the first words) get exponentially converging to 0, this implies that early words are 'forgotten'), LSTMs were the first major attempts to address this. We introduce a cell state c_t (a separate memory track) alongside the hidden state h_t , controlled by three gates:

- Forget gate f_t : how much of the previous cell state to erase
- Input gate i_t : how much of the new input to write into memory
- Output gate o_t : how much of the cell state to expose as h_t

Each gate here is a sigmoid, acting like a soft on/off switch. This lets the networks selectively remember or forget over long sequences, which helps but doesn't fully solve the compression problem.

Incorporating attention

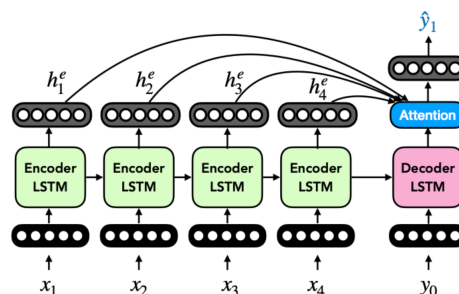
As seen before at each encoder time step, there is an output of the RNN. The idea here is to use the output of the decoder LSTM to compute an **attention** (i.e., mixture) over all the h_t^e outputs of the encoder LSTM.

Intuition

Instead of forcing everything into a single final vector h_T , **attention** keeps **all** the hidden state $h_1, \dots, h_T$ and at each step computes a weighted mixture

$$c_t = \sum_i \alpha_{it} \cdot h_i$$

Where each a_{ti} are learned weights representing 'how relevant is word i for the current step?'. This way the model can directly reach back to any earlier state rather than relying on it surviving compression.



Attention function

The goal of this function is to compute pairwise similarity between each encoder hidden state and decoder hidden state ('idea of what to decode'). We define

h_t^e = encoder output hidden states

h_t^d = decoder output hidden state

Note We also call h_t^e the keys and h_t^d the query
Therefore we define

$$a_1 = f(h_1^e, h_1^d), \quad a_2 = f(h_2^e, h_1^d)$$

Here we are asking 'how relevant is encoder position i for what I'm currently trying to decode?'. As you could have guessed there is a lot of attention function f out there here are three:

Multiplicative

$$a = h^e W h^d$$

Linear

$$a = v^T \varphi(W[h^e; h^d])$$

Scaled dot product

$$a = \frac{(W h^e)^T (U h^d)}{\sqrt{d}}$$

This convert the pairwise similarity scores to a probability distribution (using softmax) over encoder hidden states and compute weighted average. This mean that we have all a_i define as

$$a_i = f(h_i^e, h_1^d)$$

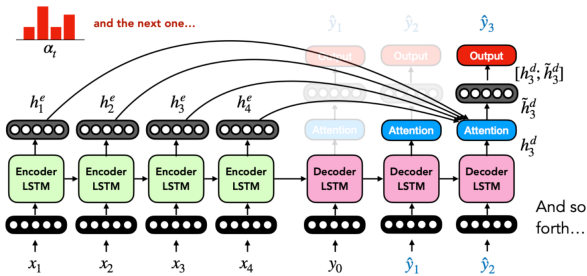
then we normalize those scores into a proper distribution

$$\alpha_t = \frac{e^{a_t}}{\sum_j e^{a_j}}$$

Therefore $a_i \in [0, 1]$ and they sum to 1. We can then compute the weighted average

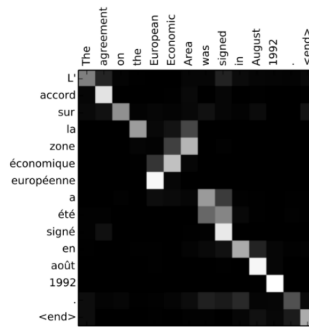
$$\tilde{h}_1^d = \sum_{t=1}^T \alpha_t h_t^e$$

This is the mixture that we talked about before.



Interpretability?

As said before attention can be visualised based on the score given to each encoder hidden state. But what is focused on when each word is generated? Training with attention gives us implicit alignment for free! Our weights a_t form a matrix which mean that we can therefore plot as a heatmap where α_{ti} tells us how much decoder step t attends to encoder position i .



So here when we are trying to translate English $\rightarrow$ French, we would typically see a near diagonal pattern for words that translate in similar order. Off-diagonal jumps for reorderings.

Problem: Encoder is recurrent

There is a big performance issue here, recurrent function can't be parallelized because previous state needs to be computed to encode next one. Encoder hidden states must still be computed in series. The solution to this are transformers.

3.3.1 Transformer

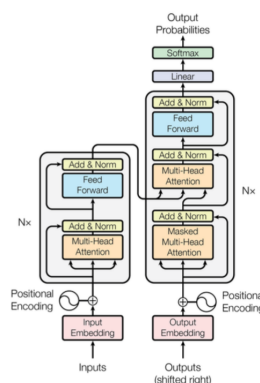
Transformers are made up of encoder and decoder, where both encoder and decoder are made up of multiple cascaded transformers blocks. (there is a slightly different architecture in encoder and decoder transformer blocks). Blocks are generally made up of **multi-headed attention** layers (self attention) and **feedforward** layers. What this implies is that we don't have recurrent computations anymore. Multiple cascaded blocks are important here, they are analogous to how deep CNNs learn. Early block might learn syntactic relationship whereas deeper block learn more semantic/abstract ones.

Transformers vs RNNs

RNNs process sequence step by step, word 1, then word 2, then word 3. The only way word 1 can influence word 5 is by surviving through the hidden state chain. Transformers **abandon recurrence entirely** and instead let every word talk to every other word simultaneously. More concretely let us take a transformer block. Instead of a hidden state that gets updated, a transformer block takes a **whole sequence of vectors** and outputs a **whole sequence of vectors**

$$[x_1, \dots, x_T] \xrightarrow{\text{transformer block}} [z_1, \dots, z_T]$$

Each output z_i is an enriched representation of word i , informed by all other words via self-attention.



Self-attention

In the RNN attention we saw earlier, attention as between two different sequences, the encoder states and the decoder states. **Self-attention** is attention where the sequence attends to itself, every position looks at every other position in the **same** sequence to build a better representation.

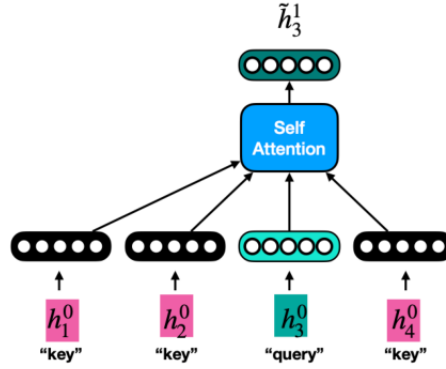
How it works

For each position i in the sequence, you create three vector from the word embedding

- Query q_i : what am I looking for?
- Key k_j : what do I contain?
- Value v_j : what do I actually pass along?

Then the attention score between position i and j is

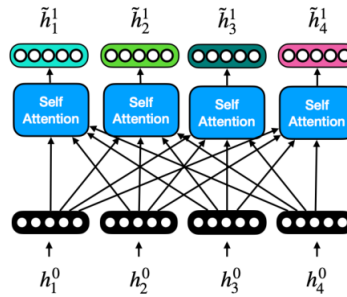
$$a_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$



We have that for each attention computation, every element is a key and value, and one element is a query. The Q, K, V vectors are obtained by learned linear projections

$$q = W^Q h_s^e \quad K = W^K \{h_t^e\} \quad V = W^V \{h_t^e\}$$

Then we do this computation for each $\tilde{h}_i^1$



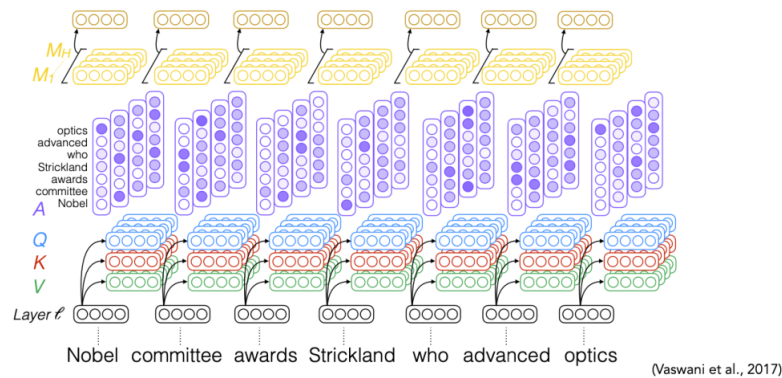
Where each $\tilde{h}_i^1 = \text{Attention}\left(h_i^0, \{h_t^0\}_{t=0}^{t=3}\right)$

Therefore the full computation is defined as

$$a = \frac{(W^Q q)(W^K K)}{\sqrt{d}} \xrightarrow{\text{softmax}} \alpha \xrightarrow{\text{weighted sum}} \tilde{h}^e = W^O \alpha (V W^V)$$

Example

Let us now take the diagram below to understand the whole picture.



Bottom layer l The small white boxes at the bottom are the current hidden state h_i^l for each word, this is the input to this transformer block.

Q, K, V projections For **every word simultaneously**, you compute the three projections:

- Q (blue): queries
- K (red): keys
- V (green): values

The arrow show that every word connects to every other word. This is a pairwise self-attention computation.

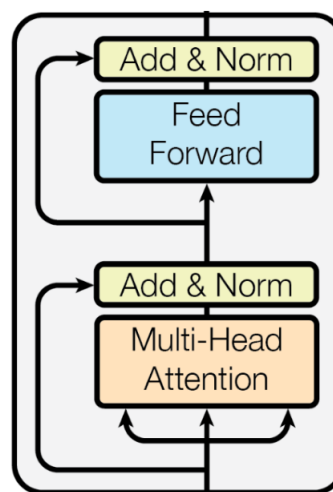
Attention outputs This is the thing in purple vectors, those are the outputs after computing α and the weighted sum of values. Notice that **varying shading** in the purple vectors means higher activation, showing how different word contributed to the current output.

Multi-head $M_1, \dots, M_H$ This is the main idea, instead of computing attention **once**, we do it H times in parallel, each with different W^Q, W^K, W^V matrices. Each head can learn to attend to **different types of relationship** (syntactic, coreference, ...). Then the output of all heads are then **concatenated** and **projected** back to the original dimension.

But why do we even need multi-head? A single attention head produces one weighted mixture, it's forced to compress all relationship into one distribution. Multiple heads let the model capture **several different relationship types simultaneously**, which is much more expressive.

The Multi-headed attention is the main innovation of the transformer model

Transformer block



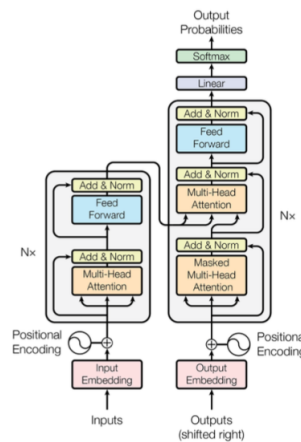
Each block here is also composed of:

- A layer of normalizations
- A feedforward network
- residual connections

Full transformer In order to have a full transformer encoder, all we need to do is to put multiple cascaded transformer block together which we'll **build up** compositional representation of inputs.

On the other hand, when looking for the decoder (right) this is similar to encoder where:

- First layer of block is **masked** multi-headed attention
- Second layer is multi-headed attention over final layer encoder outputs (**cross attention**)
- Third layer is feed-forward network



Masked multi-head self attention

As we have seen, self-attention can attend to any token in the sequence. However for the decoder, we don't want tokens to attend to future tokens, if the decoder is used to generate text (i.e., machine translation). As during training we have full target sequence, this is different here right. During generation, we can only use words we have already produced. The constraint is: when predicting a word t , we should only be allowed to look at words $1, \dots, t-1$ and **never** $t+1, t+2, \dots$

Concrete example

Say the target is 'Novel committee awards Strickland'. When predicting 'awards':

- We can attend to 'nobel' and 'committee'
- We **cannot** attend to 'Strickland', it hasn't been generated yet

If the model could peek at future words during training, it would just cheat, learning nothing useful, since at test time, those future words don't exist yet.

How masking works is pretty simple, we simply set the attention scores for future position to $-\infty$ before the softmax function:

$$a_{ij} = \begin{cases} \frac{q_i k_j}{\sqrt{d}} & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

Which implies when we apply to softmax

$$\alpha_{st} = \frac{e^{a_{st}}}{\sum_j e^{a_{sj}}} = 0$$

when $s < t$. Therefore, our attention matrix is a lower triangle matrices.

Remark 20. A question I asked Claude which was (after the 100th ones) when we are training, we are essentially training models of models, also if we don't know the length of the sentence, how do we train a model here without knowing its size?

Answer You're training one single model — not a model of models. The encoder and decoder are just two components of the same network sharing a training objective. The loss is still the same cross-entropy:

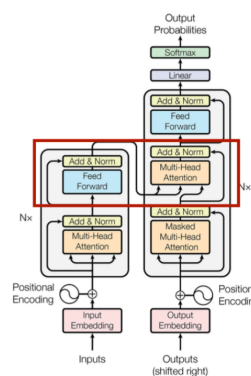
$$\mathcal{L} = \sum_{t=1}^T -\log P(y_t | y_1, \dots, y_{t-1}, x_1, \dots, x_n)$$

You have paired examples (input sentence, target sentence), and you backpropagate gradients through both decoder and encoder simultaneously. Everything is just one big computation graph with shared weights being updated together.

Variable length sequences We have already kind of seen it in RNN but here the issue is that we need to train our model before knowing the size of the input. To do so when we have to shorten sentence in comparisons to the model input size we add padding using special token (e.g. `[PAD]`) make those positions ignored (set to $-\infty$ before the softmax). We can also put special token such as `[EOS]` at the end of the sentence (we have seen one in the previous diagram) in order to have a model that can even itself choose when to end its generation. Okay this is nice but what is the concrete difference between me sending a 10 words sentence to Claude or a 10000 words sentence to Claude?

As transformer have a fixed maximum sequence length, which is called the **context window**. Therefore when having a 10 word sentence, it gets padded up to the max length, whereas if we have a sentence longer than the context window: it gets **truncated**, we simply lose the rest. The model never sees it. (this is the reason on why some model hallucinated or just repeat themselves after long discussion, the context window is full).

Cross attention **Cross attention** is the same as classical attention as in the RNN encoder-decoder. The difference here is that the attention function is the output of the masked multi-headed attention in the decoder (e.g., a decoder state). The keys and values are the output of the **final** encoder transformer. Once again, a representation from the decoder is used to **attend** to the encoder outputs.



To resume this a bit better, the difference with self-attention is

- The **Query** $Q \rightarrow$ comes from the **decoder** (masked self-attention output) and asks 'what am I trying to generate?'
- The **Key** K and **Value** $V \rightarrow$ comes from the **final encoder layer** and asks the question "what was in the input?"

So unlike self-attention where everything came from the same sequence, cross attention **mixes two difference sequences**

$$\underbrace{\text{decoder state}}_{\text{Query}} \rightarrow \text{cross attention} \leftarrow \underbrace{\text{final encoder output}}_{\text{Keys \& Values}}$$

At each decoder step, the decoder asks: "given what I want to generate, which part of the input sentence should I focus on?"

Now the question on why we only "care" about the last final layer even though we have already computed all previous one is that the final encoder layer contains the **most enriched representations**, each token has already been

Position Embeddings

The issue we have with what as been said before about context attention etc. is that for instance, shuffling tokens doesn't change the result of the attention. This is not really what we want right? For instance the sentence 'I eat apple and not pear because I hate it' and the sentence 'I eat pear and not apple because I hate it' we'll give us the same output. Therefore for each token, it would be useful to add to it information about the **position**, to embeds positions (idk if that's a saying). To do so we will use a sinusoid function that is offset by a phase shift proportional to the sequence position. The positional encoding for the index i would be given as

$$p_i = \begin{pmatrix} \sin\left(\frac{1}{10000^{2 \cdot \frac{1}{d}}}\right) \\ \cos\left(\frac{i}{10000^{2 \cdot \frac{1}{d}}}\right) \\ \vdots \\ \sin\left(\frac{i}{10000^{2 \cdot \frac{d}{2d}}}\right) \\ \cos\left(\frac{i}{10000^{2 \cdot \frac{d}{2d}}}\right) \end{pmatrix}$$

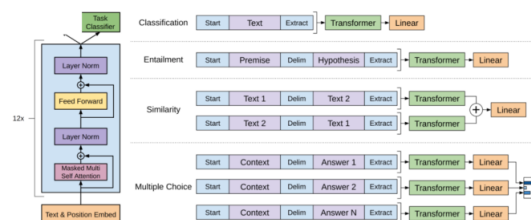
GPT: Generative Pretrained Transformer Here we have two words giving hints about what this model should do, it is **generative**, meaning it generates texts and it is **pretrained** meaning it has been trained on a lot of data. Those are also called as a decoder transformer. However the actual GPT block mixes design of encoder and decoder from the original transformer. It also uses masked multi-headed self-attention (on the decoder) meaning it cannot see the future. We also have no cross attention; we only compute a self-attention over its history in each block (encoder).

Training

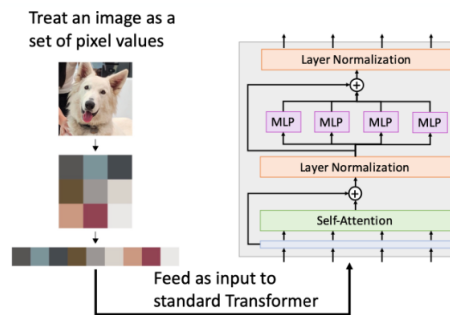
GPT are trained on an enormous amount of data, on the TorontoBooks corpus (7000 unpublished books (~ 13 GB)). Each corpus is segmented into windows of 512 tokens. The training is done by trying to predict the next word.

Fine tuning

After pre-training, model can be fine-tuned by training on individual datasets. Pretrained model are used as initialisation for training on individual tasks.



Transformers for images When trying to use transformers for images, the simplest way is to consider the pixels as words



But this is just an enormous amount of data, so instead let us try to use patches of images.

